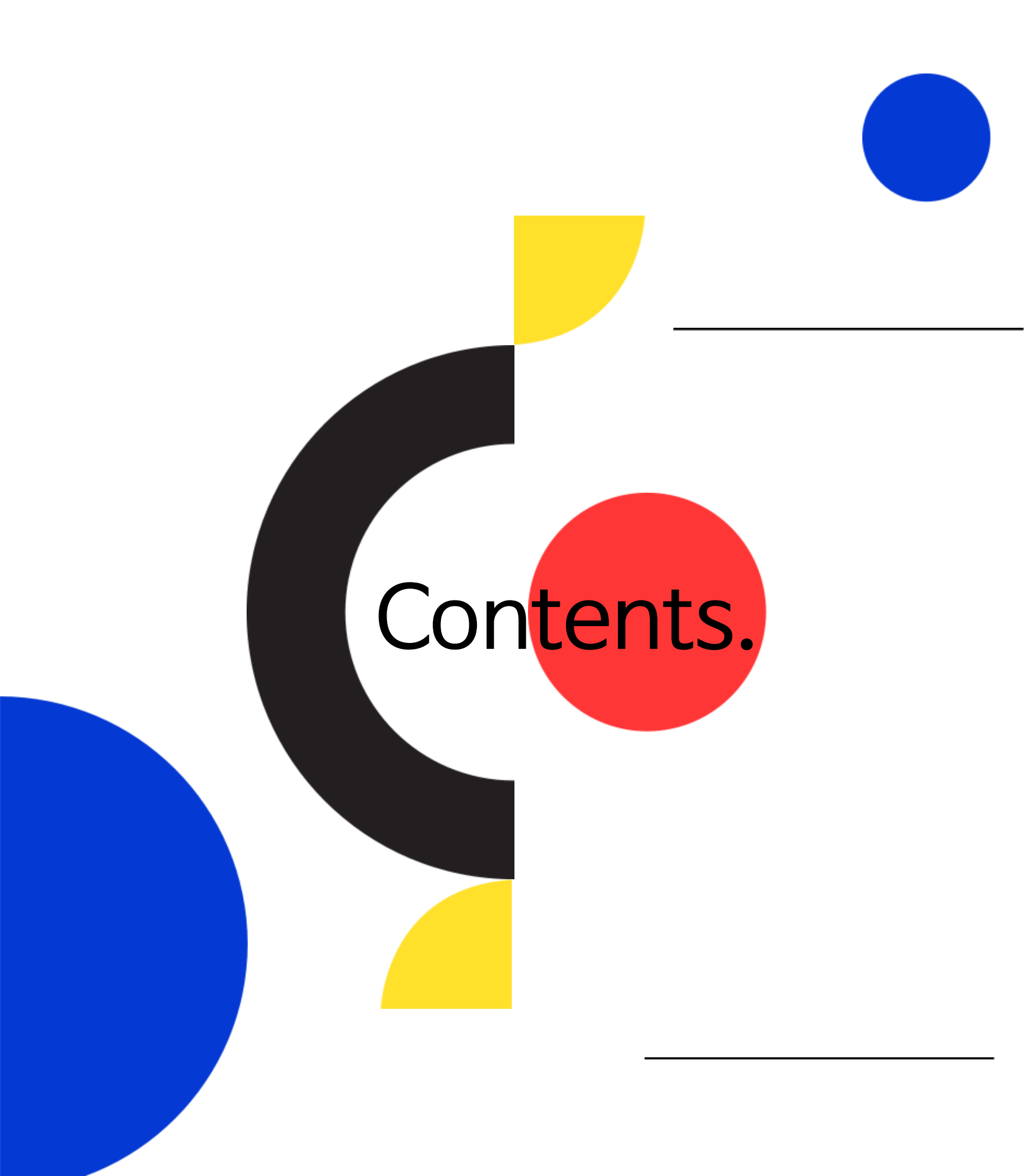




Analysis / Modeling

대구시 교통사고 데이터를 활용한 사고 위험 분석 및 예측 모델 개발



Contents.

01. 프로젝트 개요

- 프로젝트 선정 배경 및 목표
- 프로젝트 프로세스
- 개발 환경

03. 성능 지표 기반 전처리 기법 최적화

- 모델 성능 평가 점수 설계
- 데이터 셔플링 여부 및 분석 모델 선정
- 인코딩 방식 최적화
- 종속변수 변환 방법 및 손실함수 최적화

05. 피처 엔지니어링

- 연도 변환
- 시계열 변수 Sin/Cos 변환
- 시간적 변수 관련 파생변수 추가
- 공간적 변수 관련 파생변수 추가
- 환경적 변수 변형
- 통계기반 파생변수 추가

07. 최종 성능 평가

- 공식 평가 점수 측정 방식
- 모델 개선 단계별 성능 비교

02. 데이터 소개

- 데이터 출처 및 구조 소개
- 주요 변수 설명
- 데이터 기본 전처리

04. 데이터 현황 & 탐색적 분석

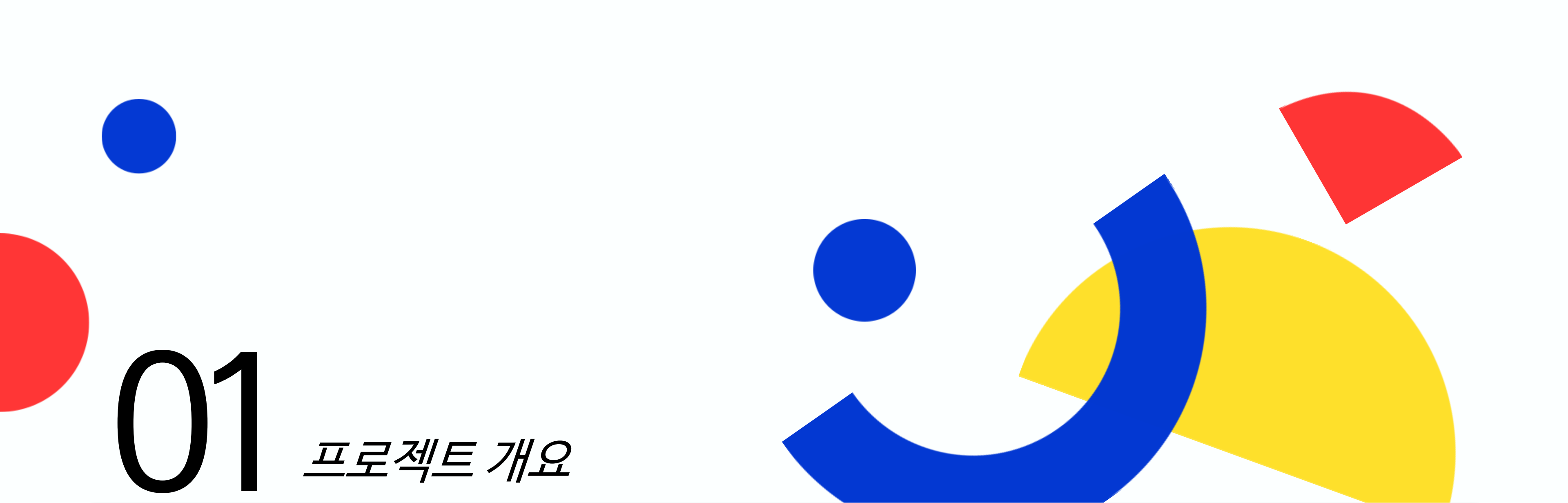
- 시간적 정보
- 공간적 정보
- 환경적 정보
- 훈련데이터에만 존재하는 변수

06. 모델 엔지니어링

- 모델 엔지니어링 개요 및 프로세스
- 하이퍼파라미터 최적화(HPO) 설계
- 일반화 성능 측정 및 최종 모델 선정

08. 결론 및 향후 개선 방향

- 최종 프로젝트 결론
- 향후 개선 방향
- 프로젝트 회고



01

프로젝트 개요

• 프로젝트 선정 배경 및 목표

• 프로젝트 프로세스

• 개발 환경

◆ 대구 교통사고 피해 예측 AI 경진대회

<https://dacon.io/competitions/official/236193/overview/description>



◆ 프로젝트 선정 배경

다양한 공공 데이터 중 데이터 분석 및 데이터 모델링에 대한 흥미를 바탕으로 DACON에서 운영하는 ‘대구 교통사고 피해 예측 AI 경진대회’를 참가를 결정했다.

본 대회는 교통사고 발생 일시와 장소에 따른 **사고 위험도(ECLO)**를 보다 정확히 예측하는 과제로서 교통사고 발생 당시의 기상상태, 도로 상태, 사고 위치 등 다양한 시/공간 및 환경적 정보가 포함된 데이터를 제공한다.

이를 바탕으로 다양한 **전처리** 및 **피처 엔지니어링 기법**을 적용하여 **머신 러닝** 및 **인공지능 모델** 학습을 위한 기반을 마련하고, 동시에 교통사고 발생 원인에 대한 **현황 분석** 및 **EDA 분석 기법**을 통해 데이터 모델링에 유의미한 특징을 도출하고자 하였다.

본 프로젝트는 이러한 과정을 통해 보다 정확한 예측 모델을 구축하는 역량을 향상시키는 데 목적을 두고 주제를 선정하였다.

◆ 프로젝트 목표

수집된 교통사고 정보 및 관련 데이터를 활용해 **사고 위험도(ECLO)**를 예측하는 **AI 알고리즘 발굴** 및 **최적화**를 목표로 한다.

ECLO(Equivalent Casualty Loss Only)란 **인명피해 심각도**를 나타내며, 계산식은 다음과 같다.

$$ECLO = \text{사망자수} \times 10 + \text{중상자수} \times 5 + \text{경상자수} \times 3 + \text{부상자수} \times 1$$

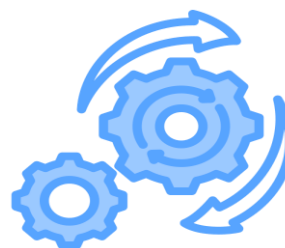
본 대회에서는 예측 성능 평가 지표로 **RMSLE**(Root Mean Squared Log Error)를 사용하며, 본 프로젝트는 해당 지표 기준에서의 예측 정확도를 향상시키는 것을 목표로 한다.

이를 위해 다양한 **AI/ML 알고리즘**을 적용하여 모델의 **일반화 성능**을 비교하고, 그 과정에서 데이터 전처리 전략의 최적화, 다변량 군집 분석, 피처 엔지니어링 등의 기법을 활용하였다.

이러한 분석을 통해 ECLO 예측에 유의미한 변수 및 구조를 도출하고, 이를 기반으로 **최적의 예측 성능을 갖춘 모델**을 구축하고자 한다.

프로젝트 프로세스

◆ 프로젝트 수행 기간 : 2025.05.01 ~ 2025.07.24 (85일)



01

데이터 수집 및 전처리

- 제공된 데이터 및 대구 관련 외부 데이터를 수집
- 모델 입력을 위한 데이터 전처리 및 정제 수행

02

현황 분석 및 탐색적 데이터 분석

- 교통사고 유발 요인 분석을 통한 인사이트 도출
- 공간 기반 다변량 군집분석을 통해 지역별 사고 위험도 분류

03

피처 엔지니어링

- 주요 사고 유발 요인을 반영한 피처 엔지니어링 및 통계 기반 파생 변수 생성

04

모델 엔지니어링

- Optuna를 활용한 하이퍼 파라미터 최적화를 수행해 최적 모델 선정

05

최종 모델 선정

- 선정된 최적 모델을 기반으로 대회에서 제공하는 테스트 데이터를 통한 최종 모델 성능 평가 수행

사용 언어



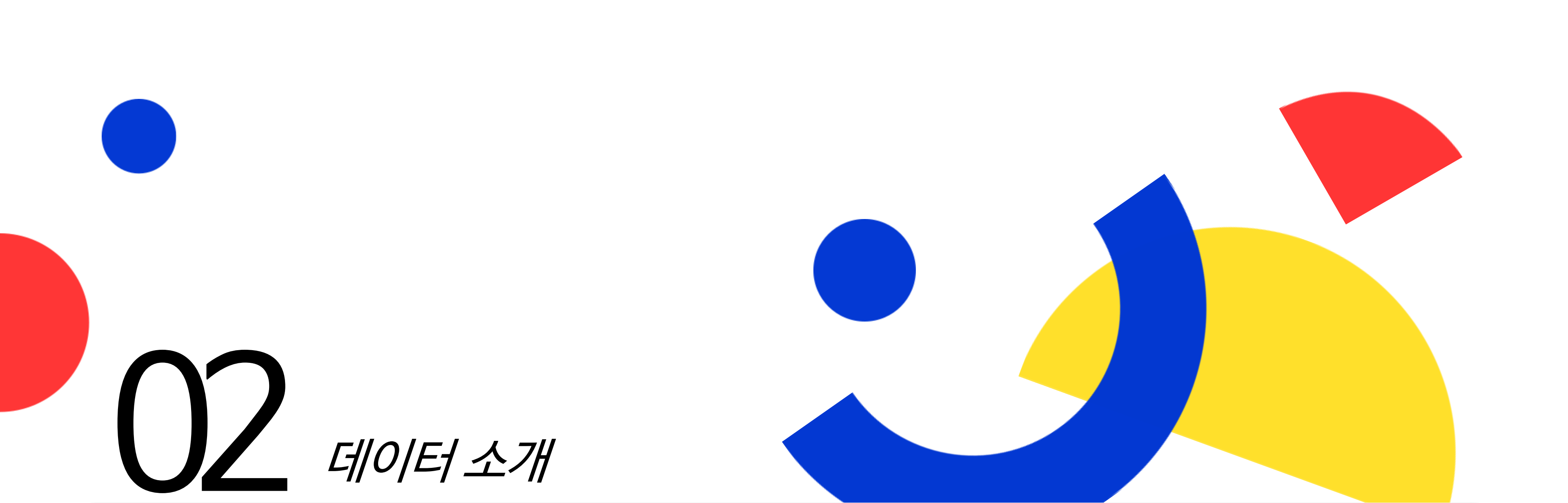
Development Environment

개발 환경



주요 라이브러리





02

데이터 소개

• 데이터 출처 및 구조 소개

• 주요 변수 설명

• 데이터 기본 전처리

❖ 데이터 출처 및 구조 소개

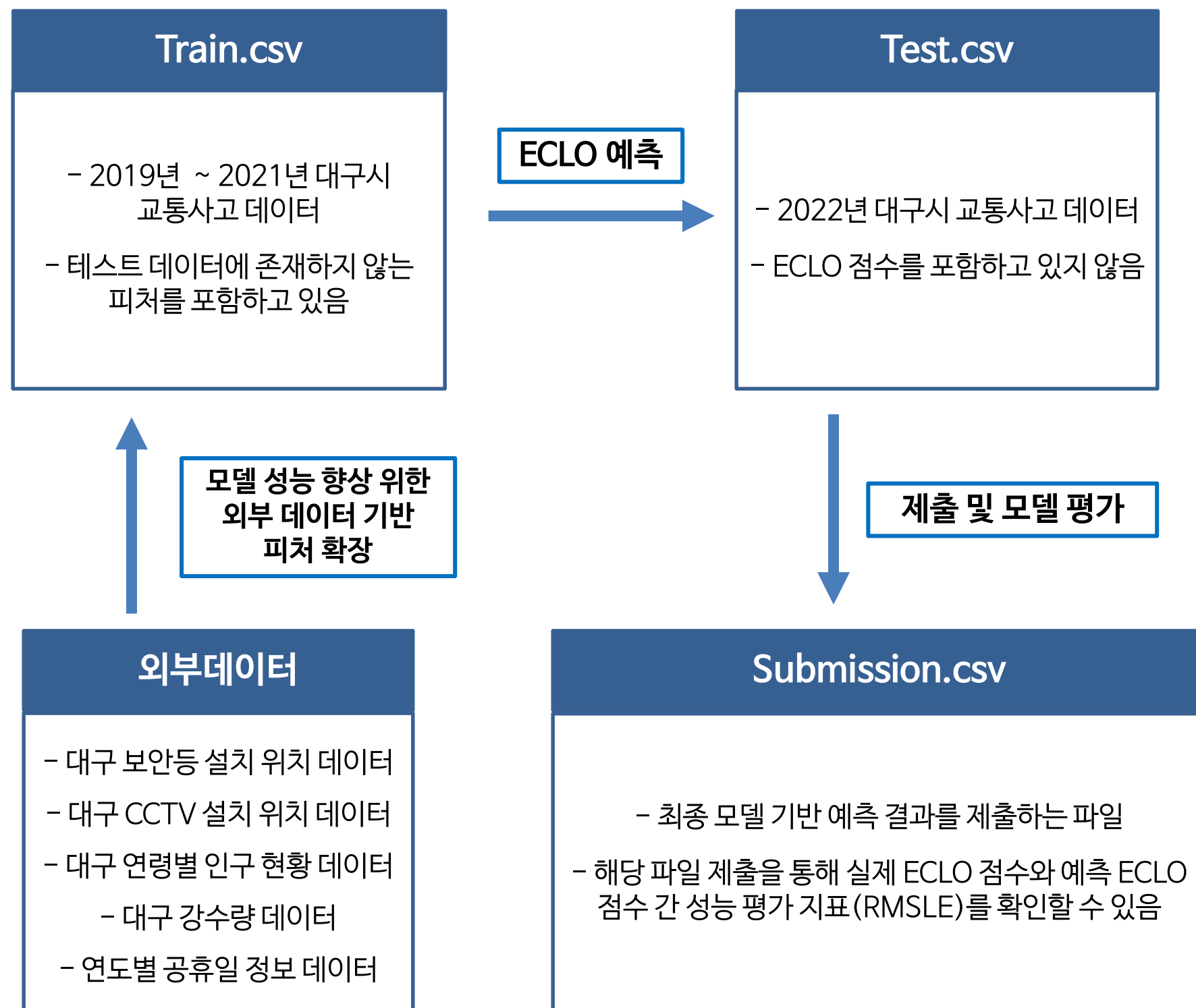
◆ 수집 데이터 목록

- 2019년 ~ 2021년 대구시 교통사고 데이터 (DACON 제공)
- 2019년 ~ 2021년 전국 교통사고 데이터 (DACON 제공)
- 대구시 보안등 설치 위치 데이터 (D-데이터허브 제공)
- 대구시 CCTV 설치 위치 데이터 (D-데이터허브 제공)
- 2019년 ~ 2021년 대구시 연령별 인구 현황 데이터 (대구통계 제공)
- 2019년 ~ 2021년 대구시 강수량 데이터 (기상청 제공)
- 연도별 공휴일 정보 데이터 (공공데이터포털 제공)

◆ 데이터 소개

- 한국자동차연구원과 대구디지털혁신진흥원에서 교통 사고의 원인을 규명하고 사고율을 낮추기 위해, **시공간** 및 **환경적 정보**로부터 **사고 위험도** (ECLO)를 예측하는 AI 알고리즘 발굴을 목표로 대구 및 전국 교통사고 데이터를 제공
- 예측 정확도 향상 위해 **대구 CCTV 설치 위치**, **보안등 설치 위치**, **연도별 공휴일 정보** 등의 **외부데이터**를 추가적으로 수집
- 본 대회에서는 2019년 ~ 2021년까지 발생한 대구시 교통사고 발생 정보와 **ECLO 점수가 포함된** 데이터를 **훈련 데이터 셋**으로 제공하며, **ECLO 점수가 포함되지 않은 2022년**에 발생한 대구시 교통사고 정보를 **테스트 데이터 셋**으로 제공한다.

◆ 모델 학습 및 예측 프로세스



❖ 주요 변수 설명

◆ Train.csv : 총 39609개의 데이터, 23개의 변수로 구성

컬럼명	설명	데이터 형태	컬럼명	설명	데이터 형태
ID	대구에서 발생한 교통사고의 고유 ID	[ACCIDENT_0000 ... ACCIDENT_39608]	가해운전자 연령	사고 발생 당시의 가해운전자 연령 정보	[8세, 9세 ... 90세 이상, 98세 이상, 미분류]
사고 일시	사고 발생 당시의 날짜 및 시간 정보	[2019-01-01 00 ... 2021-12-31 23]	가해운전자 상해정도	사고 발생 당시 가해운전자의 상해정도 정보	[경상, 중상, 사망, 부상신고, 상해없음, 기타불명]
요일	사고 발생 당시의 요일 정보	[월요일, 화요일 ... 토요일, 일요일]	피해운전자 차종	사고 발생 당시의 피해운전자 차종 정보	[개인형이동수단(PM), 건설기계, 기타불명, 농기계, 보행자, 사륜오토바이(ATV), 승용, 승합, 원동기, 이륜, 자전거, 특수, 화물, NULL]
기상상태	사고 발생 당시의 기상 정보	[눈, 맑음, 비, 안개, 흐림, 기타]	피해운전자 성별	사고 발생 당시의 피해운전자 성별 정보	[남, 여, 기타불명, NULL]
시 군 구	사고 발생 당시의 위치 정보	[대구광역시 중구 대신동 ... 대구광역시 달서구 감삼동]	피해운전자 연령	사고 발생 당시의 피해운전자 연령 정보	[1세, 2세 ... 90세 이상, 98세 이상, 미분류, NULL]
도로형태	사고 발생 당시의 도로 형태 정보	교차로 - 교차로부근, 교차로안, 교차로횡단보도내 단일로 - 고가도로위, 교량, 지하차도(도로)내, 터널, 기타 주차장 - 주차장 / 기타 - 기타	피해운전자 상해정도	사고 발생 당시 피해운전자의 상해정도 정보	[경상, 중상, 사망, 부상신고, 기타불명, 상해없음, NULL]
노면상태	사고 발생 당시의 도로 노면 상태 정보	[건조, 서리/결빙, 적설, 젖음/습기, 침수, 기타]	사망자수	사고 발생 당시의 사망자수 정보	[0, 1, 2]
사고유형	사고 발생 당시의 사고유형 정보	[차대사람, 차대차, 차량단독]	중상자수	사고 발생 당시의 중상자수 정보	[0, 1, 2, 3, 4, 5, 6]
사고유형-세부분류	사고 발생 당시의 사고 형태에 대한 세부 정보	[도로외이탈 - 기타, 추락 / 전도전복 - 전도, 전복 / 공작물충돌, 길가장자리구역통행중, 보도통행중, 정면충돌, 차도통행중, 추돌, 측면충돌, 횡단중, 후진중충돌, 기타]	경상자수	사고 발생 당시의 경상자수 정보	[0, 1, 2 ... 18, 22]
법규위반	사고 발생 당시의 법규 위반 정보	[과속, 교차로운행방법위반, 차로위반, 보행자보호의무위반, 불법유턴, 신호위반, 안전거리미확보, 안전운전불이행, 중앙선침범, 직진우회전진행방해, 기타]	부상자수	사고 발생 당시의 부상자수 정보	[0, 1, 2 ... 7, 10]
가해운전자 차종	사고 발생 당시의 가해운전자 차종 정보	[개인형이동수단(PM), 건설기계, 기타불명, 사륜오토바이(ATV), 승용, 승합, 원동기, 자전거, 특수, 화물, 농기계, 이륜]	ECLO	사고 발생 당시의 인명피해 심각도 정보 (사망자수*10 + 중상자수*5 + 경상자수*3 + 부상자수*1)	[1, 2, 3 ... 65, 66, 74]
가해운전자 성별	사고 발생 당시의 가해운전자 성별 정보	[남, 여, 기타불명]	Columns	23	Rows 39609

❖ 주요 변수 설명

◆ Test.csv : 총 10963개의 데이터, 8개의 변수로 구성

- 2022년에 발생한 대구시 교통사고 데이터로 구성
- 최종 모델 제출 위한 ECLO를 예측하기 위한 데이터 셋

컬럼명	설명	데이터 형태	
ID	대구에서 발생한 교통사고의 고유 ID	[ACCIDENT_39609 ... ACCIDENT_50571]	
사고 일시	사고 발생 당시의 날짜 및 시간 정보	[2022-01-01 01 ... 2022-12-31 21]	
요일	사고 발생 당시의 요일 정보	[월요일, 화요일 ... 토요일, 일요일]	
기상 상태	사고 발생 당시의 기상 정보	[눈, 맑음, 비, 흐림, 기타]	
시군구	사고 발생 당시의 위치 정보	[대구광역시 남구 대명동 ... 대구광역시 중구 화전동]	
도로 형태	사고 발생 당시의 도로 형태 정보	교차로 - 교차로부근, 교차로안, 교차로횡단보도내 단일로 - 고가도로위, 교량, 지하차도(도로)내, 터널, 기타 주차장 - 주차장 미분류 - 미분류 기타 - 기타	
노면 상태	사고 발생 당시의 도로 노면 상태 정보	[건조, 서리/결빙, 적설, 젖음/습기, 침수, 기타]	
사고 유형	사고 발생 당시의 사고유형 정보	[차대사람, 차대차, 차량단독]	
Rows	10963	Columns	8

◆ 훈련데이터에만 존재하는 피쳐 목록

- 피해운전자 차종
 - ~~피해운전자상해정도~~
 - 피해운전자 연령
 - 피해운전자 성별
 - ~~가해운전자상해정도~~
- 가해운전자 차종
 - 가해운전자 성별
 - 가해운전자 연령
 - 법규위반
 - 사고유형-세부분류
- ~~경상자수~~
 - ~~중상자수~~
 - ~~부상자수~~
 - ~~사망자수~~

❖ 분석 제외 피쳐 선정

- ECLO 점수 형성에 직접적인 원인이 되고 사고 발생의 원인이 아닌 사고 결과의 심각성을 결과적으로 나타내는 변수는 분석 피쳐에서 제외한다.
- 제외할 피쳐 목록:
 - 사망자수, 부상자수, 중상자수, 경상자수
 - 피해운전자상해정도
 - 가해운전자상해정도
- 위 변수들은 교통사고 발생의 원인이 아닌 결과의 심각성을 나타내는 정보이므로 현황 분석 대상 피쳐에서 제외한다.

❖ 데이터 기본 전처리

◆ 기본 파생 변수 생성 : 정규 표현식을 활용해 반복되는 패턴으로 여러 정보를 포함하고 있는 변수에 대해 분석 및 모델링에 활용 가능한 형태로 파생 변수 생성

1. 날짜 및 시간 정보 생성 - '사고일시' 피처로부터 연도, 월, 일, 시간 정보를 추출 및 변환

```
# '사고일시'에서 연, 월, 일, 시간 추출
time_pattern = r'(\d{4})-(\d{1,2})-(\d{1,2}) (\d{1,2})'

train_df[['연', '월', '일', '시간']] = train['사고일시'].str.extract(time_pattern)
train_df[['연', '월', '일', '시간']] = train_df[['연', '월', '일', '시간']].apply(pd.to_numeric) # 추출된 문자열을 숫자형으로 변환
train_df = train_df.drop(columns=['사고일시']) # 정보 추출이 완료된 '사고일시' 컬럼은 제거

# 해당 과정을 test_df에 대해서도 반복
test_df[['연', '월', '일', '시간']] = test['사고일시'].str.extract(time_pattern)
test_df[['연', '월', '일', '시간']] = test_df[['연', '월', '일', '시간']].apply(pd.to_numeric)
test_df = test_df.drop(columns=['사고일시'])
```

2. 공간(위치) 정보 생성 - '시군구' 피처로부터 의미 있는 공간 정보를 추출 및 변환

```
# '시군구'에서 도시, 구, 동 정보 추출
location_pattern = r'(\S+) (\S+) (\S+)'

train_df[['도시', '구', '동']] = train_org['시군구'].str.extract(location_pattern)
train_df = train_df.drop(columns=['시군구'])

# 테스트 데이터에도 동일하게 적용
test_df[['도시', '구', '동']] = test_org['시군구'].str.extract(location_pattern)
test_df = test_df.drop(columns=['시군구'])
```

3. 도로 형태 정보 추출 - '단일로 - 기타'와 같은 패턴으로 구성되어 있는 '도로형태' 피처를 두 종류의 독립된 정보로 간주해 두 개의 피처로 분리하여 생성

```
# '도로형태'에서 하이픈(-) 기준으로 두 부분 분리
road_pattern = r'(.+) - (.+) '

# train 데이터 분리 및 원본 컬럼 제거
train_df[['도로형태1', '도로형태2']] = train['도로형태'].str.extract(road_pattern)
train_df = train_df.drop(columns=['도로형태'])

# test 데이터에도 동일하게 적용
test_df[['도로형태1', '도로형태2']] = test['도로형태'].str.extract(road_pattern)
test_df = test_df.drop(columns=['도로형태'])
```

✓ 변환 결과

- '사고일시' → 연 / 월 / 일 / 시간

	사고일시
0	2019-01-01 00
1	2019-01-01 00
2	2019-01-01 01
3	2019-01-01 02
4	2019-01-01 04



	연	월	일	시간
0	2019	1	1	0
1	2019	1	1	0
2	2019	1	1	1
3	2019	1	1	2
4	2019	1	1	4

- '시군구' → 도시 / 구 / 동

	시군구
0	대구광역시 중구 대신동
1	대구광역시 달서구 감삼동
2	대구광역시 수성구 두산동
3	대구광역시 북구 복현동
4	대구광역시 동구 신암동



	도시	구	동
0	대구광역시	중구	대신동
1	대구광역시	달서구	감삼동
2	대구광역시	수성구	두산동
3	대구광역시	북구	복현동
4	대구광역시	동구	신암동

- '도로형태' → 도로형태1 / 도로형태2

	도로형태
0	단일로 - 기타
1	단일로 - 기타
2	단일로 - 기타
3	단일로 - 기타
4	단일로 - 기타



	도로형태1	도로형태2
0	단일로	기타
1	단일로	기타
2	단일로	기타
3	단일로	기타
4	단일로	기타



03

성능 지표 기반 전처리 기법 최적화

- 모델 성능 평가 점수 설계
- 데이터 셔플링 여부 및 분석 모델 선정
- 인코딩 방식 최적화
- 종속변수 변환 방법 및 손실함수 최적화

◆ 모델 성능 평가 점수 함수 설계

✓ 모델 성능 평가를 위한 종합 점수 계산 방식

• 모델 간 **공정하고 균형 있는 성능 비교**를 위해, 각 주요 평가지표에 **가중치**를 부여한 **종합 점수**(Score) 계산 방식을 설계

• 주요 평가지표 목록: MSE, RMSE, RMSLE, R^2

• 주요 성능 평가지표 중, **RMSLE**(Root Mean Squared Log Error)를 **핵심 지표**로 설정하고, 그 중요도를 반영해 **지표 별 가중치**를 차등 부여한다.

• 모델의 성능 지표들은 스케일이 서로 다르므로, **Min-Max 정규화**를 통해 평가 기준을 통일한다.

• MSE, RMSE, RMSLE: 낮을수록 성능이 좋으므로 정규화 후 1 - 값 적용

• R^2 : 높을수록 성능이 좋으므로 정규화된 값을 그대로 사용

❖ 종합 점수 계산 방식

지표	중요도(가중치)	방향성	정규화 방식
MSE	0.20	낮을수록 좋음	Min - Max 후 1 - 값
RMSE	0.20	낮을수록 좋음	Min - Max 후 1 - 값
RMSLE	0.50	낮을수록 좋음	Min - Max 후 1 - 값
R^2	0.10	높을수록 좋음	Min - Max 후 값 그대로 사용

❖ 모델별 성능 종합 점수 계산 함수

```
# 모델 성능별 점수 계산
def scoring_model(results):
    # 가중치 설정
    weights = {'MSE': 0.2, 'RMSE': 0.2, 'RMSLE': 0.5, 'R2': 0.1}

    # 정규화 함수
    # 평가지표 별 값의 범위를 통일하기 위해 Min-Max 스케일링 수행
    def min_max_normalize(series, is_better_lower=True):
        min_val = series.min()
        max_val = series.max()
        normalized = (series - min_val) / (max_val - min_val)
        # R2는 높을수록, MSE, RMSE, RMSLE는 낮을수록 성능이 좋은 특성을 반영
        if is_better_lower:
            return 1 - normalized
        else:
            return normalized

    # NaN 값 처리
    # 예측값에 음수가 포함될 경우, 해당 값을 최저 성능으로 간주하여 적절한 대체 값으로 변환한 후 평가지표를 계산
    # MSE, RMSE, RMSLE는 NaN 값 일 경우, 해당 컬럼의 최대값으로 대체
    results['MSE'].fillna(results['MSE'].max(), inplace=True)
    results['RMSE'].fillna(results['RMSE'].max(), inplace=True)
    results['RMSLE'].fillna(results['RMSLE'].max(), inplace=True)

    # R2는 NaN 값 일 경우, 해당 컬럼의 최소값으로 대체
    results['R2'].fillna(results['R2'].min(), inplace=True)

    # 정규화 적용
    df_normalized = results.copy()
    df_normalized['MSE'] = min_max_normalize(results['MSE'], is_better_lower=True)
    df_normalized['RMSE'] = min_max_normalize(results['RMSE'], is_better_lower=True)
    df_normalized['RMSLE'] = min_max_normalize(results['RMSLE'], is_better_lower=True)
    df_normalized['R2'] = min_max_normalize(results['R2'], is_better_lower=False)

    # 성능 종합 점수 계산
    # 평가지표별 Min-Max 스케일링 값 * 가중치
    df_normalized['Score'] = (df_normalized['MSE'] * weights['MSE'] +
                             df_normalized['RMSE'] * weights['RMSE'] +
                             df_normalized['RMSLE'] * weights['RMSLE'] +
                             df_normalized['R2'] * weights['R2'])

    results['Score'] = df_normalized['Score']

    # 점수에 따라 모델 정렬 (내림차순)
    results = results.sort_values(by='Score', ascending=False).reset_index(drop=True)

    return results
```


◆ 데이터 셔플링 여부가 모델 성능에 미치는 영향 분석

✓ 분석 목적

- 훈련 및 테스트 데이터 분할 시, **데이터의 순서를 섞음 여부**에 따른 실험 모델 별 성능 비교를 수행해 데이터가 **시계열적 특성**을 보유하고 있는지 판단
- 총 3개의 **ML** (Machine Learning) 및 **DL** (Deep Learning) 모델을 대상으로 성능을 비교하며, 모델의 대상은 다음과 같다:
 - ML: Light GBM, CatBoost
 - DL: ANN (Artificial Neural Network)
- 위 3개의 모델을 대상으로 데이터 분할 시, **Shuffle = True/False** 설정에 따른 **모델별 성능 점수**를 비교

✓ 실험 전제 조건

1. 각 모델은 모델의 **기본 구조**를 사용해 성능을 측정
2. 모델별 학습 반복 횟수 **100회**
3. 범주형 변수에 대해 **One-Hot 인코딩** 방식 적용
4. **RMSE Loss** 사용
5. 훈련 및 테스트 데이터 비율 - Train(**8**) : Test(**2**)

✓ 모델링 함수 설계

[Light GBM]

```
# LightGBM
def lgbm():
    model = lgb.LGBMRegressor(objective='rmse', n_estimators=100)
    return model
```

[CatBoost]

```
# CatBoost
def catboost():
    model = CatBoostRegressor(loss_function='RMSE', n_estimators=100)
    return model
```

[ANN]

```
# ANN
def ann():
    model = Sequential()
    model.add(Dense(64, activation='relu', input_dim=X_train.shape[1]))
    model.add(Dense(32, activation='relu'))
    model.add(Dense(1))

    model.compile(optimizer=Adam(), loss=rmse_loss)

    return model
```

✓ 데이터 셔플링 설정

- Shuffle=True

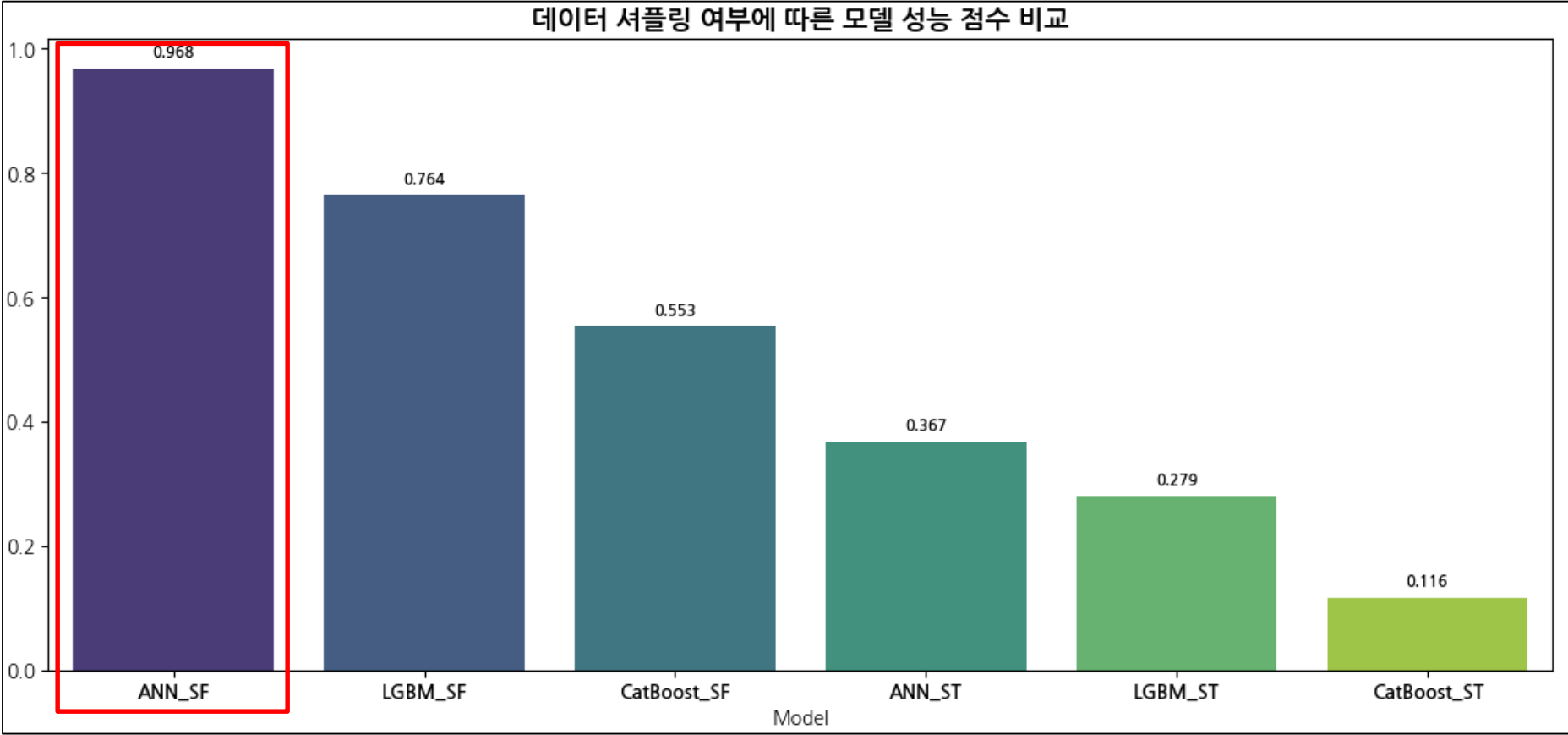
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, shuffle=True)
```

- Shuffle=False

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
```


◆ 데이터 셔플링 여부가 모델 성능에 미치는 영향 분석

❖ 모델별 성능 점수 시각화 (* Shuffle=False → SF / Shuffle=True → ST)



❖ 모델별 성능 평가

Model	MSE	RMSE	RMSLE	R ²	Score
ANN_SF	8.995985	2.999331	0.436095	0.024288	0.967835
LGBM_SF	9.067976	3.011308	0.442931	0.016479	0.763940
CatBoost_SF	9.183759	3.030472	0.446088	0.003921	0.553099
ANN_ST	9.699868	3.114461	0.452377	0.025052	0.367086
LGBM_ST	9.668639	3.109444	0.461089	0.028191	0.279008
CatBoost_ST	9.769824	3.125672	0.462910	0.018020	0.116188

◆ 분석 결과

1. 전반적인 결과 요약

- 데이터 셔플링을 적용하지 않은 모델(Suffle=False)의 성능이 전반적으로 더 우수한 경향을 보였다.
- 특히, ANN_SF 모델은 모든 모델 중 가장 높은 종합 점수(0.968)를 기록하며, 셔플링 미적용 시 최적의 성능을 나타냈다.

2. 주요 인사이트

- 데이터 셔플링 적용 시 전반적인 성능 저하가 관측되었으며, 이는 셔플링 방식이 교통사고 데이터의 내재된 구조적 특성과 맞지 않음을 시사할 수 있다.
- 특히, 셔플링 미적용 시 모델의 예측 성능이 일관되게 향상된 점을 고려할 때, 교통사고 데이터는 시계열적 또는 순서에 민감한 특성을 내포하고 있을 가능성이 높다고 볼 수 있다.
- 따라서 해당 데이터는 시간적 혹은 순차적 흐름을 고려한 전처리 방식이 모델 성능 개선에 보다 적합할 것으로 판단할 수 있다.

3. 분석 모델 선정

- 실험 결과, 데이터 셔플링 여부에 관계없이 ANN(Artificial Neural Network) 모델이 전반적으로 가장 우수한 예측 성능을 보였다.
- 특히, 주요 평가지표(RMSLE) 전반에서 안정적인 결과를 보였으며, 다른 모델 대비 일관된 성능 우위를 나타냈다.
- 이에 따라, ANN 모델을 본 프로젝트의 기본 분석 모델(Baseline Model)로 채택하며, 이후 실험은 해당 모델을 중심으로 진행한다.

◆ 사전 모델 구조 최적화

모델 구조 선 확정의 필요성

- 전처리 방식 최적화 수행에 앞서, **일관된 기준 모델 설정이 필요**
- ANN 기반의 모델 성능이 가장 우수하였기에 이를 **기본 분석 모델로 선정**
- 사전 하이퍼 파라미터 최적화를 통해 **구조적 안정성 및 학습 효율성 확보**

최적화 요소

은닉층 구조
(Hidden Layers)

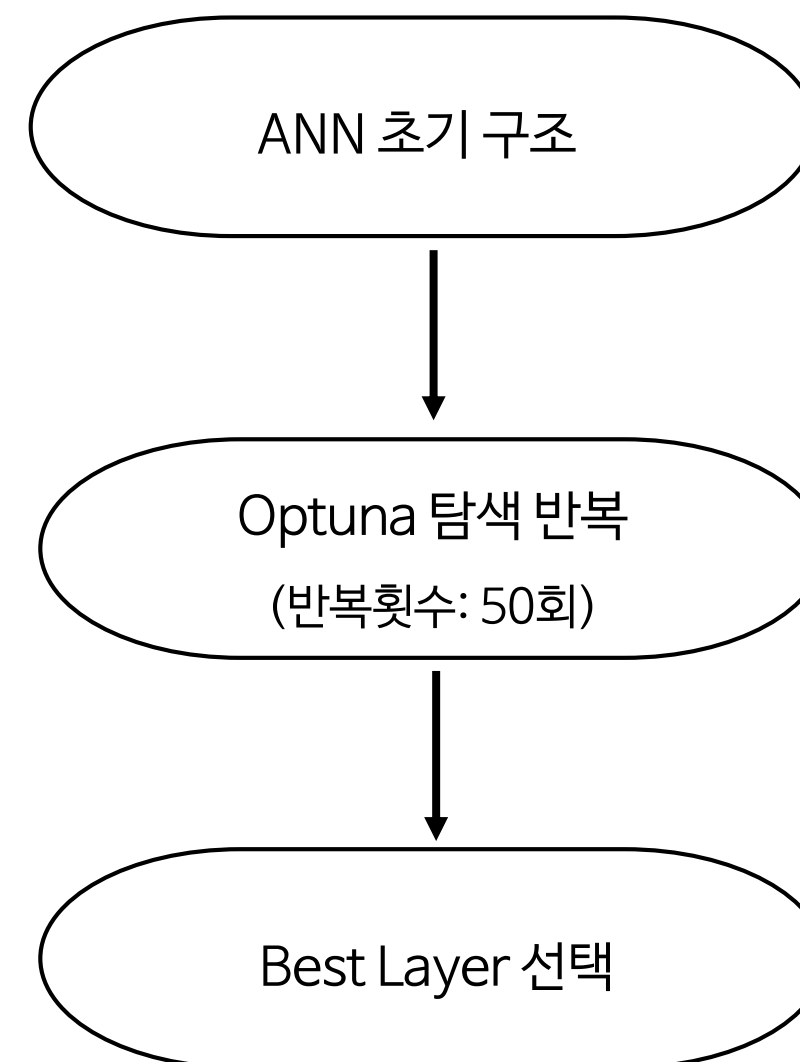
학습률
(Learning Rate)

최적화 기법

Optuna 기반 자동 탐색
(Bayesian Optimization 방식)

평가 기준:
RMSLE 중심 종합 Score

◆ ANN 구조 최적화 흐름도



사전 모델 구조 최적화를 통해 **모델 신뢰성과 성능 기반 분석 프레임 구축**
→ 이후 분석은 해당 구조를 기준으로 수행한다.

◆ Optuna 기반 ANN 구조 최적화 로직 구성

```
def objective(trial):
    n_layers = trial.suggest_int('n_layers', 3, 7)
    initial_units = trial.suggest_categorical('initial_units', unit_val)
    lr = trial.suggest_loguniform('learning_rate', 1e-5, 1e-2)
    min_units = trial.suggest_categorical('min_units', [16, 32])

    # 현재 실험 중인 하이퍼파라미터 출력
    print(f"\nTrial {trial.number}: Current Parameters: {trial.params}")

    model = Sequential()

    # 첫번째 레이어 유닛 수 설정
    units = initial_units

    model.add(Dense(units, activation='relu', input_dim=X_train.shape[1]))

    for _ in range(n_layers - 1): # 첫 번째 레이어 제외
        units = max(units // 2, min_units) # 최소 1개의 뉴런
        model.add(Dense(units, activation='relu'))

        if units == min_units:
            break

    model.add(Dense(1)) # 출력 레이어

    model.compile(optimizer=Adam(learning_rate=lr), loss=rmse_loss)
    model.summary()

    # 조기종료 옵션 설정
    early_stopping = EarlyStopping(monitor='val_loss', patience=20, restore_best_weights=True)

    history = model.fit(X_train, y_train, validation_data=(X_val, y_val),
                        epochs=200, batch_size=32, verbose=1,
                        callbacks=[early_stopping])

    # 훈련/검증 손실 그래프
    plt.figure(figsize=(10, 4))
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.xlabel('Epoch')
    plt.title('Training and Validation Loss Comparison')
    plt.legend()
    plt.show()

    y_pred = model.predict(X_test)

    rmse = np.sqrt(mean_squared_log_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)

    # RMSLE 값이 inf 또는 NaN일 경우, 성능을 최악으로 처리
    if np.isinf(rmse) or np.isnan(rmse):
        rmse = 1e10 # 아주 큰 값으로 설정 (최악의 성능으로 처리)

    # EarlyStopping이 적용되어 학습이 중단된 epoch 출력
    best_epoch = early_stopping.stopped_epoch + 1 if early_stopping.stopped_epoch is not None else None

    # trial 번호와 함께 성능 기록
    performance_df.loc[len(performance_df)] = [trial.number, mse, rmse, rmse, r2, n_layers, initial_units, lr, min_units, best_epoch]

    # 성능 출력
    print(f"Trial {trial.number}: RMSLE = {rmse}, Params = {trial.params}")

    return rmse
```

✓ 하이퍼 파라미터 탐색 구조 설계

- Trial.suggest를 사용해 모델 구조 및 학습 설정을 동적으로 제안
- n_layers(은닉층 개수), initial_units(초기 시작 유닛수), learning_rate(학습률), min_units(마지막 은닉층 최소 유닛 수) 등 다양한 조합을 실험
- 레이어 수 (n_layers) 증가에 따라 유닛 수를 점진적으로 줄이는 계단식 감소 구조 설계

✓ 모델 학습 및 조기 종료 로직

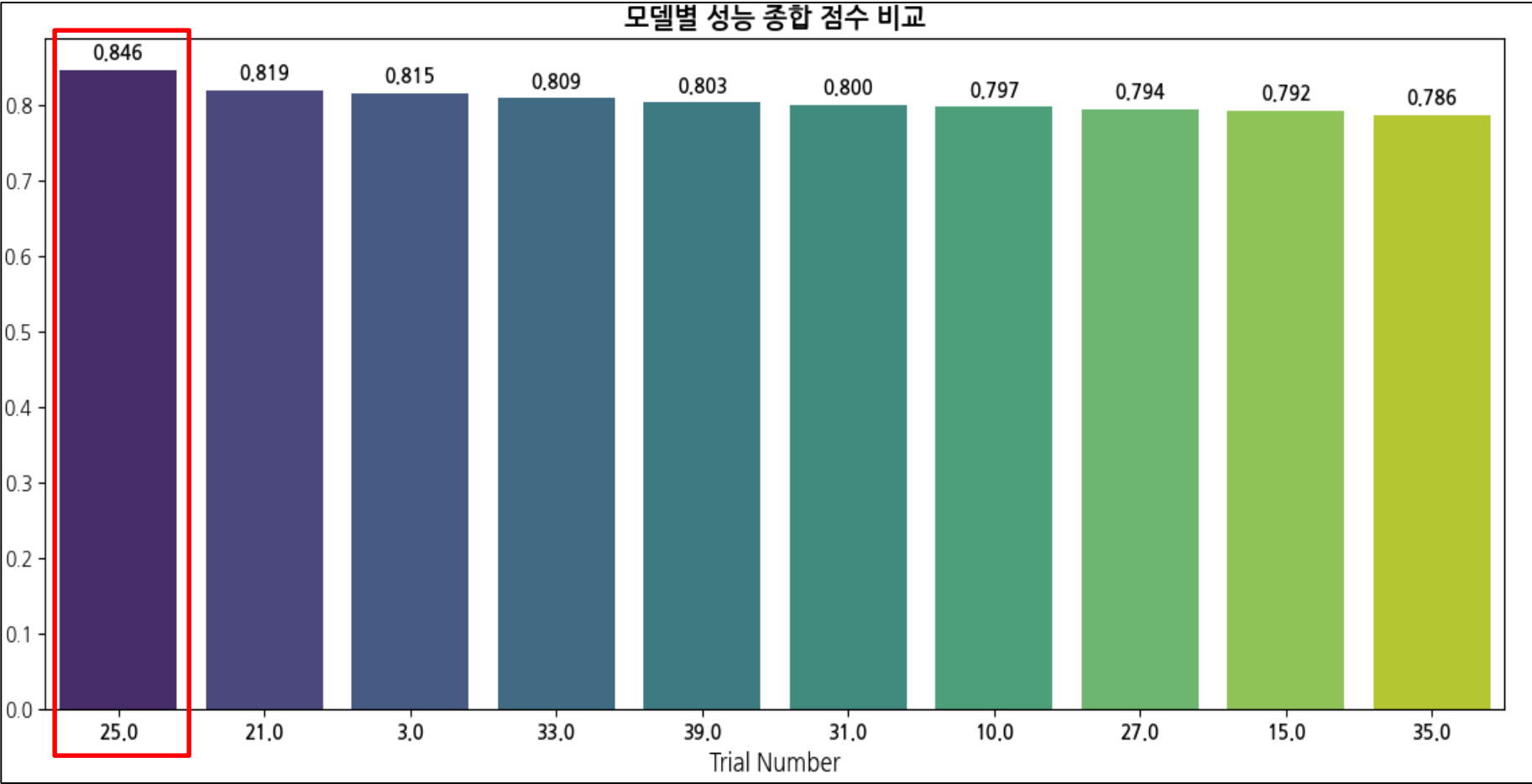
- EarlyStopping 적용으로 과적합 방지 및 효율적인 학습 유지(patience=20)
- restore_best_weights=True로 최적 시점 가중치 보존

✓ 성능 평가 지표 및 예외 처리 설계

- 평가 지표: MSE, RMSE, RMSLE, R^2 등 종합 사용
- RMSLE 중심의 return값 설정으로 로그 스케일 예측 안정성 중점
- 예외 상황 (NaN, inf) 발생 시 RMSLE=1e10 처리 → 최악 성능으로 간주하여 탐색 유도 방지

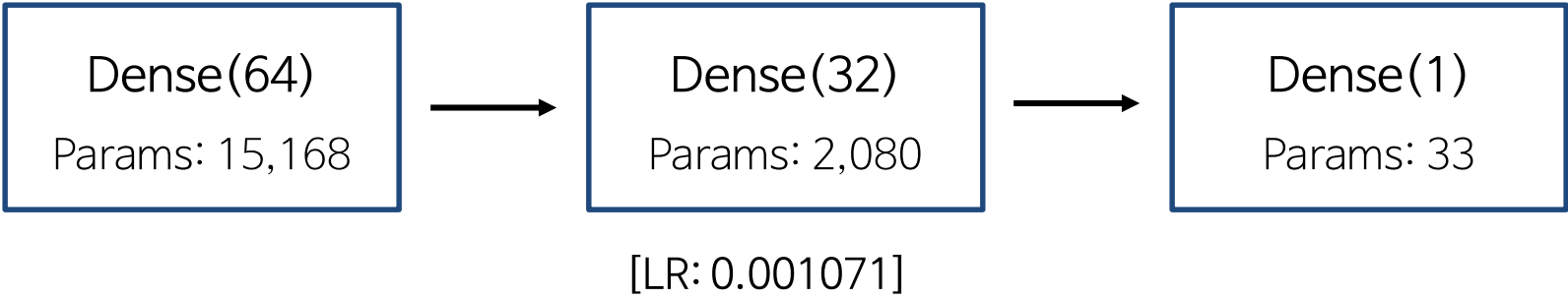
◆ 은닉층 개수 및 학습률 최적화 결과

❖ Top 10 모델 구조에 따른 종합 성능 비교 (Score 기준)



Trial	MSE	RMSE	RMSLE	R ²	Layers Num	Initial Unit	Learning Rate	Min Unit	Score
25	9.107475	3.017859	0.433427	0.023316	3	64	0.001071	32	0.846447
21	9.116570	3.019366	0.433430	0.022341	3	64	0.004359	32	0.819336
3	9.101503	3.016870	0.434890	0.023957	5	256	0.001635	32	0.814873
33	9.114532	3.019028	0.433930	0.022560	3	64	0.008857	32	0.808546
39	9.107266	3.017825	0.434732	0.023339	6	1024	0.001137	64	0.803111
31	9.122792	3.020396	0.433449	0.021674	3	64	0.009575	32	0.800237
10	9.100216	3.016656	0.435529	0.024095	5	256	0.000692	32	0.797174
27	9.097858	3.016266	0.435840	0.024348	3	64	0.001165	32	0.793713
15	9.104537	3.017373	0.435305	0.023631	3	64	0.000302	32	0.791901
35	9.105857	3.017591	0.435350	0.023490	3	64	0.006253	32	0.786463

✓ 최우수 성능 모델 구조



✓ 주요 인사이트

• 얇고 간결한 구조가 가장 효과적

→ 은닉층 3~5개, 초기 유닛 수 64개 수준의 경량화 모델이 복잡한 구조 대비 더 우수한 예측 성능 확보

• 과도한 깊이와 파라미터 수는 성능 향상에 기여하지 않음

→ 깊은 네트워크(6층 이상)나 대규모 유닛(256~1024)은 성능 개선 없이 학습 시간과 과적합 가능성만 증가

• 학습률은 1e-3 전후 구간에서 성능과 안정성 균형 확보

→ 과도하게 큰 또는 작은 학습률은 수렴 불안정 및 성능 저하 유발



은닉층 개수 및 학습률에 대한 하이퍼 파라미터 최적화 결과, **종합 성능 점수가 가장 우수한 Trial 25번 모델을 기준 모델로 채택**

해당 모델은 **구조적 효율성과 예측 안정성 측면에서 우수한 균형을** 보였으며, 향후 **데이터 전처리 기법 또는 입력 특성 조합에 따른 성능 비교 실험의 기준 모델**로 활용하기로 결정

◆ 인코딩 방식 최적화

→ 인코딩 방식별 모델 성능 비교를 통해, 데이터 특성에 알맞은 최적 인코딩 방식 설정

❖ 비교할 인코딩 방법 목록

라벨 인코딩 (Label Encoding)

- 각 범주형 값을 0, 1, 2, ...처럼 정수로 매핑
- 간단하지만, 숫자 간 순서가 의미 있는 것으로 오해될 수 있음

타겟 인코딩 (Target Encoding)

- 각 범주를 해당 범주에 대한 타겟 변수의 평균값으로 변환
- 범주와 타겟 간의 관계를 반영할 수 있어 예측력 향상 가능

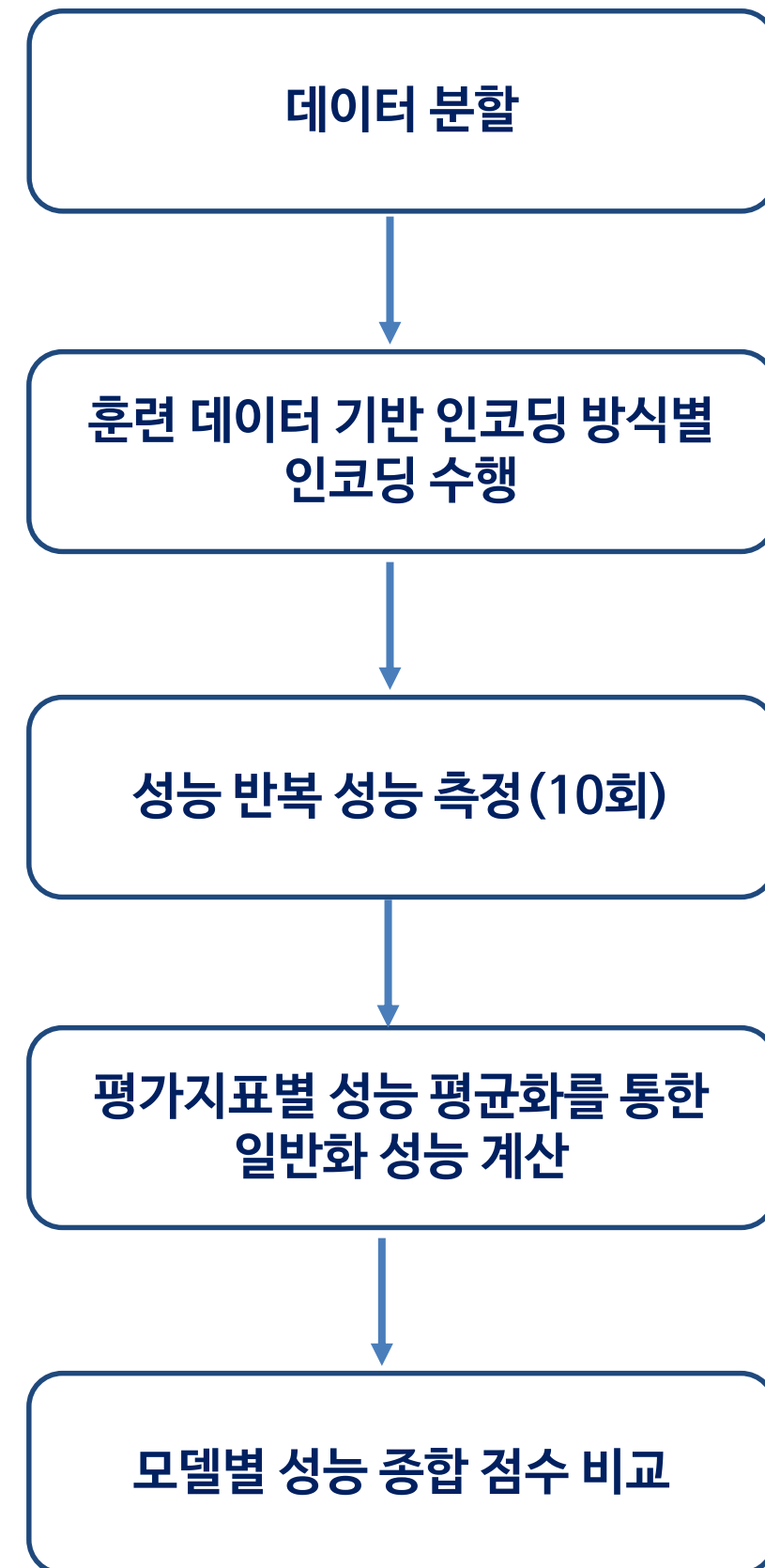
원-핫 인코딩 (One-Hot Encoding)

- 각 범주를 이진 벡터로 표현, 해당 위치만 1이고 나머지는 0
- 순서 정보가 없고, 모델에 편향을 주지 않음
- 범주 수가 많으면 차원이 커지는 단점 있음(희소 벡터)

임베딩 인코딩 (Embedding Encoding)

- 범주형 값을 연속된 밀집 벡터로 표현 (주로 딥러닝에서 사용)
- 고차원 범주형 변수 처리에 유리, 의미론적 유사성 반영 가능
- 임베딩 레이어를 학습해야 하므로 학습 비용이 큼

❖ 실험 절차



❖ 모델 성능 반복 측정 로직 구성

- ✓ 1. 데이터 분할
- 훈련 데이터: 2019 ~ 2020년 데이터
 - 검증 데이터: 훈련데이터의 20%
 - 테스트 데이터: 2021년 데이터

```
def repeat_train_model(df, categorical_columns, temporal_columns, encoding_method=None, num_experiments=None):
    # 훈련 데이터 (2019년 / 2020년 데이터)
    train_data = df[(df['연'] == 2019) | (df['연'] == 2020)]

    # 검증 데이터 (훈련 데이터의 20%)
    train_data, val_data = train_test_split(train_data, test_size=0.2, shuffle=False)

    # 테스트 데이터 (2021년의 대구 데이터)
    test_data = df[df['연'] == 2021]
```

- ✓ 2. 인코딩 방식에 맞는 훈련 데이터 기반 인코딩 수행

- 매개변수 입력 방식에 따른 인코딩 방식별 인코딩 함수 설계

```
if encoding_method=='label' or encoding_method=='embedding':
    X_train, X_val, X_test = label_encode_append_unknown_cate(X_train, X_val, X_test, categorical_columns)

    if encoding_method=='embedding':
        X_train_combined = [X_train[temporal_columns].values] + [X_train[col].values for col in categorical_columns]
        X_val_combined = [X_val[temporal_columns].values] + [X_val[col].values for col in categorical_columns]
        X_test_combined = [X_test[temporal_columns].values] + [X_test[col].values for col in categorical_columns]

        input_dims = {col: len(X_train[col].unique()) + 1 for col in categorical_columns} # 모델에 사용할 input_dims 설정
        embedding_dims = {col: round(np.sqrt(input_dims[col])) for col in categorical_columns} # 각 범주형 변수에 대한 임베딩 차원을 개별적으로 설정

    elif encoding_method=='onehot':
        X_train, X_val, X_test = one_hot_encode_unknown_ignore(X_train, X_val, X_test, categorical_columns, temporal_columns)
```

- ✓ 3. 성능 반복 측정 및 측정된 성능 저장

- Num_experiments만큼 반복 측정 후, 각 지표별 성능을 리스트로 저장
- 반복당 학습 횟수 : Epochs - 200 / Patience - 20

```
for i in range(num_experiments): # 반복 실험 횟수
    early_stopping = EarlyStopping(monitor='val_loss', patience=20, restore_best_weights=True)

    if encoding_method=='embedding':
        model = ANN_64_32_Embedding(temporal_columns, categorical_columns, input_dims, embedding_dims)

        history = model.fit(X_train_combined, y_train, validation_data=(X_val_combined, y_val),
                            epochs=200, batch_size=32, verbose=1,
                            callbacks=[early_stopping])

        y_pred = model.predict(X_test_combined) # 예측

    else:
        model = ANN_64_32(X_train)

        history = model.fit(X_train, y_train, validation_data=(X_val, y_val),
                            epochs=200, batch_size=32, verbose=1,
                            callbacks=[early_stopping])

        y_pred = model.predict(X_test) # 예측

    # 훈련/검증 손실 그래프
    plt.figure(figsize=(10, 4))
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.xlabel('Epoch')
    plt.title(f'Experiment {i+1} - Training and Validation Loss Comparison') # 폴드 번호를 제목에 추가
    plt.legend()
    plt.show()

    # 평가지표 계산
    rmsle = np.sqrt(mean_squared_log_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)

    print(f'Experiment {i+1} - MSE: {mse}, RMSE: {rmse}, RMSLE: {rmsle}, R2: {r2}')

    mse_scores.append(mse)
    r2_scores.append(r2)
    rmse_scores.append(rmse)
    rmsle_scores.append(rmsle)

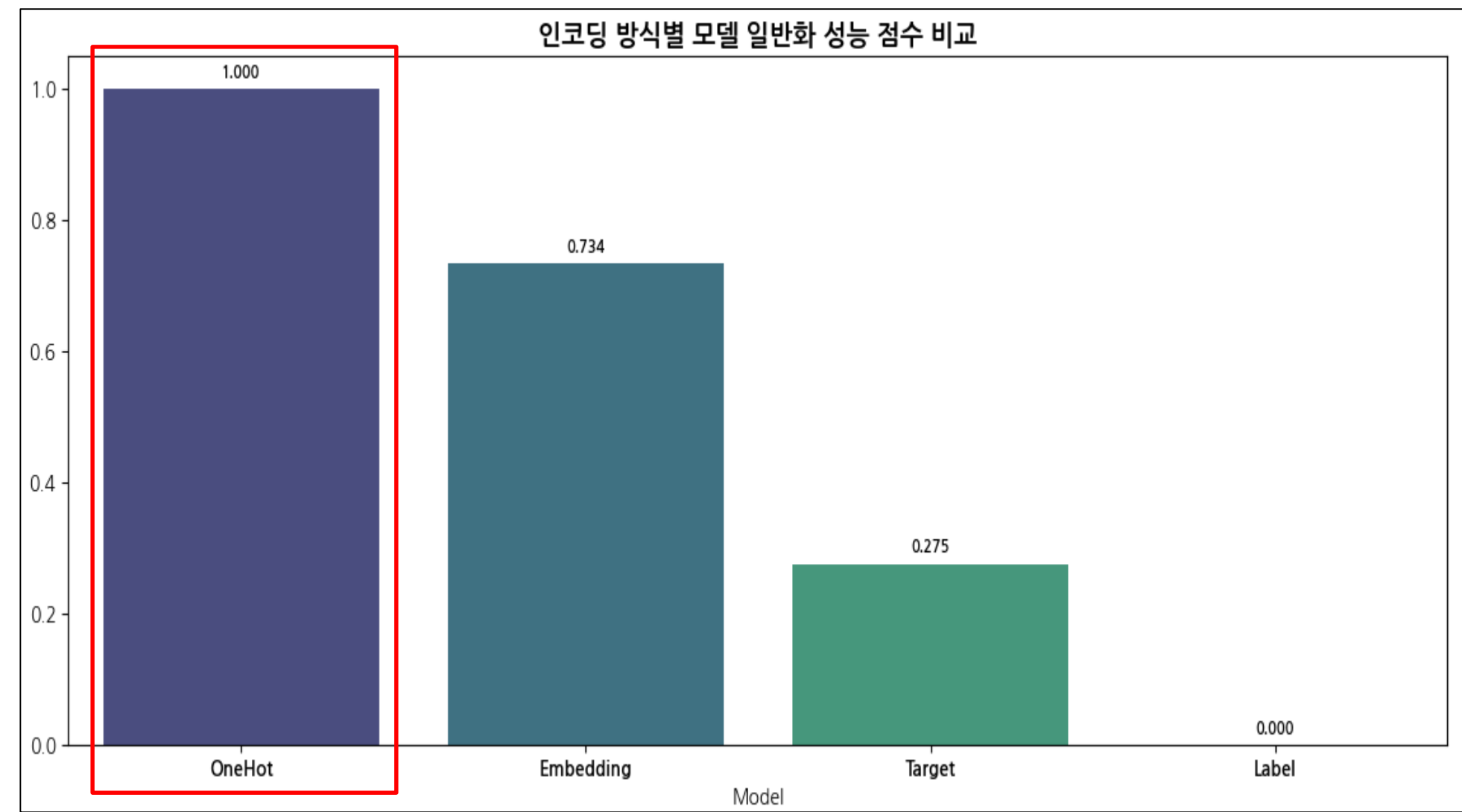
    # 각 지표 리스트만 반환
    return mse_scores, rmse_scores, rmsle_scores, r2_scores
```

- ✓ 4. 평가지표별 성능 평균화를 통한 일반화 성능 계산

```
# 각 성능 지표 출력
print("MSE Scores:", mse_scores)
print("RMSE Scores:", rmse_scores)
print("RMSLE Scores:", rmsle_scores)
print("R2 Scores:", r2_scores)

# 성능 평균화
results = {
    "Model": "Target",
    "MSE": np.mean(mse_scores),
    "RMSE": np.mean(rmse_scores),
    "RMSLE": np.mean(rmsle_scores),
    "R2": np.mean(r2_scores)
}
```


◆ 인코딩 방식 최적화 실험 결과



Model	MSE	RMSE	RMSLE	R ²	Score
One-Hot	9.102281	3.016999	0.436998	0.023873	1.000000
Embedding	9.117688	3.019549	0.438286	0.022221	0.733778
Target	9.181522	3.030098	0.439734	0.015376	0.275121
Label	9.248226	3.041089	0.440016	0.008222	0.000000

✓ 전반적인 결과 요약

- One-Hot 인코딩을 적용한 모델이 전반적으로 가장 우수한 성능을 보였으며, 모든 성능 평가 지표에서 가장 높은 일반화 성능을 기록
- 반면, Label 인코딩은 가장 낮은 성능을 보여, 범주 간 순서를 잘못 해석할 위험성이 있는 모델에는 부적합함을 드러냈다.

✓ 주요 인사이트

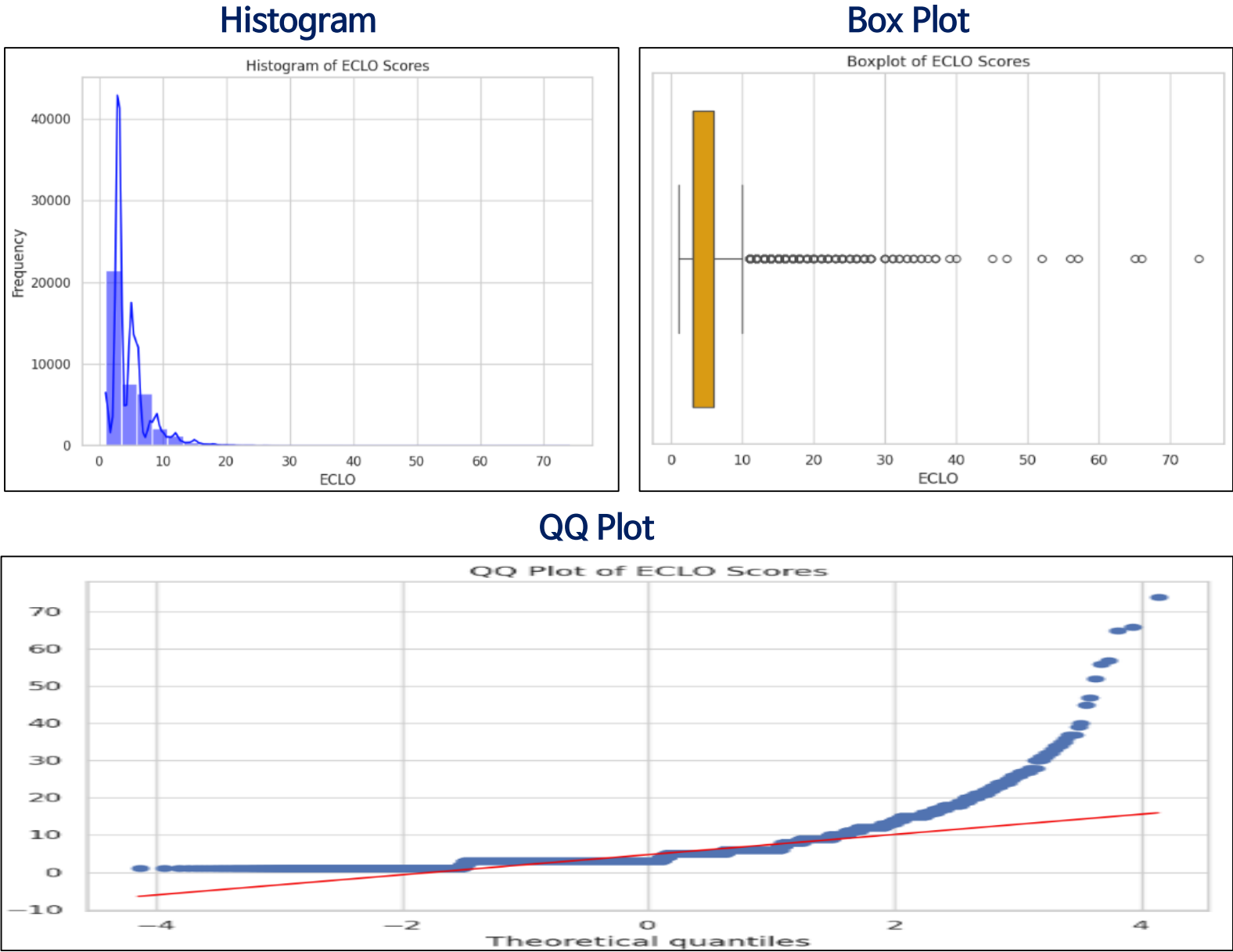
- 인코딩 방식에 따라 모델의 일반화 성능에 큰 차이가 발생
 - 범주형 피처 처리 방식이 모델 학습 및 예측 정확도에 직접적인 영향을 미칠 수 있음을 시사한다.
- 순서에 민감한 인코딩은 성능 저하 유발 가능
 - 종합 성능이 가장 낮은 Label Encoding 방식은 범주 간 의미 없는 순서를 부여해, 모델이 잘못된 연속값으로 해석할 수 있는 문제점을 발견
 - 이는 딥러닝 및 선형 모델에서 성능 저하로 이어질 수 있어 순서에 민감한 데이터 구조에는 부적합함을 보인다.

✓ 최적 인코딩 방법 선정

해당 실험 결과를 바탕으로, One-Hot 인코딩 방식이 가장 안정적이고 높은 일반화 성능을 보여 본 프로젝트의 표준 인코딩 방식으로 채택한다.

◆ 종속 변수(ECLO) 변환의 필요성

✓ ECLO 분포 및 문제점 확인



- 정규성 검정 및 왜도/첨도 확인
 - 정규성 검정(Shapiro-Wilk Test) : Statistic = 0.7157 / p-value = 4.7916e-119
 - 왜도(Skewness) : 3.352
 - 첨도(Kurtosis) : 27.73

✓ 분포 분석 및 문제점 요약

1. 시각적 분포 특성

- 히스토그램:
 - 데이터가 왼쪽에 몰려있고 긴 오른쪽 꼬리를 가진 강한 오른쪽 비대칭 분포(positive skew)
- Boxplot:
 - 중앙값 기준으로 많은 이상치(outliers)가 오른쪽에 분포
- QQ Plot:
 - 직선에서 크게 벗어남 → 정규분포에서 현저히 이탈

2. 수치적 지표

항목	결과
Shapiro-Wilk Test	Statistic = 0.716 p-value $\approx 0 \rightarrow$ 정규성 기각
Skewness	3.35 $\rightarrow$ 강한 오른쪽 비대칭
Kurtosis	27.73 $\rightarrow$ 매우 높은 첨도 (극단값 존재)

3. 문제점 요약

- ✓ 비정규성
 - 통계 검정 결과와 시각화 모두 정규분포 가정을 만족하지 않음
 - 모델링 시 오차 분포 가정 또는 손실 함수 선택에 부정적 영향을 줄 가능성 존재
- ✓ 심한 왜도와 첨도
 - 평균과 중앙값 차이 큼 $\rightarrow$ 대표값 왜곡 가능성 존재
 - 고첨도: 극단값이 모델 학습을 지배할 위험이 있음
- ✓ 이상치 다수
 - 학습 안정성 저해
 - 특히 MSE, RMSE와 같은 제곱 기반 손실 함수 사용 시 민감할 수 있음

4. 개선 필요 사항

- 해당 분석을 통해 종속변수(ECLO)는
- 높은 왜도(Skewness = 3.35) 및 과도한 첨도(Kurtosis = 27.73)
 - 정규성 기각(Shapiro-Wilk $p < 0.001$)
 - 극단적 이상치 다수 존재

등의 특성을 보이며, 이는 모델 학습 안정성 저하, 손실 함수의 왜곡된 반응을 유발할 수 있음을 시사한다.

이에 따라, 다양한 종속 변수 변환 기법 및 손실 함수 간 조합 최적화를 목표로 성능이 일관되고 재현성 있는 기본 모델 구조를 확보한다.

◆ 종속 변수 변환 및 손실 함수 최적화

✓ 실험 설계

- 실험 목적

- 종속변수의 왜도 및 이상치의 영향을 완화하고

- 분석 모델의 예측 안정성과 일반화 성능을 향상시키기 위해

- 다양한 종속 변수 변환 기법과 손실 함수 조합에 따른 성능을 체계적으로 비교·분석
- 실험 조건 및 반복

- 각 변환 방식 × 각 손실 함수 = 총 20가지 조합 평가

- Epochs = 200 / Patience = 20 / 반복성능측정 = 10회

- 범주형 변수에 대해 One-Hot Encoding 방식 사용

- 평가 지표: MSE, RMSE,RMSLE, R^2 기반 종합 성능 점수로 일반화 성능 비교
- 분석 방향성

- 변환 기법과 손실 함수 간 조합 최적화

- 고왜도, 고첨도 종속변수에 가장 적합한 모델 학습 전략 도출

• 종속 변수 변환 기법 (5종)

변환 방식	설명
Log 변환	양의 값을 가진 분포를 압축해 비대칭성 감소
제곱근 변환	완만한 비선형 압축 → 이상치 완화
Box-Cox 변환	정규성 향상 목적의 파라메트릭 변환
로그-제곱근 변환	로그 후 제곱근 → 고왜도 완화에 특화
Z-Score 변환	표준 정규분포로 변환하여 분산 통제

〈변환 방법 별 왜도 및 첨도 측정〉

변환 방법	Skewness	Kurtosis
Original (ECLO)	3.351966	27.729966
Log 변환	0.462225	0.971508
제곱근 변환	1.282783	3.802206
Box-Cox 변환	0.004734	0.993985
로그-제곱근 변환	0.366727	1.057635
Z-Score 변환	3.351966	27.729966

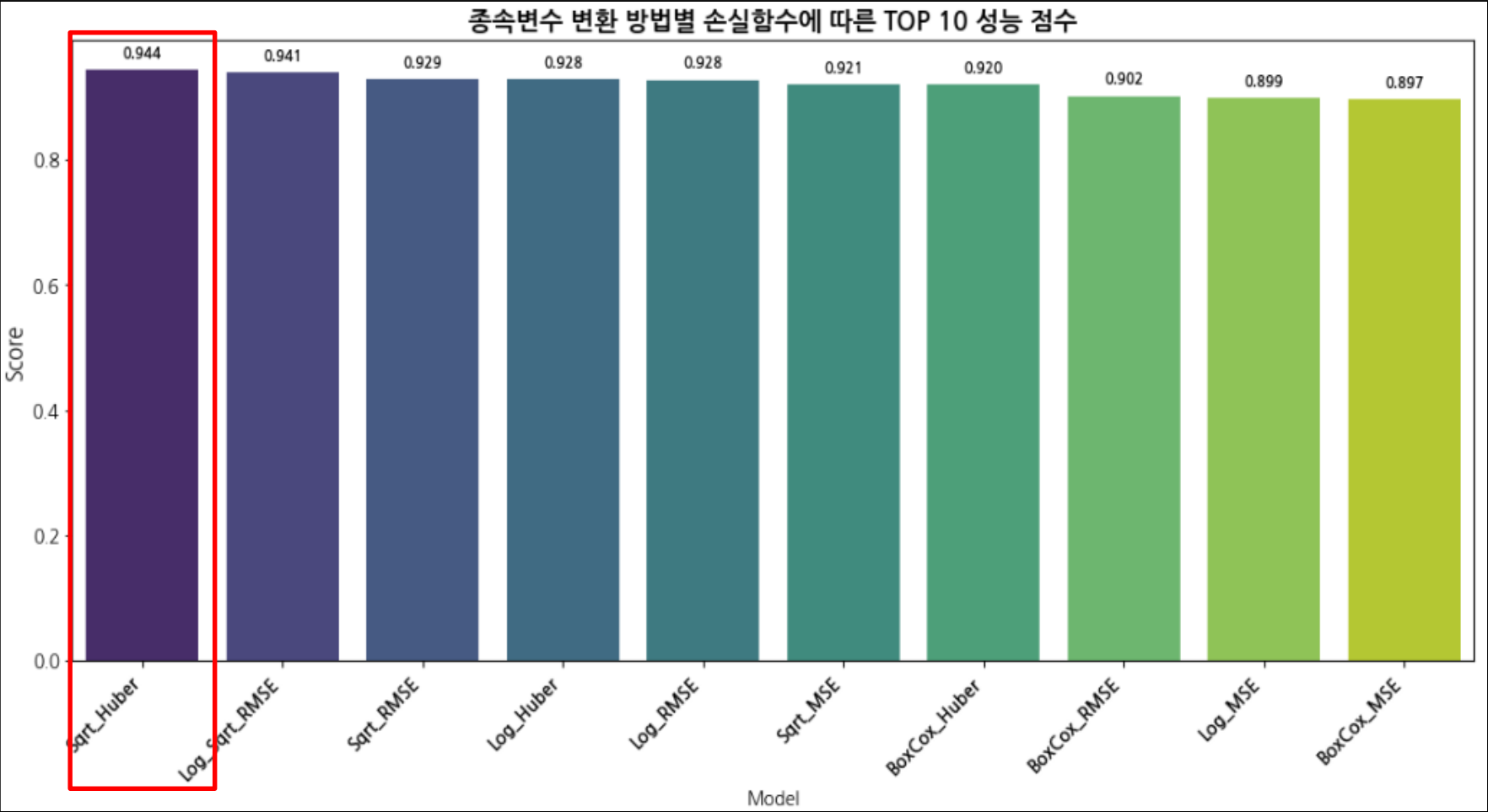
• 손실 함수 설정 (4종)

손실 함수	특성 요약
MSE	양의 값을 가진 분포를 압축해 비대칭성 감소
RMSE	완만한 비선형 압축 → 이상치 완화
MAE	정규성 향상 목적의 파라메트릭 변환
Huber	로그 후 제곱근 → 고왜도 완화에 특화

◆ 종속 변수 변환 및 손실 함수 최적화

✓ 실험 결과 요약

1 TOP 10 상위 성능 점수 조합



❖ 최상위 성능 조합 평가지표

변환 방식	손실 함수	MSE	RMSE	RMSLE	R ²	Score
Sqrt	Huber	9.283770	3.046921	0.427297	0.004410	0.943916
Log-Sqrt	RMSE	9.357300	3.058966	0.426071	-0.003475	0.940557
Sqrt	RMSE	9.256199	3.042346	0.429215	0.007367	0.928773
Log	Huber	9.335519	3.055386	0.427603	-0.001139	0.928382
Log	RMSE	9.375113	3.061845	0.426828	-0.005385	0.927879

2 최고 성능 조합

- Sqrt 변환 + Huber Loss
 - Score: 0.944 (1위)
 - MSE: 9.2838, RMSE: 3.0469, RMSLE: 0.4273, R²: 0.0044
 - 왜도·이상치 완화와 손실 함수의 이상치 대응 특성이 시너지를 발휘
- 상위권 조합 특징
 - Log, Sqrt 등 비선형 변환 + RMSE/Huber 조합이 상위권을 차지하는 모습을 보임

3 변환 방식별 경향

- 비선형 변환(Log, Sqrt)
 - 대부분의 손실 함수 조합에서 원본 데이터(변환 X) 대비 Score 향상
 - 특히 RMSE, Huber와 조합 시 안정적인 성능을 보임
- Box-Cox
 - Huber·RMSE 조합에서는 중간 수준, MAE에서는 저조한 성능을 보임
- Original(변환 X)
 - RMSE, MSE는 준수하나, MAE에서는 최하위
- MAE 전반
 - 모든 변환에서 하위권
 - 해당 문제에서는 이상치 완화보다 정밀 오차 측정이 필요

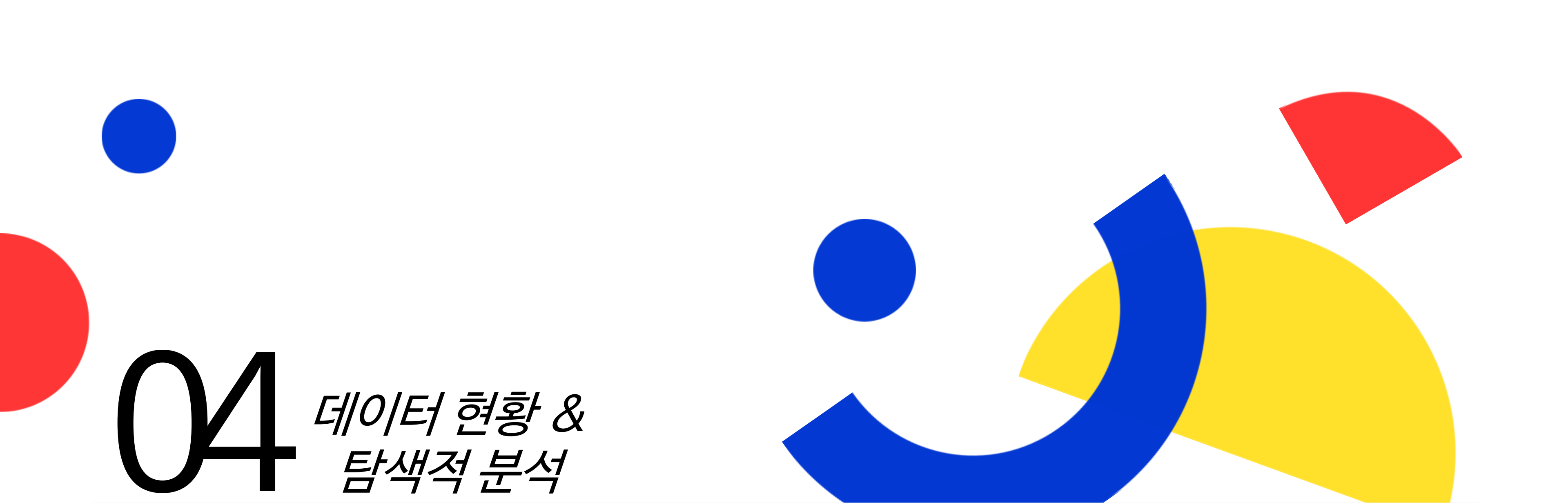
4 주요 인사이트

- Sqrt + Huber 조합은 왜도 감소 + 이상치 완화 → 최고 Score 달성
- RMSE Loss는 변환 기법과의 결합 시 일관된 안정적인 성능 유지
- MAE는 변환 효과와 무관하게 하위권 → 문제 특성상 적합도 낮음
- Box-Cox는 기대보다 낮은 성능 → λ 최적화 재검토 필요



5 결론

- ✓ 해당 실험을 통해 종속변수 변환을 통한 정규성 개선 방법이 모델의 일반화 성능 향상에 유의미한 효과를 보이는것을 검증
- ✓ 손실 함수 비교 결과, Huber Loss와 RMSE Loss가 전반적으로 안정적 성능을 기록
- ✓ 모든 실험 조합 중 Sqrt 변환 + Huber Loss가 최고 성능(Score 0.944) 달성
- ✓ 따라서 해당 조합을 기본 모델 설정으로 채택하여 향후 모델링 및 성능 비교의 기준으로 활용한다.



04

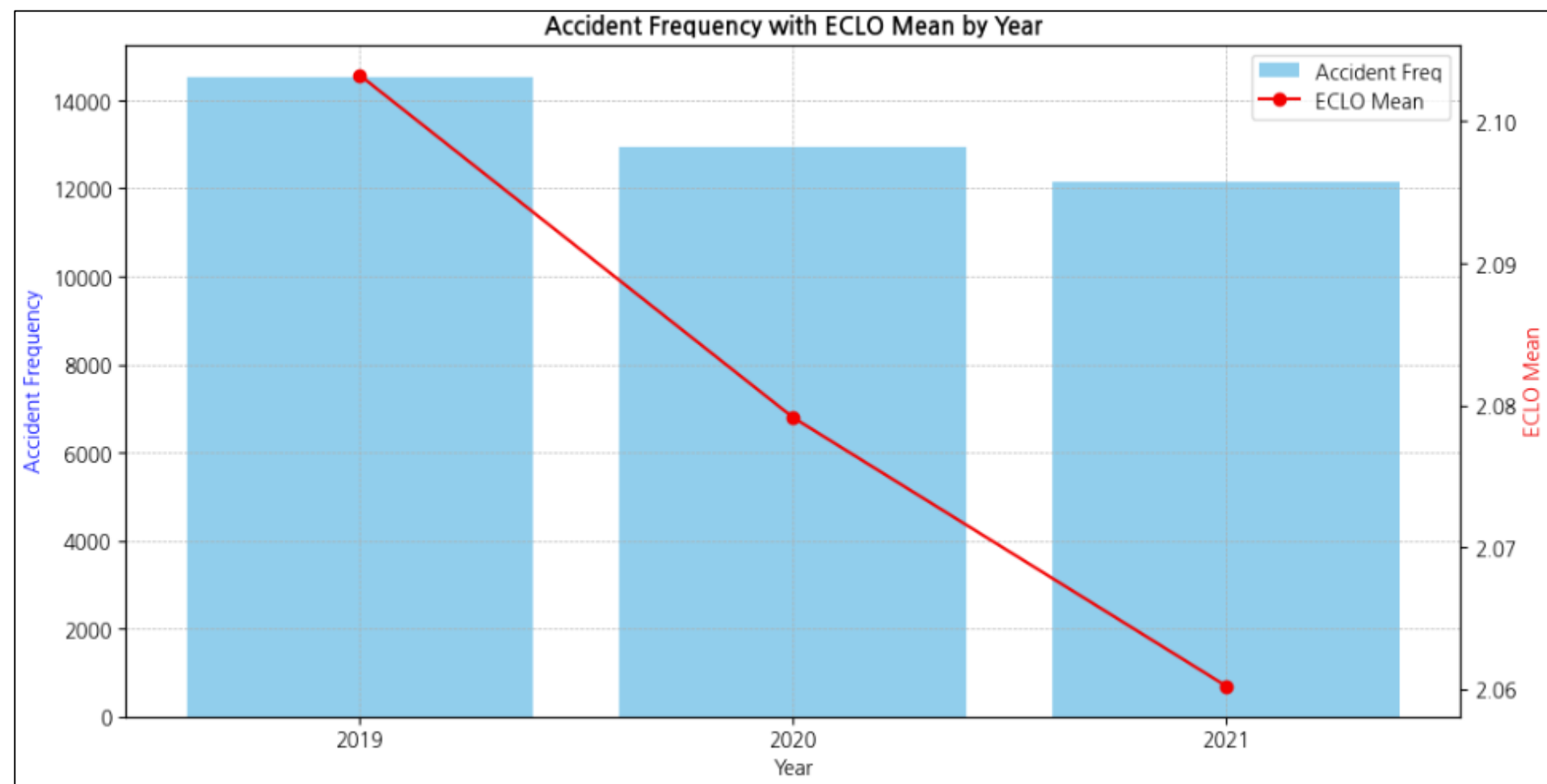
데이터 현황 & 탐색적 분석

- 시간적 정보
- 공간적 정보
- 환경적 정보
- 훈련데이터에만 존재하는 변수

◆ 시간적 정보

- ✓ 시간적 정보를 포함하고 있는 연, 월, 일, 시간, 요일 피처에 대한 현황 및 EDA 분석 수행

◆ 연도에 따른 ECLO 및 사고발생빈도 분석



✓ 연도에 따른 ECLO 변화 분석

1. ANOVA 분산 분석

지표	값	해석
F-statistic	15.8962	집단 간 평균 차이 큼
P-value	1.2565e-07	($p < 0.001$), 유의함

2. 상관 분석

상관분석 방법	상관계수	P-value
Pearson	-0.028	0.000
Spearman	-0.026	0.000

◆ 주요 인사이트 - 연도별 ECLO와 사고발생빈도 변화

- ✓ 사고 건수와 위험도 모두 감소 추세
 - 2019~2021년 동안 사고 발생 빈도와 평균 ECLO 점수가 모두 하락
 - 빈도 감소폭이 위험도 감소폭보다 큼 → 종합 위험도 전반적 완화
- ✓ 연도-ECLO 간 음의 상관관계
 - 상관 분석 결과, 연도가 증가할수록 ECLO 점수 하락 경향
 - 통계적으로 유의미한 음의 상관 확인($p < 0.001$)
- ✓ 연도별 차이의 통계적 검증
 - ANOVA 분석 결과, 연도별 평균 ECLO에 유의미한 차이 존재
 - 귀무가설(연도별 평균이 동일하다) 기각

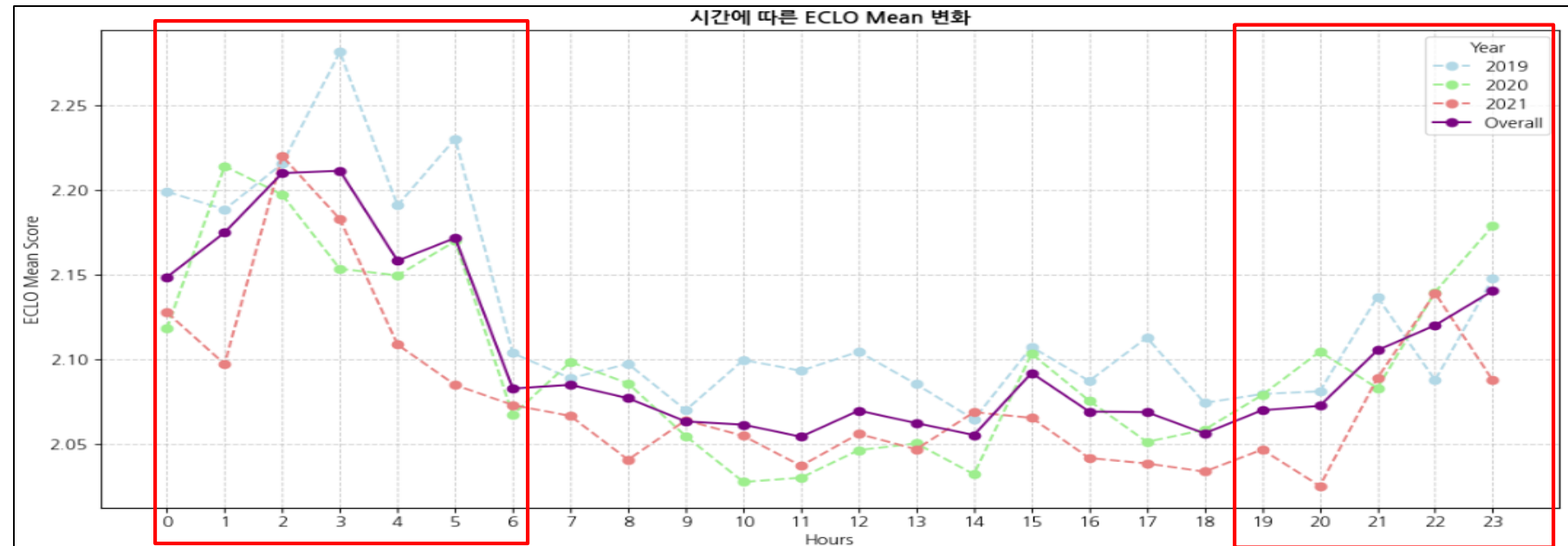


분석 결과

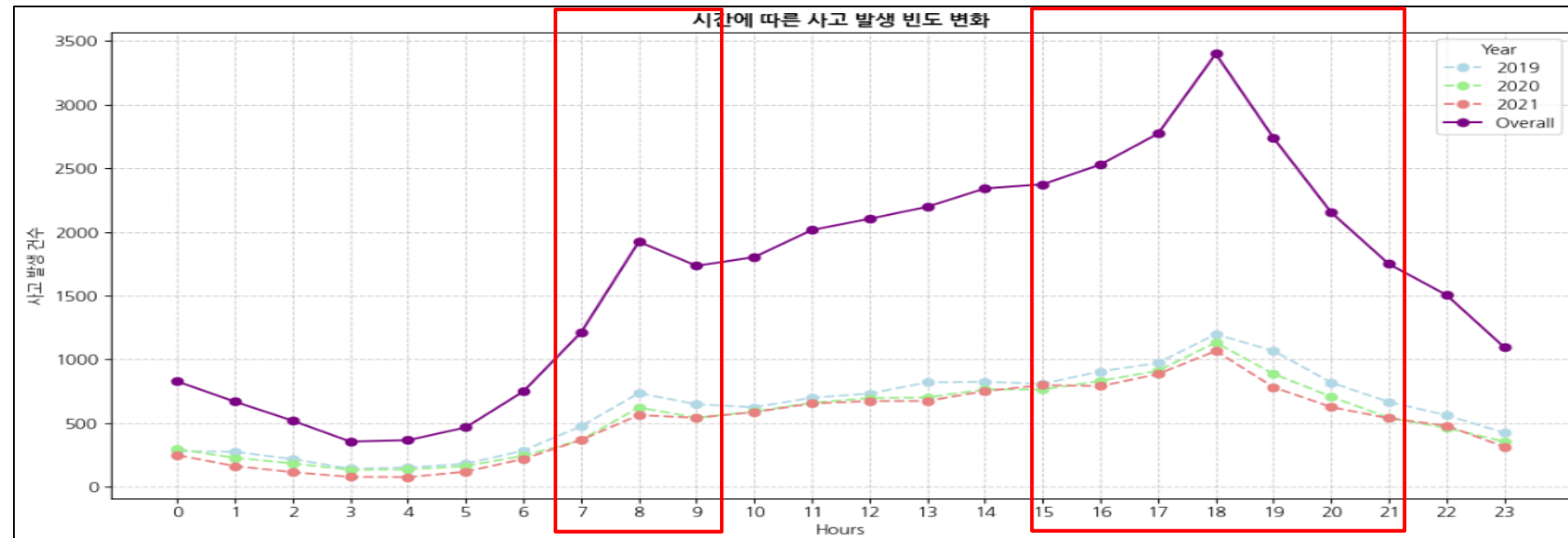
1. 대구시 교통사고 데이터는 연도가 지남에 따라 평균 사고 심각도(ECLO)가 완화되는 경향을 확인
→ 이는 교통 인프라 개선과 안전 정책 강화, 차량 안전 기술 발전 등이 복합적으로 작용한 결과로 추정
2. 연도 변수는 위험도 예측 모델에서 유효한 설명 변수라고 볼 수 있으며, 특히 사고 위험도 장기 추세 분석에서 의미 있는 인사이트를 제공한다.

◆ 시간적 정보 - 시간에 따른 ECLO 및 사고발생빈도 변화 분석

✓ 시간에 따른 ECLO 평균 추세



✓ 시간에 따른 사고발생빈도 추세



✓ 시간에 따른 ECLO 변화 ANOVA 분석

지표	값	해석
F-statistic	5.18	집단 간 평균 차이 큼
P-value	0.000	($p < 0.001$), 유의함

◆ 주요 인사이트 - 시간대별 ECLO와 사고발생빈도 변화

- ✓ 심야 새벽 시간대(02~05시) 평균 ECLO 점수가 가장 높음
 - 전체 평균보다 2~5시 구간에서 평균 사고 위험도(ECLO Mean)가 상대적으로 높음
 - 사고 발생 빈도는 낮지만 사고가 발생하면 더 심각한 결과로 이어질 가능성이 큼
 - 졸음, 음주, 과속 등 고위험 요인 집중 가능성
- ✓ 출·퇴근 시간대 사고 빈도 급증
 - 7~8시, 17~18시 구간에서 사고 발생 건수 급격히 증가
 - ECLO 점수는 해당 시간대에 상대적으로 낮거나 평균 수준을 보임 → 잦지만 경미한 사고 비중 높음
- ✓ 저녁~야간 시간대(20~24시) ECLO 재상승
 - 사고 발생 빈도는 ECLO 평균이 다시 높아지는 경향을 보임
 - 해당 시간대에 피로 누적, 음주운전, 시야 확보 어려움 등 환경적 요인 가능성 존재

◆ 분석 결과

1. 사고 위험도와 사고 발생 빈도의 비연관성

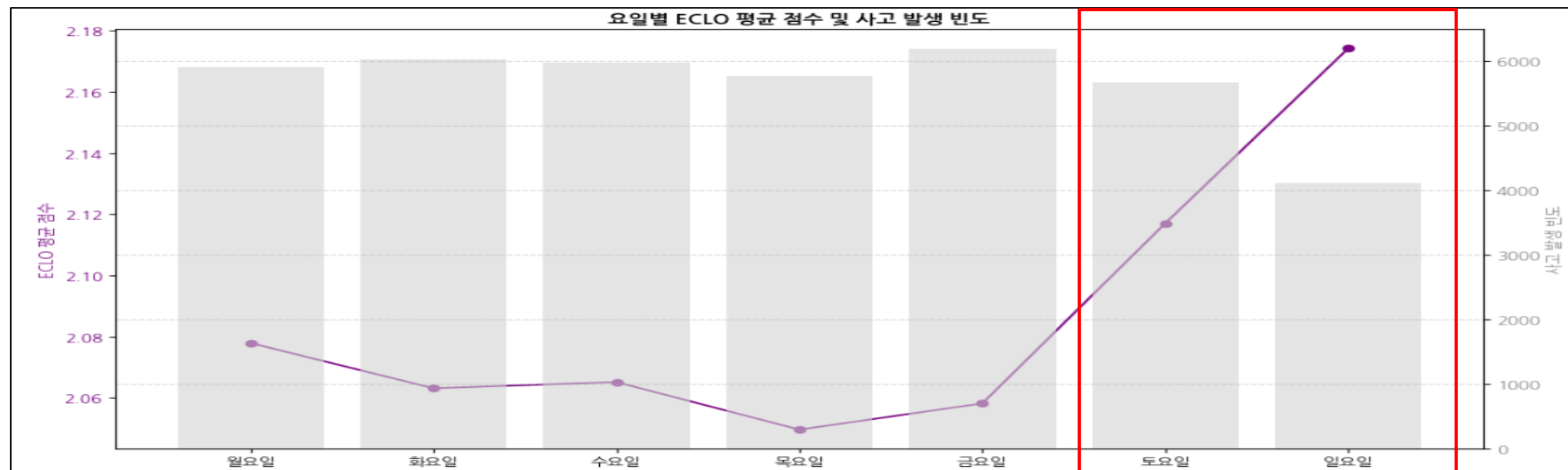
- 분석 결과, ECLO가 높은 시간대는 사고 발생 빈도가 낮고, 반대로 ECLO가 낮은 시간대는 사고 발생 빈도가 높은 경향을 보임
- 이는 사고의 심각도(위험도)와 발생 빈도 간에 유의미한 상관성이 존재하지 않음을 시사하며, 빈도 저감 정책과 심각도 완화 대책을 별도의 접근으로 설계할 필요가 있음을 의미함.

2. 시간대별 일관된 사고 위험도 패턴

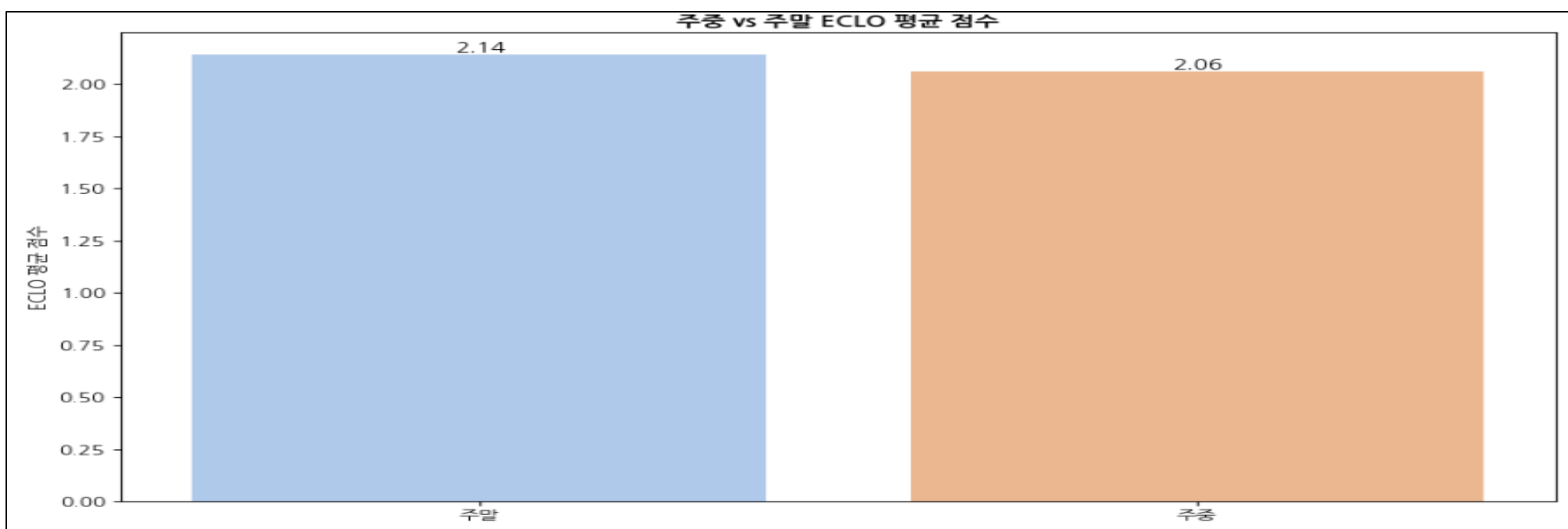
- 2019~2021년 사고 데이터에서 연도별로 유사한 시간대별 ECLO 추세가 관찰됨
- 이러한 패턴은 시간대별 위험 특성이 구조적으로 고정되어 있을 가능성을 시사하며, 시간대 구분 피처를 모델에 반영해 ECLO의 시간적 흐름을 반영하는 것이 모델 예측 성능 개선에 기여할 수 있음을 의미할 수 있다.

◆ 시간적 정보 - 요일에 따른 ECLO 및 사고발생빈도 변화 분석

✓ 요일에 따른 ECLO 및 사고발생빈도 분석



✓ 주말/주중에 따른 ECLO 평균 비교



✓ 주중/주말에 따른 ECLO 변화 ANOVA 분석

지표	값	해석
F-statistic	115.78	집단 간 평균 차이 큼
P-value	0.000	($p < 0.001$), 매우 유의함



◆ 주요 인사이트

1. 주중 대비 주말의 높은 사고 위험도

→ 분석 결과, 요일별 ECLO 평균은 금요일부터 점진적으로 상승해 일요일에 최고치를 기록하는 패턴을 보였다.

→ 이는 사고 발생 빈도와 관계없이 **주말, 특히 일요일에 고위험도 교통사고가 집중되는 경향**을 의미하며, 주말/주중에 따른 ECLO 평균 비교 분석 또한 주말이 주중보다 높은 ECLO 평균 점수가 관찰됨.

→ 따라서 이러한 패턴을 모델에 효과적으로 반영하기 위해, **주말·주중 구분을 명시적으로 나타내는 파생 피처의 추가를 고려해볼 필요가 있다.**

2. 사고 위험도와 사고 발생 빈도의 비연관성

→ 해당 분석 또한, 사고 발생 빈도와 사고 심각도 간에 뚜렷한 상관성 및 연관성은 보이지 않는것으로 확인됨

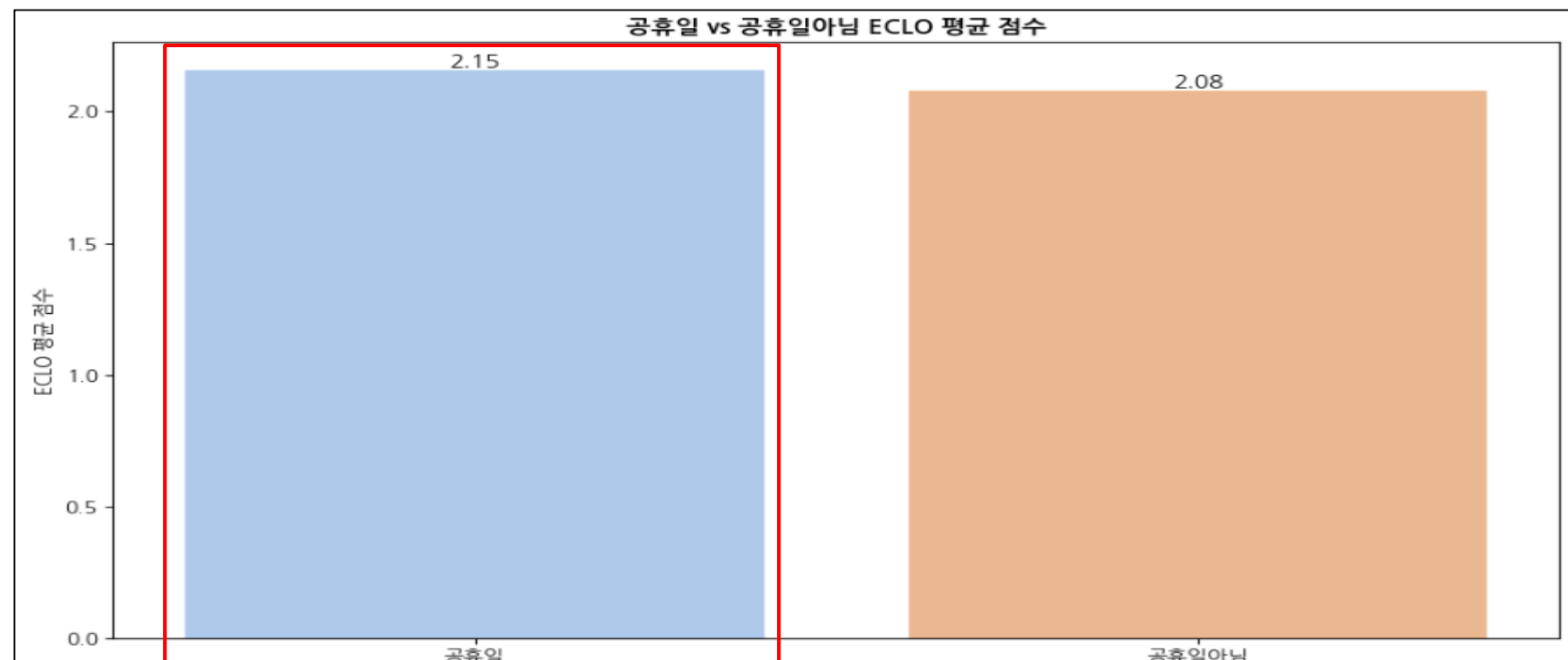
3. 공휴일 사고 피해 특성 분석의 필요성

→ 주말에 고위험도 사고가 집중되는 경향은, **공휴일 역시 유사한 사고 특성**을 가질 가능성을 시사한다.

→ 따라서 공휴일 데이터를 별도로 분석해 **공휴일과 주말 특성을 같이 반영하는 파생 피처 추가 방식**도 고려해볼 수 있다.

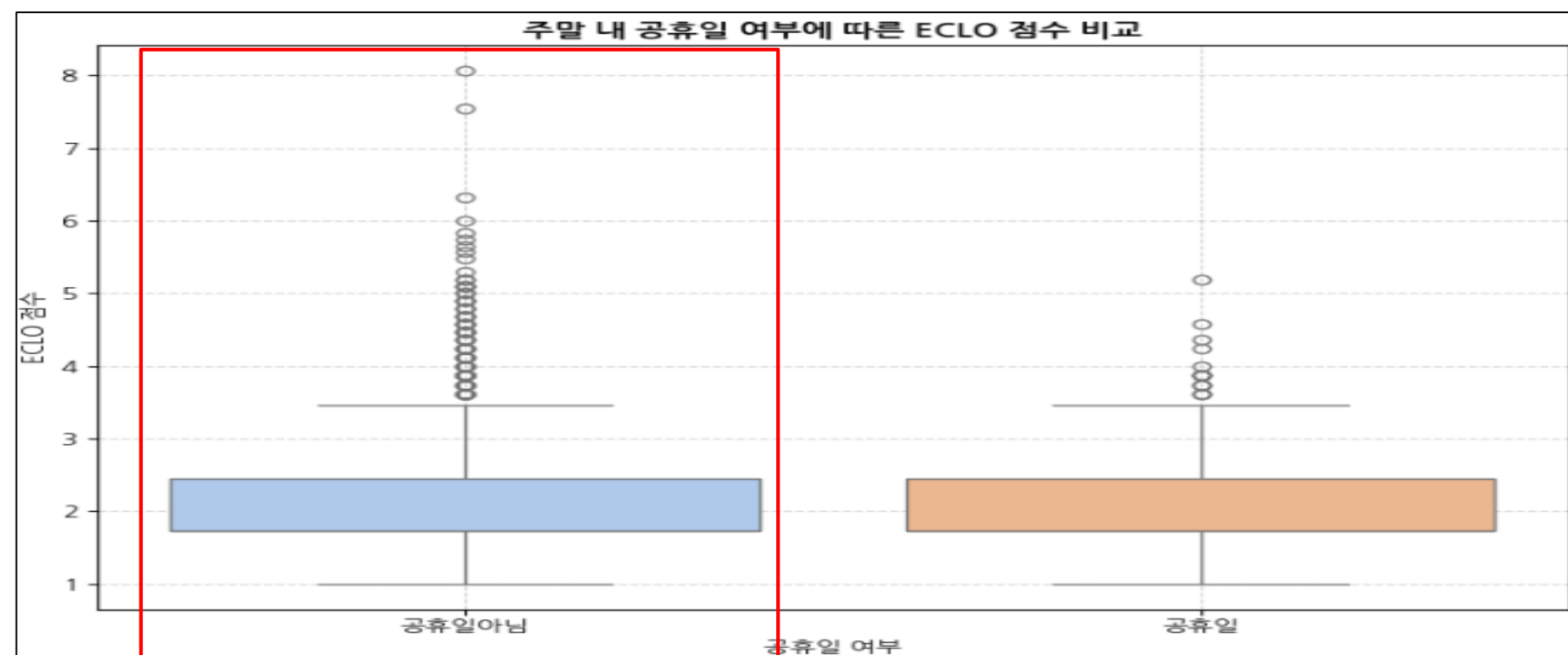
◆ 시간적 정보 - 공휴일 구분에 따른 ECLO 변화 분석 (공휴일 정보가 담긴 외부데이터를 활용해 2019~2021년 공휴일에 해당 하는 날짜를 “공휴일” 그렇지 않은 날짜는 “공휴일아님”으로 구분한 공휴일 구분 피쳐 추가)

✓ 공휴일 구분에 따른 ECLO 평균 비교



✓ 주말 내 공휴일 여부에 따른 ECLO 분포 비교

가설 설정 : 주말에 포함된 공휴일이 주말 기간의 고위험 교통사고 발생에 유의미한 영향을 미칠까?



〈공휴일 여부에 따른 ECLO 변화 ANOVA 분석〉

지표	값	해석
F-statistic	20.18	집단 간 평균 차이 어느 정도 존재
P-value	0.000	($p < 0.001$), 매우 유의함



◆ 주요 인사이트

- 분석 결과, **공휴일의 평균 사고 심각도 (ECLO)가 비공휴일 대비 높게 나타나는 경향**이 확인되었다.
- ANOVA 분석 결과, 두 그룹 간 ECLO 점수의 차이는 **통계적으로 유의미**하였으며, 이는 공휴일에 평상시보다 **심각도가 높은 교통사고가 상대적으로 더 빈번하게 발생함**을 시사한다.
- 이러한 분석 결과를 바탕으로, **공휴일 여부를 나타내는 파생 피쳐를 생성하여 모델에 반영하는 것은 사고 심각도 예측 성능 향상에 기여할 것으로 기대된다.**

〈주말 내 공휴일 여부에 따른 ECLO 변화 ANOVA 분석〉

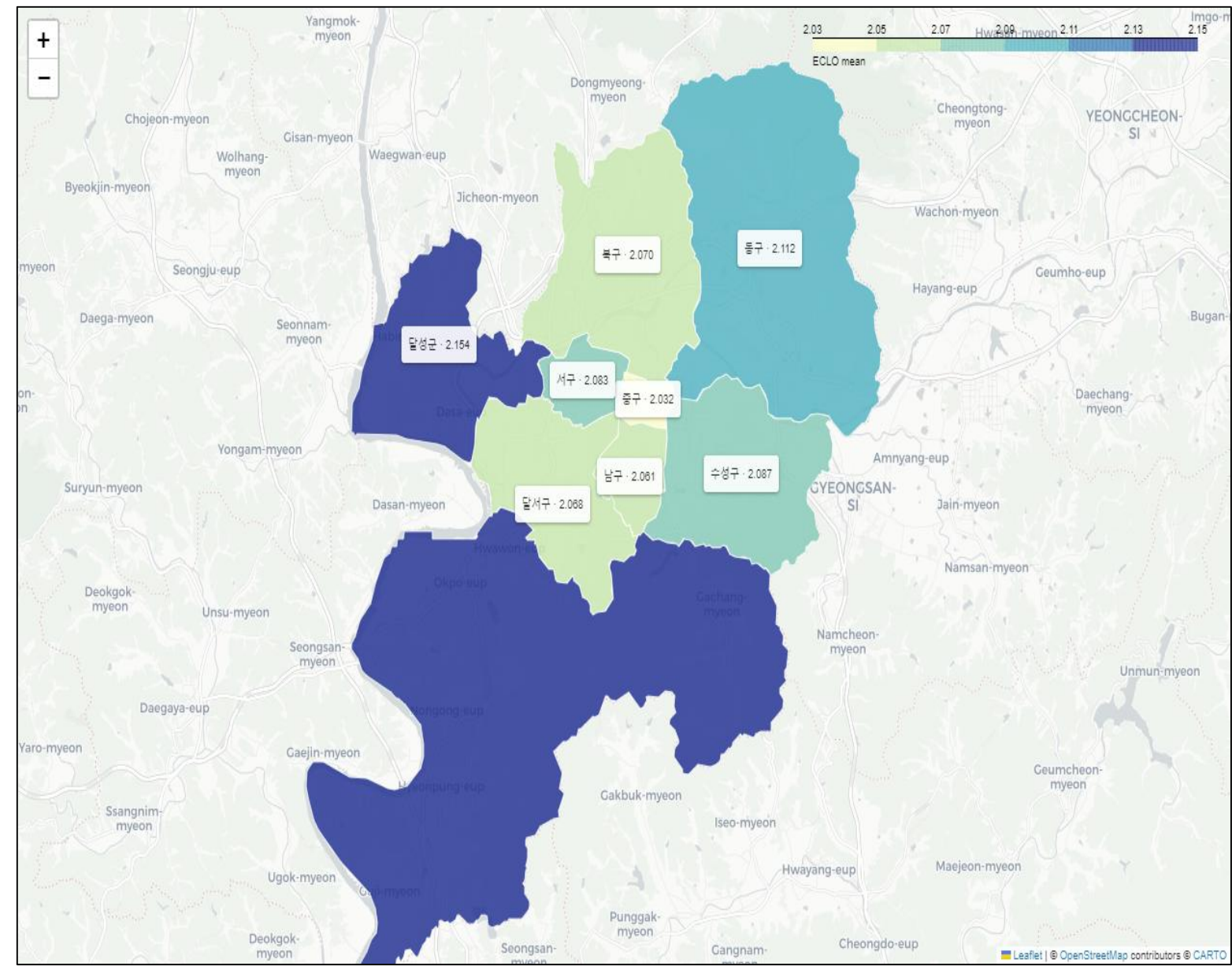
지표	값	해석
F-statistic	0.00	집단 간 평균 차이가 거의 없음
P-value	0.9830	$p > 0.05 \rightarrow$ 통계적으로 유의하지 않음

◆ 비교 결과

- 박스 플롯 분석 결과, 주말 내 비공휴일의 ECLO 분포에서 고위험 사고를 나타내는 이상치가 더 빈번하게 나타났으며, 주말 내 공휴일보다 심각도가 높은 사고 발생 비중이 상대적으로 높았다.
- ANOVA 분석 결과, 주말 내 공휴일과 비공휴일 간 ECLO 점수 차이는 통계적으로 유의미하지 않은 것으로 나타났다.
- 따라서 주말 내 공휴일 여부는 고위험 교통사고 발생 여부에 유의한 영향을 미치지 않는 것으로 해석할 수 있으며, 기존 가설은 기각한다.

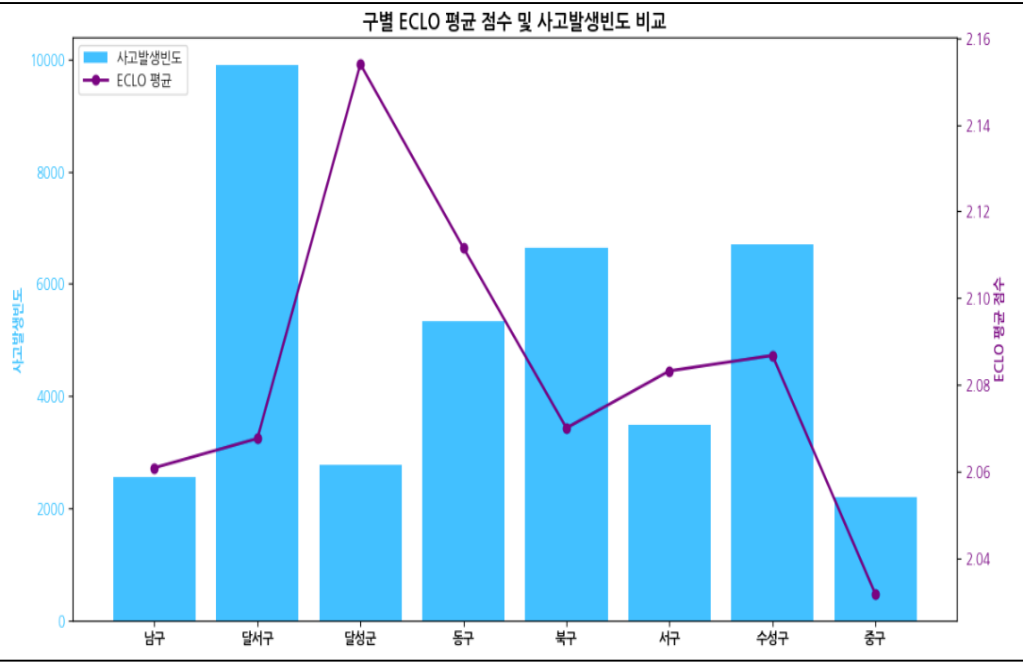
◆ 공간적 정보 - 구가 ECLO에 미치는 영향 분석

✓ 대구시 구별 공간 기반 ECLO 분포



✓ 구별 면적 크기(km²)	1. 달성군 : 428.36	5. 달서구 : 62.37
	2. 동구 : 182.15	6. 남구 : 17.43
	3. 북구 : 93.99	7. 서구 : 17.32
	4. 수성구 : 76.54	8. 중구 : 7.06

✓ 구별 ECLO 평균 및 사고발생빈도 비교

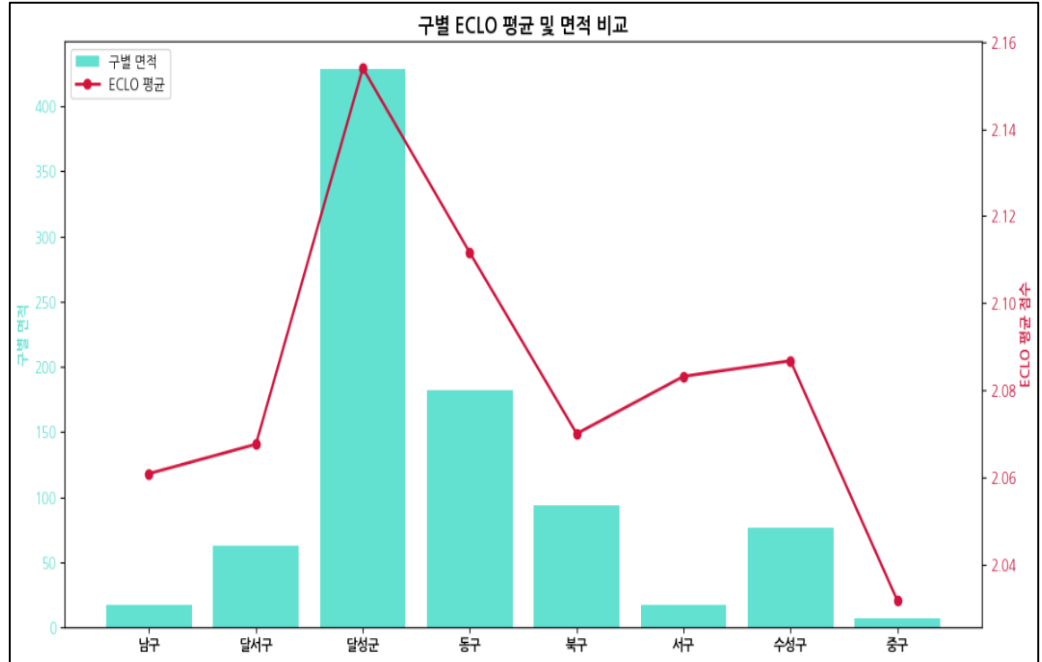


〈구에 따른 ECLO 변화 ANOVA 분석〉		
지표	값	해석
F-statistic	10.57	집단 간 차이 어느 정도 존재
P-value	0.000	매우 유의함

★ 주요 인사이트

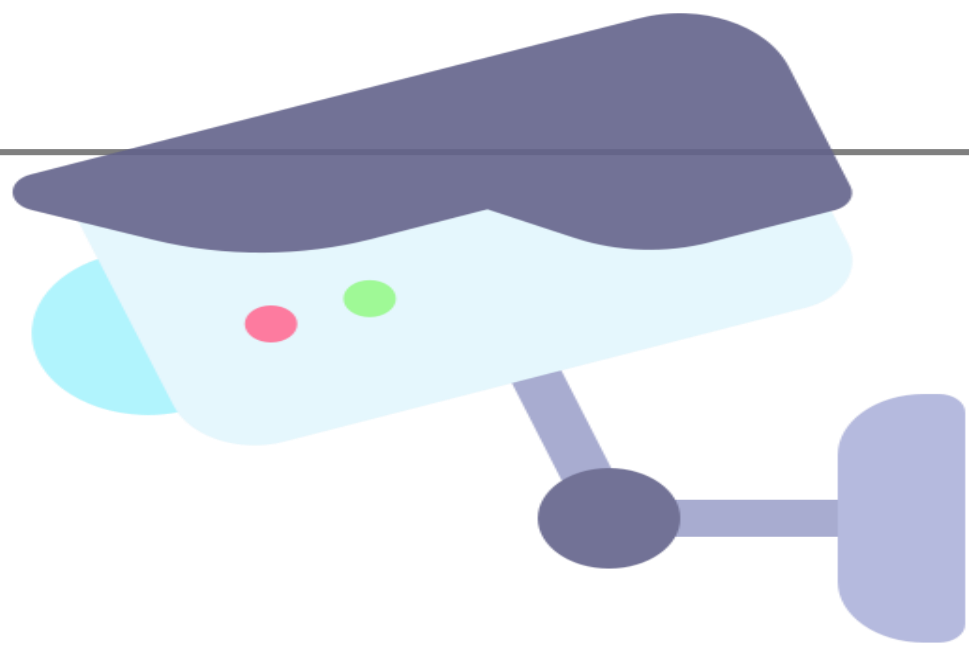
1. 외곽 지역 및 면적이 넓은 지역(달성군, 동구)의 높은 사고 심각도
 - 두 지역 모두 대구시 외곽에 위치하며, **속도 제한이 높은 도로**(ex. 터널, 고속도로)가 다수 분포할 가능성이 큼
 - 또한, 넓은 면적을 보유한 지역 특성상 **고속 주행 빈도 증가와 응급 대응 지연**으로 인해 **고위험·고심각도 사고** 발생 가능성이 높은 경향을 보임
2. 도심·주거 혼합 지역(달서구, 수성구, 북구)의 높은 사고 발생 빈도
 - 교통량 집중 특성 지역으로 인해 사고 발생 빈도가 높게 측정
 - 하지만, 도심 지역 특성상 고속 주행 빈도가 낮기 때문에 전반적으로 사고 심각도는 높지 않음

✓ 구별 ECLO 평균 및 면적 비교



상관분석 [사고 발생 빈도 - ECLO]		상관분석 [구별 면적 - ECLO]	
상관계수	P-value	상관계수	P-value
-0.0114	0.0232	0.0387	0.00

3. ECLO와 발생 빈도 간 상관성은 미약, 면적 간 상관성은 강세
 - ECLO와 사고 발생 빈도는 음의 상관을 나타낸다. 이는 사고 발생 빈도가 높더라도 반드시 ECLO가 높은 것은 아님을 의미
 - 반면, **면적과 ECLO는 뚜렷한 양의 상관관계**를 보이며, **지역 면적이 클수록 고위험·고심각도 사고 발생 가능성이 높음**을 시사한다.
4. 구별 ECLO 점수 간 유의미한 통계적 차이
 - ANOVA 분석 결과, 구별 ECLO 점수 차이는 통계적으로 **매우 유의미**하며, 이는 **지역(구) 요인이 ECLO에 일정한 영향을 미칠 가능성**을 시사한다.

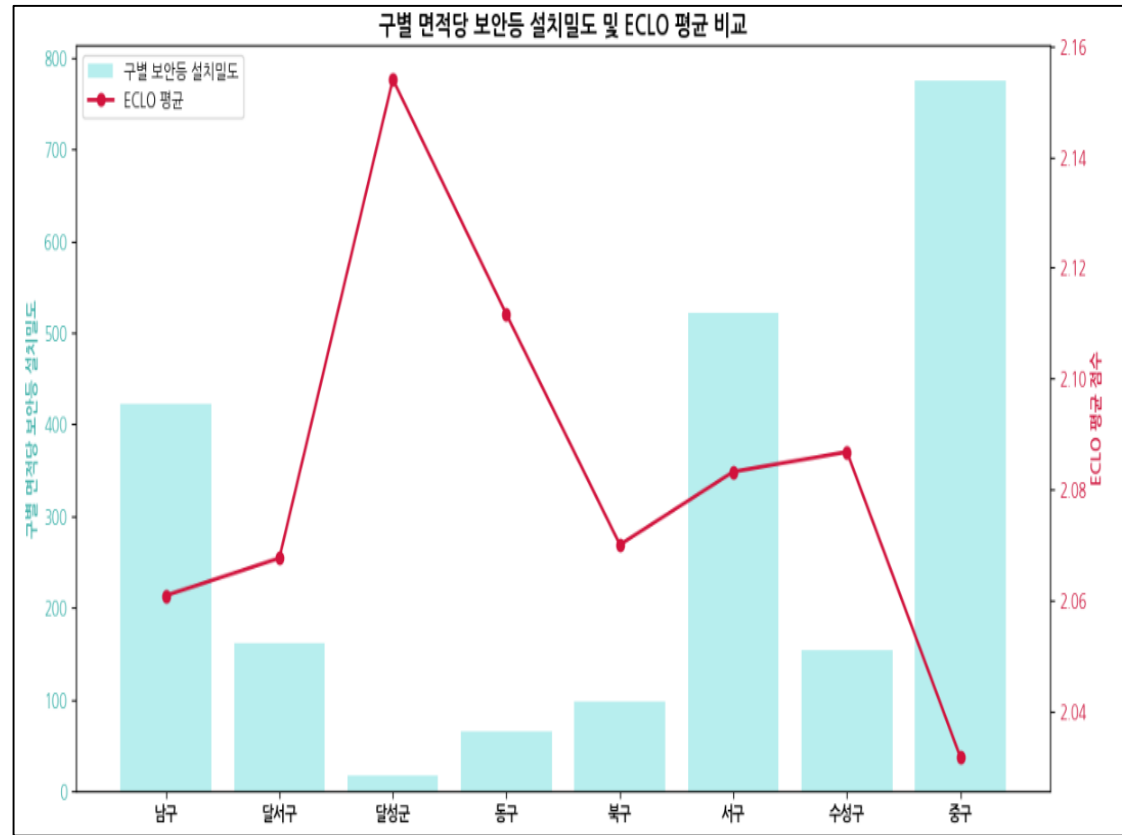


◆ 공간적 정보 - 구가 ECLO에 미치는 영향 분석

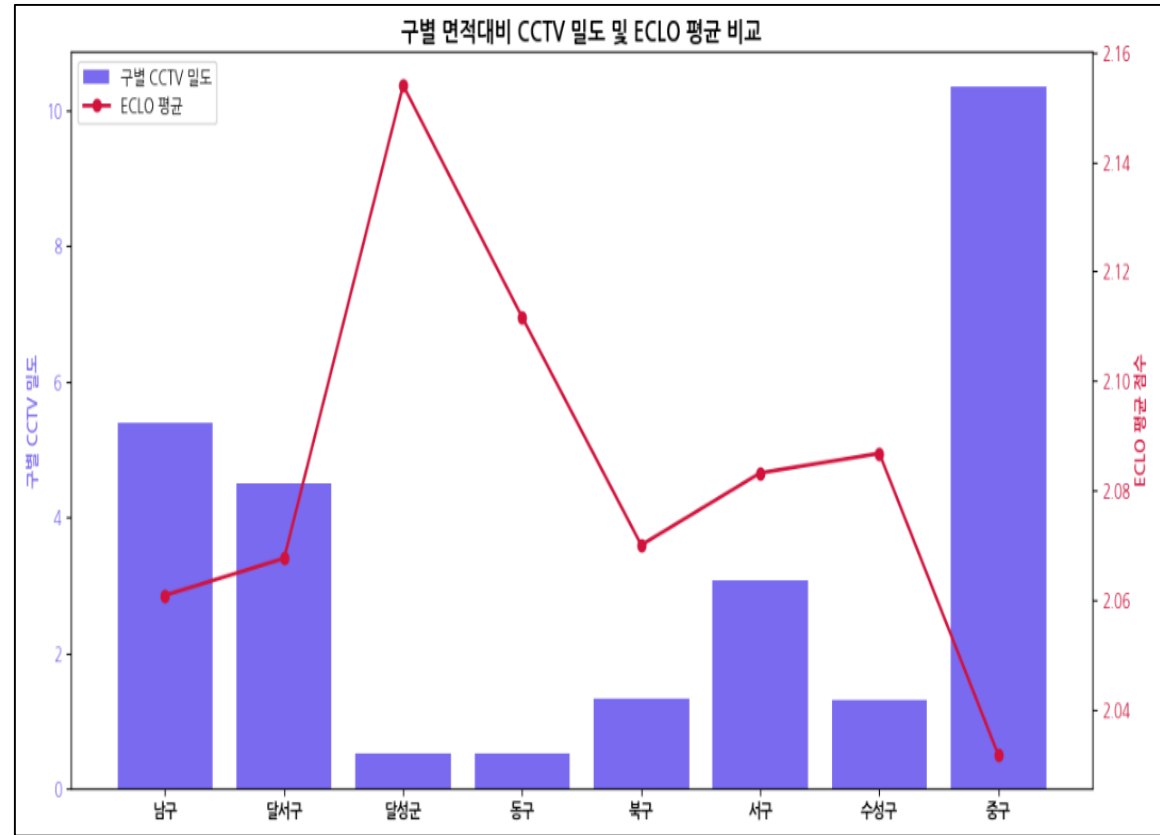
! 구별 CCTV 및 보안등 설치 밀도에 따른 ECLO와의 관계성 분석

- 대구시 CCTV 및 보안등 설치 위치 외부 데이터를 수집 후, 구별 설치 개수 전처리 수행
- 구 면적이 사고 위험도와 유의미한 상관성을 내포하고 있기 때문에, 면적을 고려한 구별 CCTV 및 보안등 설치 밀도를 계산해 ECLO에 미치는 영향 분석
- 가설 설정 : 지역(구) 단위에서 **면적 대비 보안등 및 CCTV 설치 밀도가 높을 경우**, 그렇지 않은 지역에 비해 **ECLO 점수가 유의하게 낮게 나타날 것이다.**

✓ 면적대비 보안등 설치 밀도와 ECLO 평균 점수 비교



✓ 면적대비 CCTV 설치 밀도와 ECLO 평균 점수 비교



★ 분석 인사이트

- ! 보안등 및 CCTV 모두 설치 밀도가 낮은 지역(구)일수록 ECLO 평균이 상대적으로 높게 나타나는 경향이 관찰
- ! 두 변수 모두 ECLO와 통계적으로 유의한 음(-)의 상관관계를 보여, **설치 밀도가 높을수록** 해당 지역의 사고 위험도가 낮게 형성됨을 시사한다.
- ! 이는 사전에 설정한 가설(설치 밀도 ↑ / ECLO ↓)을 **지지하는 결과**로 해석 가능
- ! 따라서, **구별 면적·보안등 설치 밀도·CCTV 설치 밀도**와 같이 ECLO에 유의미한 영향을 미치는 지역(구) 특성 변수를 **파생 변수**로 반영하여, **모델의 예측 성능 개선 가능성**을 검증할 필요가 있다.

✓ 상관 분석

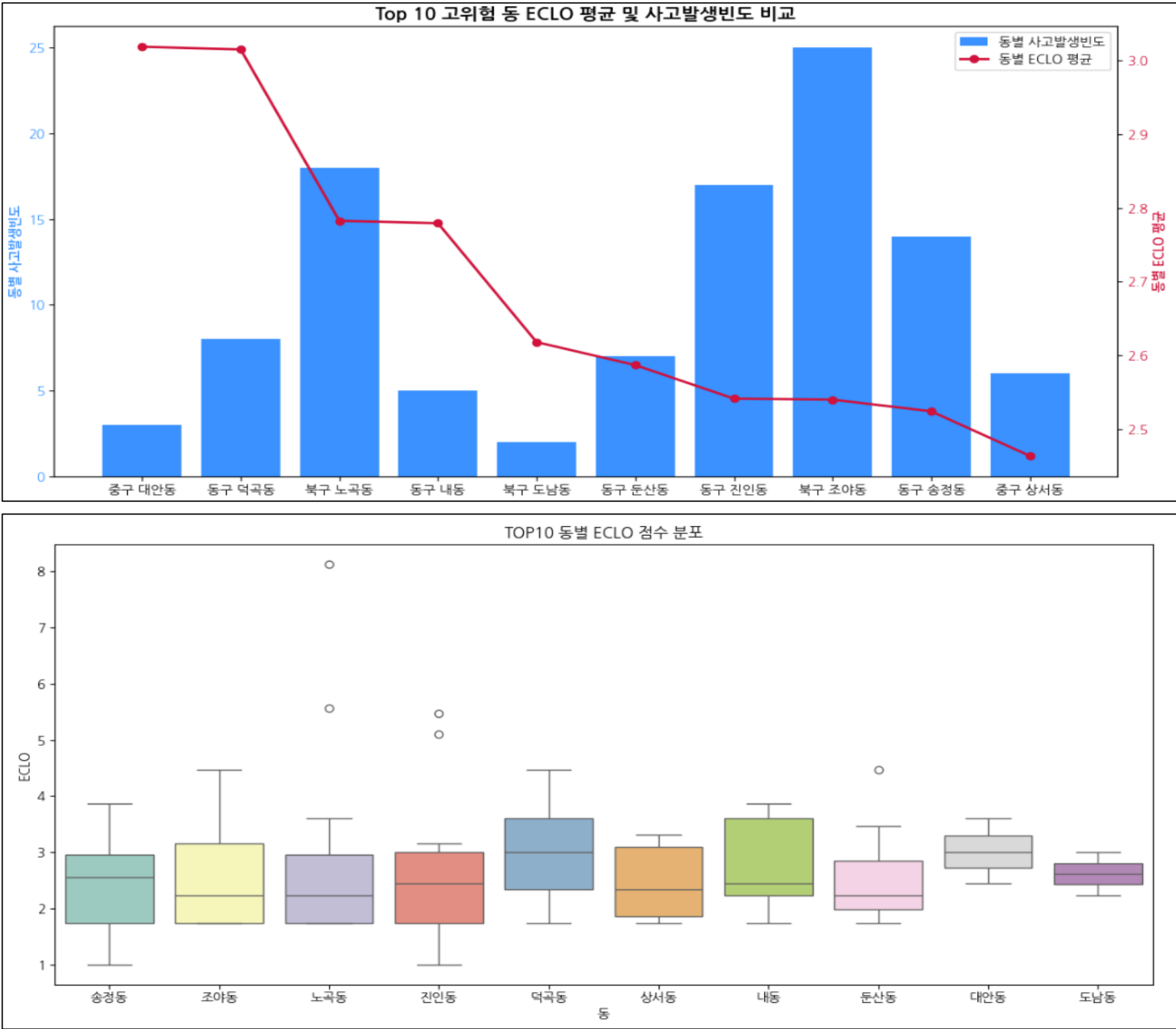
구분	상관계수	P-value
보안등 설치 밀도 - ECLO	-0.0249	0.000
CCTV 설치 밀도 - ECLO	-0.0320	0.000

◆ 공간적 정보

-동이 ECLO에 미치는 영향 분석

✓ 고위험 동에 대한 ECLO 평균 및 분포

→ 동별 ECLO 평균 기준 상위 10개 동에 대한 ECLO 평균 및 분포 시각화



✓ 회귀분석

→ 모든 피처에 대해 ECLO를 종속변수로 한 타깃 인코딩을 수행하여 동일한 기준으로 수치화한 뒤, ‘동’ 변수가 다른 변수 대비 ECLO에 미치는 상대적 중요도를 회귀분석을 통해 평가

OLS Regression Results						
=====						
Dep. Variable:	ECLO	R-squared:	0.044			
Model:	OLS	Adj. R-squared:	0.044			
Method:	Least Squares	F-statistic:	152.8			
Date:	Sat, 31 May 2025	Prob (F-statistic):	0.00			
Time:	07:32:56	Log-Likelihood:	-36718.			
No. Observations:	39609	AIC:	7.346e+04			
Df Residuals:	39596	BIC:	7.357e+04			
Df Model:	12					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	-15.5765	1.078	-14.452	0.000	-17.689	-13.464
연	1.0091	0.174	5.808	0.000	0.669	1.350
월	0.9289	0.177	5.256	0.000	0.582	1.275
일	0.9455	0.167	5.665	0.000	0.618	1.273
시간	1.1076	0.091	12.230	0.000	0.930	1.285
요일	0.9082	0.082	11.025	0.000	0.747	1.070
기상상태	-0.0191	0.582	-0.033	0.974	-1.160	1.122
사고유형	0.8792	0.035	24.818	0.000	0.810	0.949
도로형태1	0.4527	0.089	5.087	0.000	0.278	0.627
도로형태2	0.3673	0.075	4.884	0.000	0.220	0.515
노면상태	0.8579	0.457	1.878	0.060	-0.038	1.754
구	0.0313	0.123	0.254	0.800	-0.211	0.273
동	1.0125	0.050	20.206	0.000	0.914	1.111

★ 동 변수의 ECLO 영향 분석

- ✓ ‘동’ 변수는 회귀계수(coef) 1.1025, p-value 0.000으로 통계적으로 ECLO에 매우 유의하며, t-값 20.206으로 영향력의 신뢰성이 높게 나타났다.
- ✓ 이는 ‘동’ 특성이 ECLO에 강한 양(+)의 영향을 미치며, 지역별 미시적 요인이 사고 위험도에 중요한 설명력을 가짐을 시사한다.
- ✓ 다만, ‘동’은 범주 수가 많아 면적·교통량 등 세부 지역 특성 파악이 제한적이므로, 다변량 군집 분석을 통한 동별 위험도 분류와 같은 방법으로 고위험 지역 효과를 부각하는 방안을 고려해볼 필요가 있다.

◆ 공간적 정보

-동이 ECLO에 미치는 영향 분석



다변량 군집 분석을 통한 동별 위험도 구분

동별 ECLO 통계량을 기반으로 K-Means 다변량 군집분석을 수행하여 동별 위험도를 구분하고, 해당 위험도 구분 방식이 ECLO에 미치는 영향을 분석

◆ 군집 분석 절차

1. 동별 ECLO 통계량 계산 및 표준화 수행

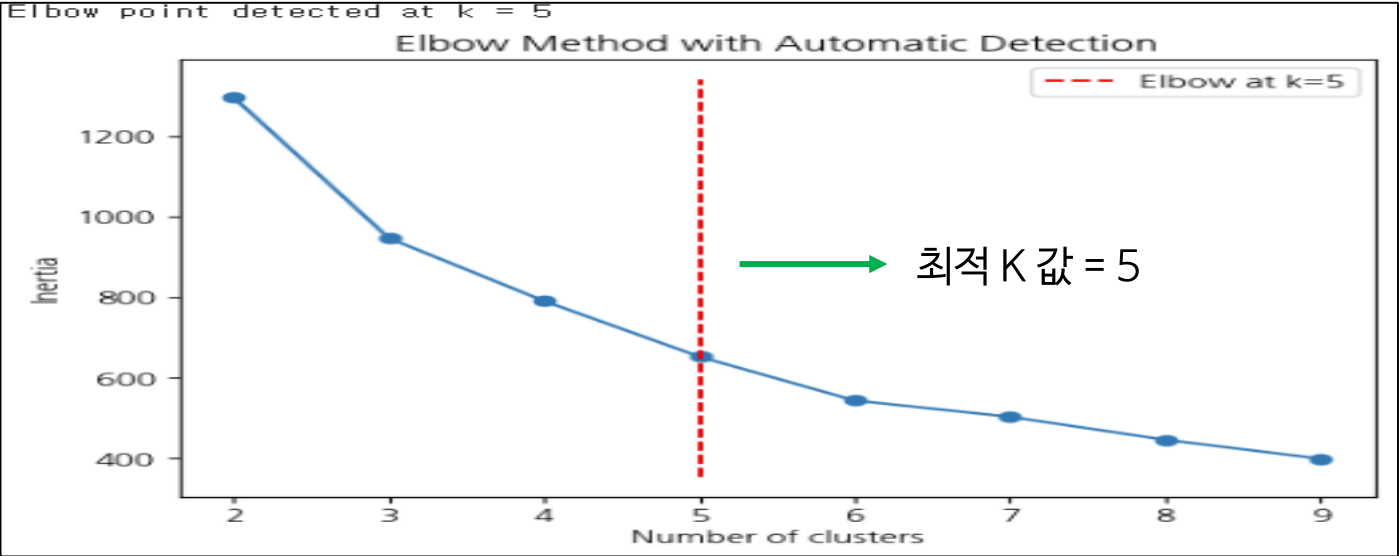
	동	count	mean	std	min	25%	50%	75%	max
0	대안동	3.0	3.018347	0.578249	2.449490	2.724745	3.000000	3.302776	3.605551
1	덕곡동	8.0	3.014722	0.948050	1.732051	2.337117	3.000000	3.598076	4.472136
2	노곡동	18.0	2.782520	1.656298	1.732051	1.732051	2.236068	2.957107	8.124038
3	내동	5.0	2.779229	0.919163	1.732051	2.236068	2.449490	3.605551	3.872983
4	도남동	2.0	2.618034	0.540182	2.236068	2.427051	2.618034	2.809017	3.000000
...	...	...	...	...	...	...	...	...	...
191	수창동	28.0	1.762688	0.544401	1.000000	1.549038	1.732051	2.236068	2.828427
192	북성로1가	18.0	1.728725	0.604258	1.000000	1.103553	1.732051	2.110064	2.828427
193	화전동	3.0	1.727180	0.724757	1.000000	1.366025	1.732051	2.090770	2.449490
194	종로2가	4.0	1.549038	0.366025	1.000000	1.549038	1.732051	1.732051	1.732051
195	동일동	3.0	1.488034	0.422650	1.000000	1.366025	1.732051	1.732051	1.732051

196 rows × 9 columns

동별 통계량 표준화

array([[-0.72998086, 4.46429912, -0.32334294, ..., 4.10117032, 3.28554698, -0.61403821],
 [-0.71164762, 4.44618063, 1.7365664 , ..., 4.10117032, 4.38257411, 0.11498256],
 [-0.67498115, 3.28550217, 5.68173003, ..., 1.20377021, 2.00140414, 3.1871725],
 ...,
 [-0.72998086, -1.98967464, 0.49275279, ..., -0.70783889, -1.21699345, -1.58658353],
 [-0.72631421, -2.88012835, -1.5054955 , ..., -0.70783889, -2.54961867, -2.19013434],
 [-0.72998086, -3.18506162, -1.19008024, ..., -0.70783889, -2.54961867, -2.19013434]])

2. Elbow Method를 활용한 최적 군집 수 탐색



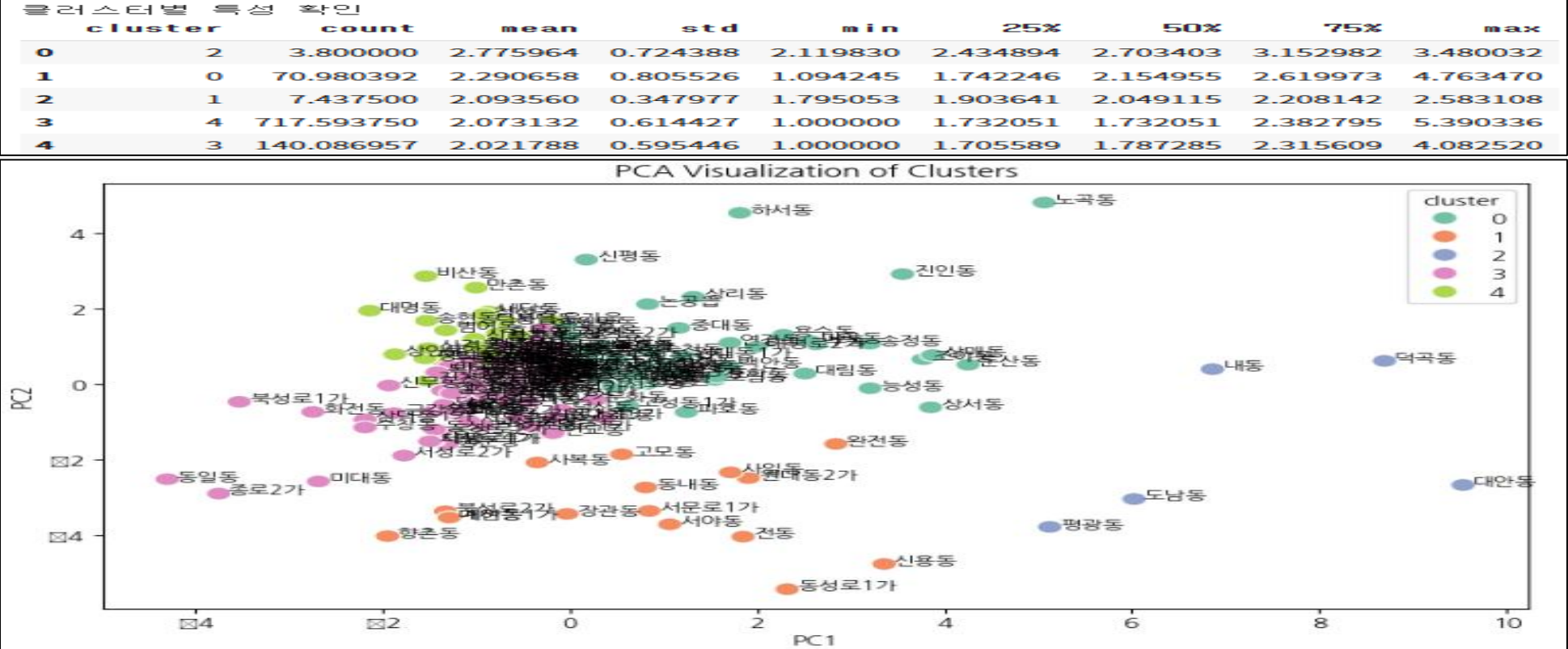
3. 최적 K값 활용 K-Means 클러스터링 수행

클러스터 확인

	동	count	mean	std	min	25%	50%	75%	max	cluster	PC1	PC2
0	대안동	3.0	3.018347	0.578249	2.449490	2.724745	3.000000	3.302776	3.605551	2	9.532795	-2.670101
1	덕곡동	8.0	3.014722	0.948050	1.732051	2.337117	3.000000	3.598076	4.472136	2	8.693334	0.616668
2	노곡동	18.0	2.782520	1.656298	1.732051	1.732051	2.236068	2.957107	8.124038	0	5.063609	4.812441
3	내동	5.0	2.779229	0.919163	1.732051	2.236068	2.449490	3.605551	3.872983	2	6.859256	0.397022
4	도남동	2.0	2.618034	0.540182	2.236068	2.427051	2.618034	2.809017	3.000000	2	6.023204	-3.042256
...	...	...	...	...	...	...	...	...	...	...	...	...
191	수창동	28.0	1.762688	0.544401	1.000000	1.549038	1.732051	2.236068	2.828427	3	2.182059	-1.147265
192	북성로1가	18.0	1.728725	0.604258	1.000000	1.103553	1.732051	2.110064	2.828427	3	3.522472	-0.466091
193	화전동	3.0	1.727180	0.724757	1.000000	1.366025	1.732051	2.090770	2.449490	3	2.737260	-0.735489
194	종로2가	4.0	1.549038	0.366025	1.000000	1.549038	1.732051	1.732051	1.732051	3	3.741493	-2.898321
195	동일동	3.0	1.488034	0.422650	1.000000	1.366025	1.732051	1.732051	1.732051	3	4.290531	-2.515786

196 rows × 12 columns

4. 클러스터별 특성 요약 및 PCA 활용 시각화

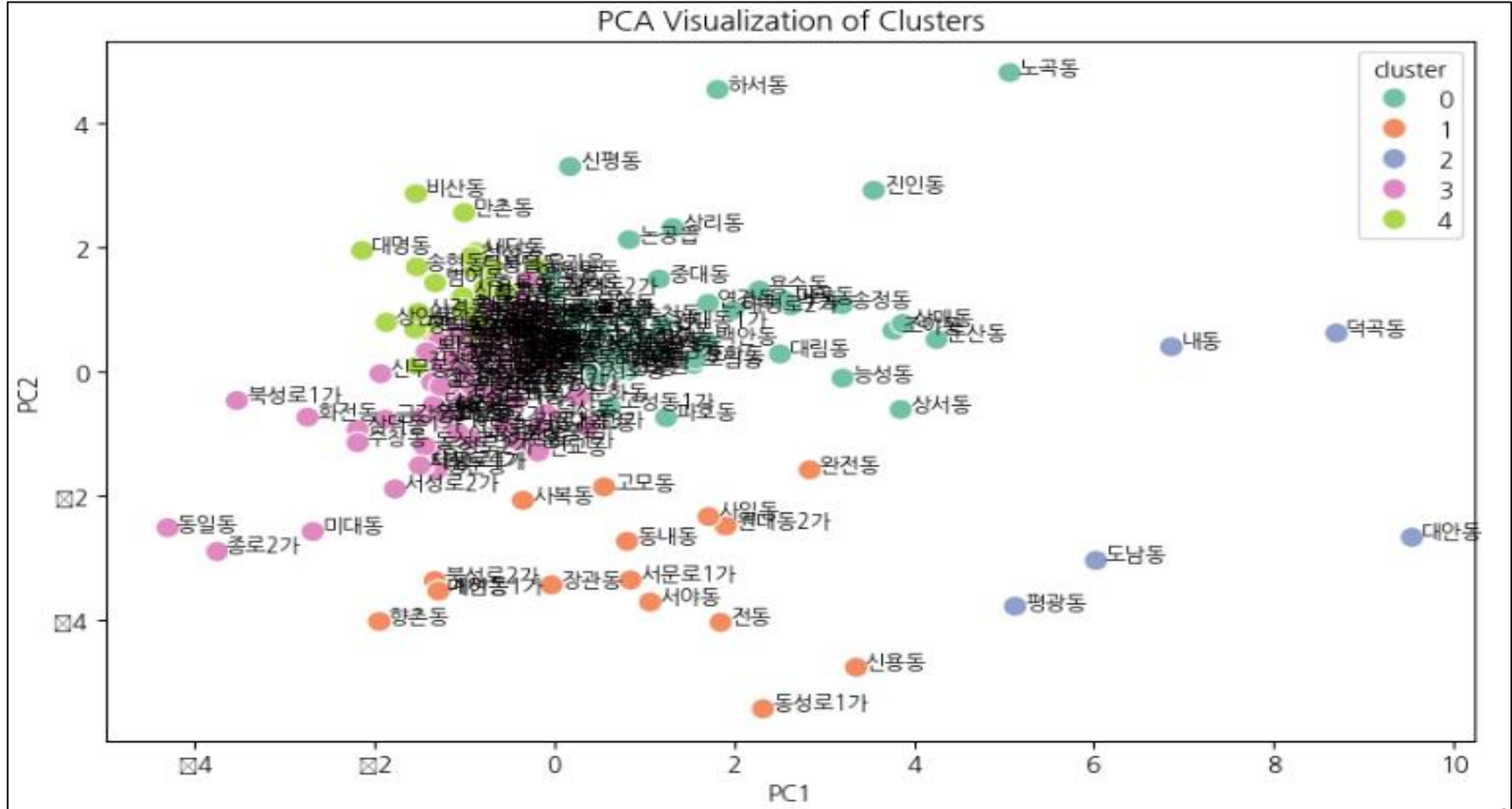


◆ 공간적 정보

-동이 ECLO에 미치는 영향 분석

✓ 동별 위험도 군집 해석

- 군집별 ECLO 통계 요약 및 PCA 차원 축소 동별 군집 분포도



클러스터별 특성 확인									
	cluster	count	mean	std	min	25%	50%	75%	max
0	2	3.800000	2.775964	0.724388	2.119830	2.434894	2.703403	3.152982	3.480032
1	0	70.980392	2.290658	0.805526	1.094245	1.742246	2.154955	2.619973	4.763470
2	1	7.437500	2.093560	0.347977	1.795053	1.903641	2.049115	2.208142	2.583108
3	4	717.593750	2.073132	0.614427	1.000000	1.732051	1.732051	2.382795	5.390336
4	3	140.086957	2.021788	0.595446	1.000000	1.705589	1.787285	2.315609	4.082520

✓ 위험도 수준 설정

군집	위험도 구분	설명
2	고위험	전체 군집 중 평균 ECLO가 가장 높음. 전반적으로 고위험 사고가 주로 분포되어 있는 동을 포함함
0	중위험	ECLO 평균이 두번째로 높으며, 표준편차가 커서 극단적으로 높고 낮은 사고가 분포하는 군집
1	일반	평균이 중간 수준이며, 표준편차가 작아 군집 내 위험도 편차가 적음
4	저위험	평균이 낮은 편이지만, 일부 동에서는 극단적으로 높은 값(최대 5.39) 존재
3	극저위험	평균이 가장 낮으며, 편차도 작음. 대부분 동에서 안정적인 저위험 수준

✓ 군집에 따른 ECLO 점수 차이 ANOVA 분석

지표	값	해석
F-statistic	63.923	군집 간 변동이 군집 내 변동보다 매우 크다는 것을 의미
P-value	5.79×10^{-54}	군집 간 ECLO 평균에 통계적으로 유의미한 차이가 있음

★ 동별 위험도 구분 분석 결과

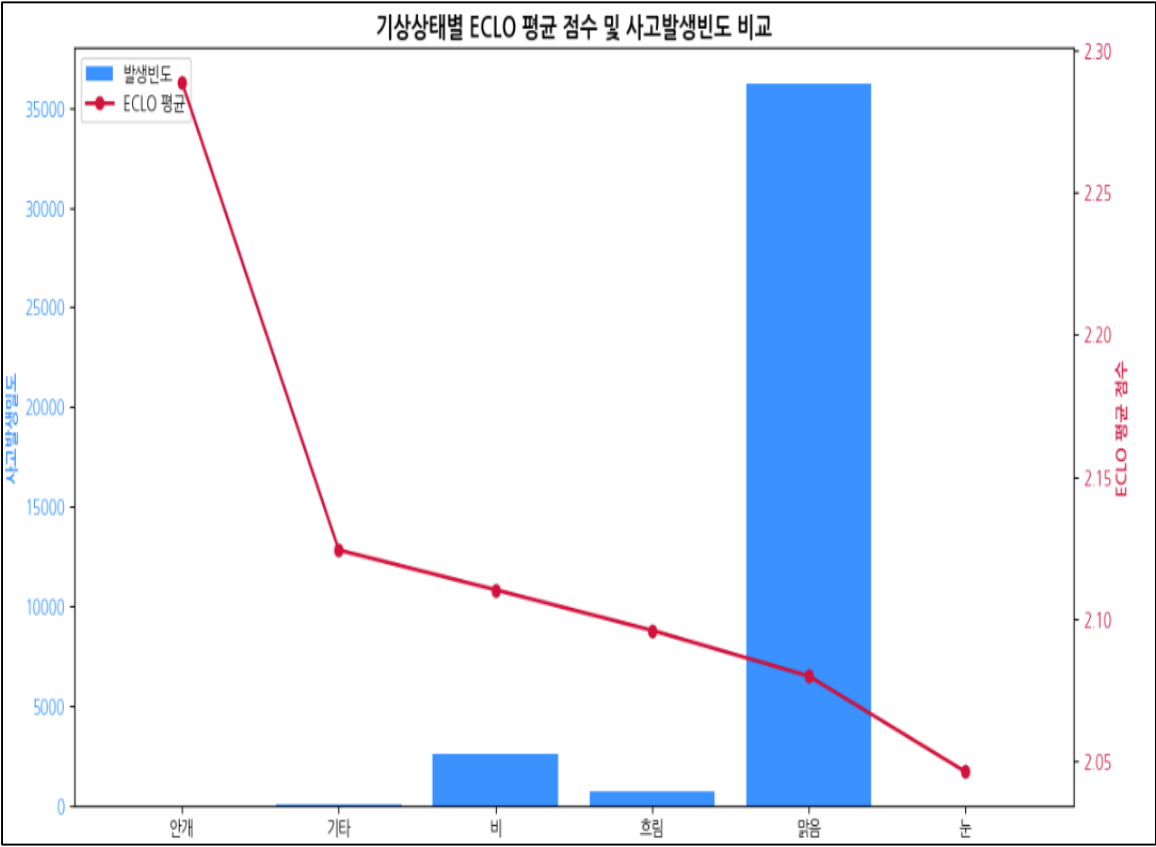
동별 ECLO 통계량을 기반으로 수행한 위험도 군집분석 결과, ANOVA 검정에서 **군집 간 ECLO 점수의 차이가 통계적으로 매우 유의**하게 나타났다.

이는 군집별로 사고 위험도의 분포가 뚜렷하게 상이함을 의미하며, 본 분석을 통해 도출된 군집 수준은 **동별 사고 위험도를 효과적으로 구분**할 수 있음을 시사한다.

따라서 해당 파생변수를 모델에 추가하는 방식은 **지역적 위험도의 설명력을 강화**하고, 사고 위험 **예측의 정밀도를 향상**시키는 데 기여할 수 있을 것으로 예상된다.

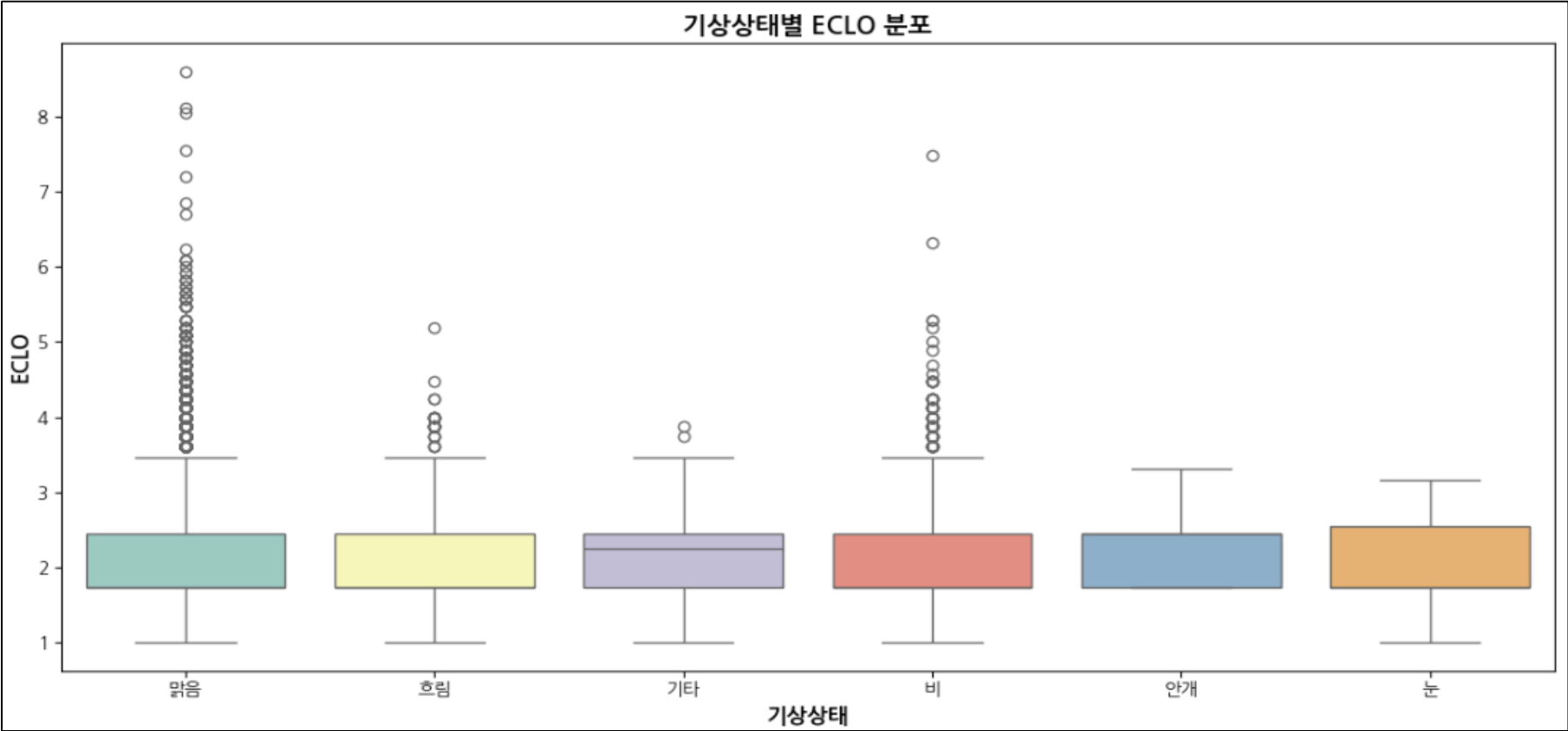
◆ 환경적 정보 - 기상상태

기상상태별 ECLO 평균 및 사고발생빈도 비교 및 분포 시각화



〈기상상태별 ECLO 통계〉

기상상태	ECLO 평균	사고 발생 빈도
안개	2.288842	8
기타	2.124175	56
비	2.110176	2627
흐림	2.095863	729
맑음	2.079780	36181
눈	2.046050	8



회귀분석 및 ANOVA 분석

OLS Regression Results						
=====						
Dep. Variable:	ECLO	R-squared:	0.044			
Model:	OLS	Adj. R-squared:	0.044			
Method:	Least Squares	F-statistic:	152.8			
Date:	Sat, 16 Aug 2025	Prob (F-statistic):	0.00			
Time:	05:18:39	Log-Likelihood:	-36718.			
No. Observations:	39609	AIC:	7.346e+04			
Df Residuals:	39596	BIC:	7.357e+04			
Df Model:	12					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]
const	-15.5765	1.078	-14.452	0.000	-17.689	-13.464
연	1.0091	0.174	5.808	0.000	0.669	1.350
월	0.9289	0.177	5.256	0.000	0.582	1.275
일	0.9455	0.167	5.665	0.000	0.618	1.273
시간	1.1076	0.091	12.230	0.000	0.930	1.285
요일	0.9082	0.082	11.025	0.000	0.747	1.070
기상상태	-0.0191	0.582	-0.033	0.974	-1.160	1.122
사고유형	0.8732	0.035	24.818	0.000	0.810	0.949
도로형태1	0.4527	0.089	5.087	0.000	0.278	0.627
도로형태2	0.3673	0.075	4.884	0.000	0.220	0.515
노면상태	0.8579	0.457	1.878	0.060	-0.038	1.754
구동	0.0313	0.123	0.254	0.800	-0.211	0.273
동	1.0125	0.050	20.206	0.000	0.914	1.111
=====						

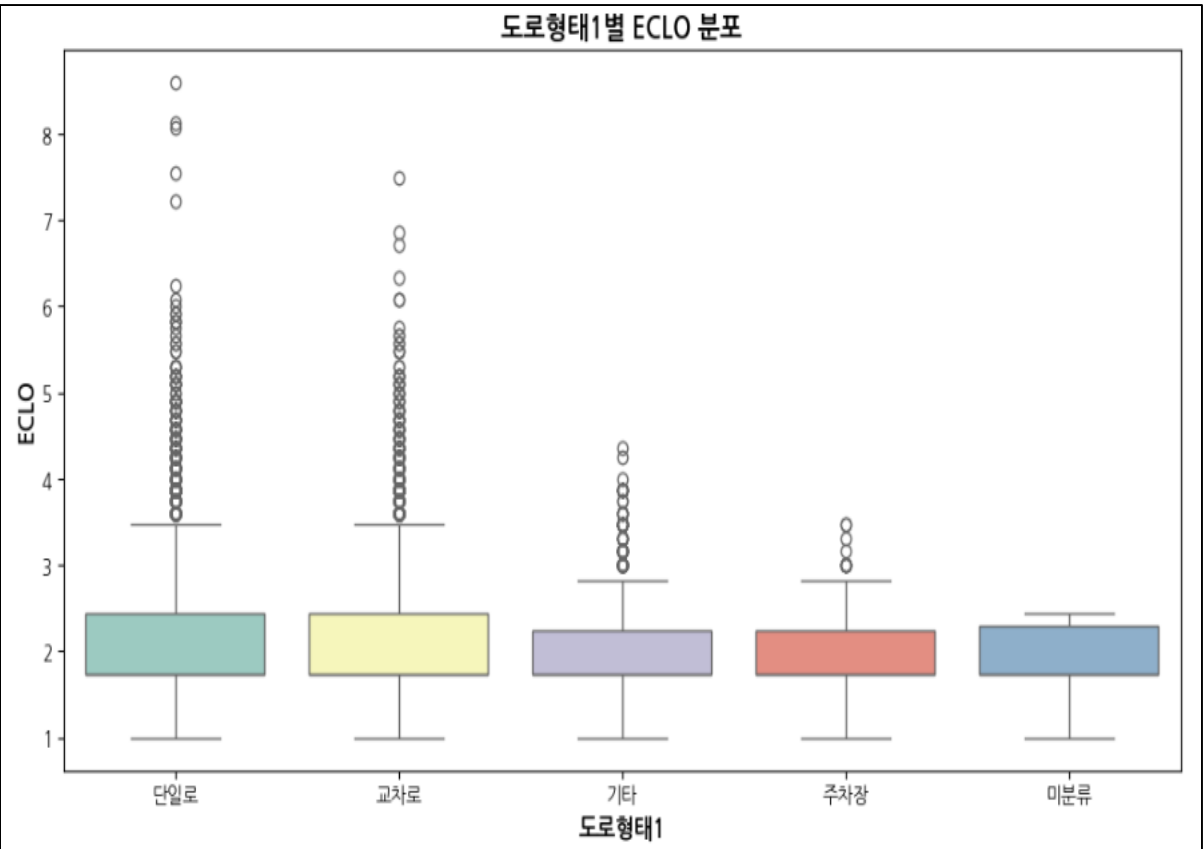
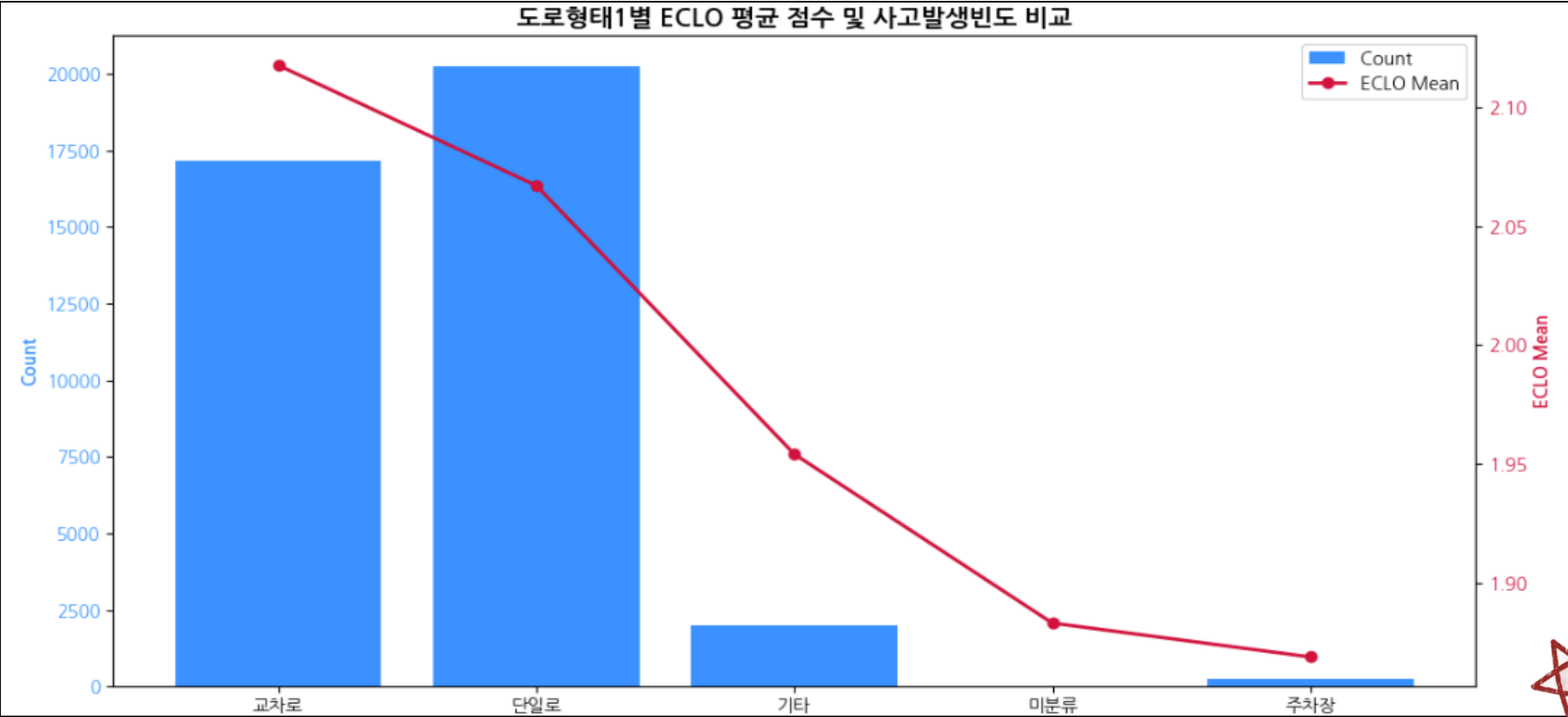
ANOVA 분석 지표	값	해석
F-statistic	1.460	집단 간 ECLO의 차이가 크지 않음
P-value	0.1995	P-value > 0.05으로, 기상상태에 따른 ECLO 점수의 차이는 유의하지 않음

기상상태가 ECLO에 미치는 영향 분석 결과

- 기상상태별 평균 ECLO 차이
 - 기상상태가 **안개, 기타, 비, 흐림**일 때 상대적으로 사고 심각도가 다소 높게 나타났다.
 - 특히 **안개**의 경우 평균 ECLO 점수가 가장 높았으며, 그 뒤로 **기타 → 비 → 흐림** 순으로 높은 값을 보임
 - 다만 안개 조건은 사고 건수가 **8건**에 불과해, 일부 고위험 사고로 평균치가 높아진 결과로 볼 수 있다.
- 통계적 유의성 검정 결과
 - 회귀분석 및 ANOVA 검정에서 **기상상태 변수의 p-value가 0.05를 상회**하여, ECLO에 대한 영향은 통계적으로 유의하지 않은것으로 확인되었다.
 - 이는 **대부분의 사고가 맑은 날 발생**하며, 기상상태 범주 간 ECLO 점수 차이가 뚜렷하지 않음을 시사한다.
- 결론 및 시사점
 - 이상의 결과를 종합하면, **기상상태는 교통사고 심각도에 본질적으로 큰 영향을 미치지 않는 변수**로 판단된다.
 - 따라서 모델 학습 효율성과 해석력을 높이기 위해, **기상상태 변수를 제거하거나 중요도가 낮은 보조 변수로 처리하는 방안**을 고려해볼 필요가 있다.

◆ 환경적 정보 - 도로형태1

✓ 도로형태1별 ECLO 평균 및 사고발생빈도 비교 및 분포 시각화



〈도로형태1별 ECLO 통계〉

기상상태	ECLO 평균	사고 발생 수
교차로	2.117803	17151
단일로	2.067146	20228
기타	1.954058	1986
미분류	1.882906	8
주차장	1.868637	236

✓ 도로형태1에 따른 ECLO 점수 차이 ANOVA 분석

ANOVA 분석 지표	값	해석
F-statistic	44.9437	범주 간 ECLO 점수의 차이가 매우 크게 나타남
P-value	1.01698e-37	P-value가 매우 작음으로 도로형태1 범주 간 ECLO 점수의 차이는 통계적으로 매우 유의함

✓ Tukey HSD 사후검정

• 사후 검정을 통한 도로형태1 범주 간 유의미하지 않은 조합 확인

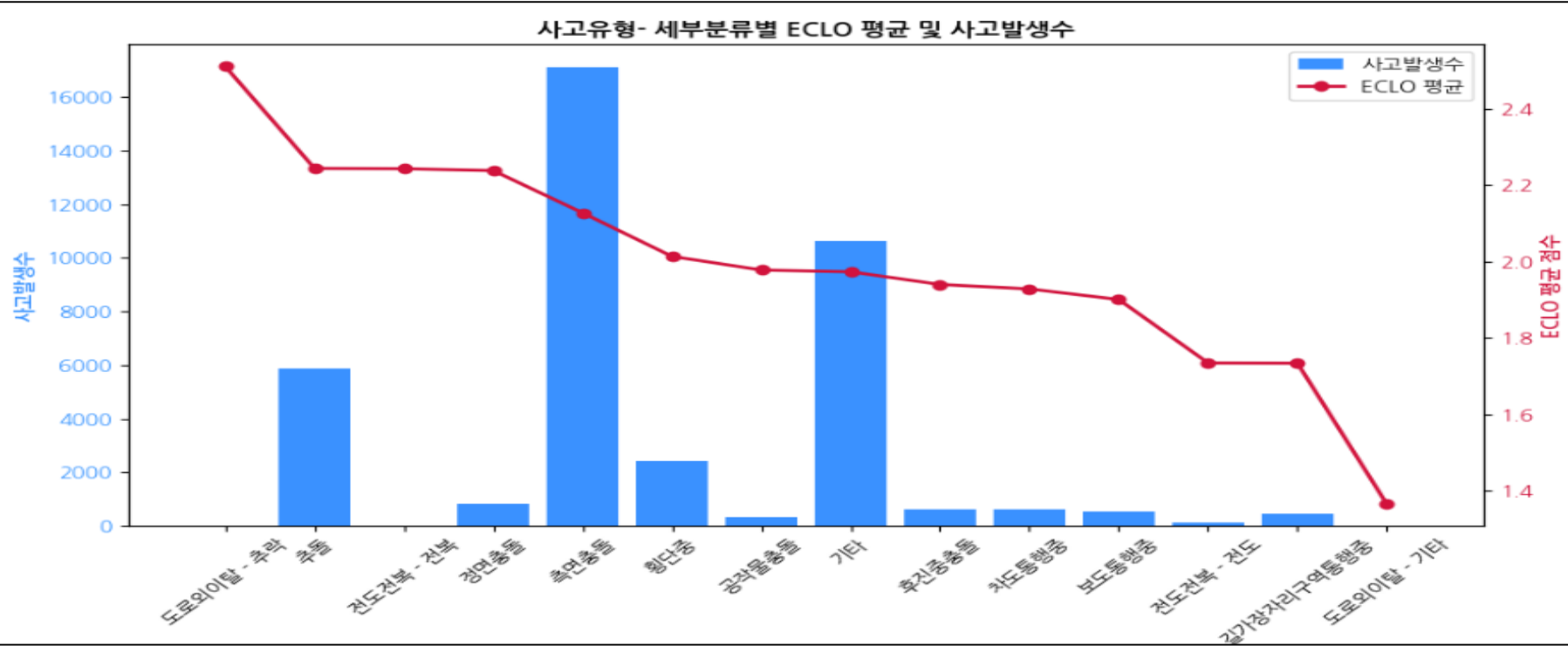
그룹1	그룹2	meandiff	P-adj	lower	upper	reject
교차로	미분류	-0.2349	0.8248	-0.8369	0.3671	False
기타	미분류	-0.0712	0.9977	-0.6743	0.5320	False
기타	주차장	-0.0854	0.2718	-0.2026	0.0318	False
단일로	미분류	-0.1842	0.9198	-0.7863	0.4178	False
미분류	주차장	-0.0143	1.0000	-0.6263	0.5977	False

✓ 도로형태1이 ECLO에 미치는 영향 분석

- 교차로 및 단일로에서의 사고 심각도 특성
 - 분석 결과, **교차로와 단일로 구간에서 사고 발생 빈도가 높고 평균 ECLO역시 상대적으로 높은 수준**을 보였다. 이는 해당 도로 형태가 **심각도가 큰 사고 발생과 밀접한 관련**이 있음을 시사한다.
- 도로형태1 범주 간 ECLO 차이의 통계적 유의성
 - ANOVA 분석 결과, **도로형태1 범주 간 ECLO 점수의 차이는 통계적으로 매우 유의미($p < 0.001$)**한 것으로 나타났다. 이는 도로형태 유형에 따라 **사고 심각도의 분포가 뚜렷하게 달라진다는 점**을 의미한다.
- 영향력이 낮은 범주의 결함 필요성
 - 사후 검정(Tukey HSD) 결과, **미분류 및 주차장 범주**는 다른 모든 범주와의 비교에서 ECLO 점수 차이가 통계적으로 유의하지 않은 것으로 나타났다. 또한 이들 범주는 **사고 발생 수와 평균 ECLO 모두 낮은 수준**을 보였다.
- 결론 및 시사점
 - 종합적으로 볼 때, **도로형태1 변수는 ECLO에 매우 유의미한 영향을 미치는 핵심 요인**으로 판단된다. 따라서 모델링 시 **영향력이 낮은 범주(미분류, 주차장)를 통합하여 변수의 설명력을 강화**하는 방안을 고려할 수 있으며, 이는 **예측 성능 개선**에 기여할 것으로 기대된다.

◆ 훈련데이터에만 존재하는 변수 - 사고유형 (세부분류)

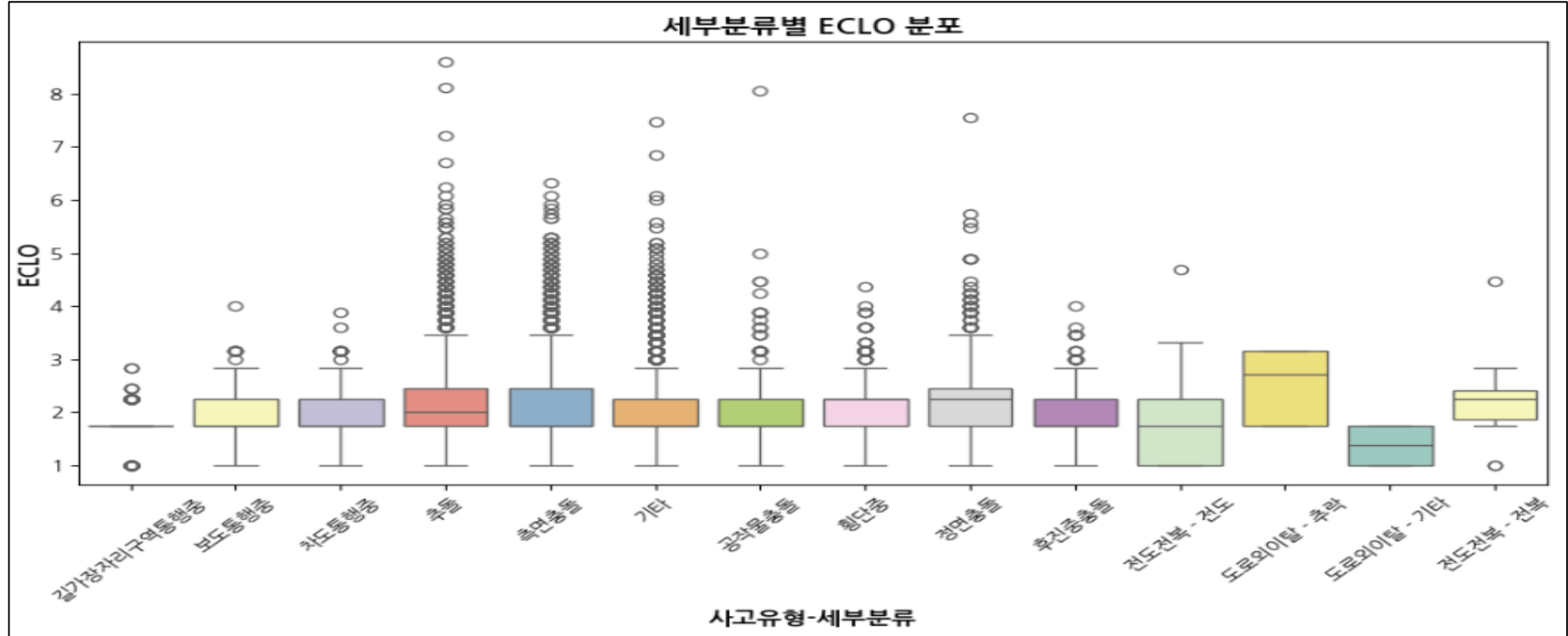
✔ 사고유형-세부분류에 따른 ECLO 평균 및 사고발생빈도 비교



✔ 사고유형-세부분류에 따른 ECLO 통계량

사고유형-세부분류	Count	Mean	std	Min	25%	50%	75%	max
도로외이탈 - 추락	8	2.510166	0.716392	1.732051	1.732051	2.699173	3.162278	3.162278
추돌	5885	2.243383	0.734941	1.000000	1.732051	2.000000	2.449490	8.602325
전도전복 - 전복	10	2.242638	0.983518	1.000000	1.858055	2.236068	2.396134	4.472136
정면충돌	837	2.237771	0.730097	1.000000	1.732051	2.236068	2.449490	7.549834
측면충돌	17104	2.127143	0.627570	1.000000	1.732051	1.732051	2.449490	6.324555
횡단충	2443	2.013433	0.416831	1.000000	1.732051	2.236068	2.236068	4.358899
공작물충돌	324	1.977848	0.837493	1.000000	1.732051	1.732051	2.236068	8.062258
기타	10630	1.973194	0.577200	1.000000	1.732051	1.732051	2.236068	7.483315
후진충돌	613	1.940052	0.438712	1.000000	1.732051	1.732051	2.236068	4.000000
차도통행중	616	1.928348	0.435028	1.000000	1.732051	1.732051	2.236068	3.872983
보도통행중	524	1.900792	0.416629	1.000000	1.732051	1.732051	2.236068	4.000000
전도전복 - 전도	144	1.734850	0.682461	1.000000	1.000000	1.732051	2.236068	4.690416
길가장자리구역통행중	467	1.734259	0.380382	1.000000	1.732051	1.732051	1.732051	2.828427
도로외이탈 - 기타	4	1.366025	0.422650	1.000000	1.000000	1.366025	1.732051	1.732051

✔ 사고유형-세부분류별 ECLO 분포



! 사고유형-세부분류별 ECLO 특성 분석

- 도로외이탈 - 추락은 평균 ECLO가 가장 높은 범주로 나타났다. 비록 사고 발생 건수는 8건에 불과하지만, 평균점수가 두드러지게 높아 치명적 사고를 유발하는 고위험 범주로 해석할 수 있다..
- 추돌은 도로외이탈 - 추락 다음으로 높은 평균 ECLO를 기록하였다. 특히 5,800건 이상의 사고가 발생하였으며, 분포 역시 높은 ECLO 구간에 집중되는 양상을 보여, 보다 일반화된 고위험 사고 유형으로 판단된다.
- 이외에도 전복 → 정면충돌 → 측면충돌 순으로 높은 평균 ECLO가 확인되었으며, 이를 통해 충돌 형태의 사고가 상대적으로 높은 심각도(ECLO)를 유발함을 알 수 있다.
- 다만, 해당 변수(사고유형 - 세부분류)가 ECLO에 대해 통계적으로 유의미한 영향을 미치는지 추가 검토가 필요하다. 만약 유의성이 확인될 경우, 해당 변수는 훈련 데이터에만 존재하는 특수 변수이므로, 모든 데이터셋에 공통적으로 존재하는 변수 중 연관성이 높은 변수를 모색해 해당 변수를 기반으로 한 확률기반 파생변수를 생성하는 것이 바람직해보임

◆ 훈련데이터에만 존재하는 변수 - 사고유형 (세부분류)

[사고유형 - 세부분류 변수에 대한 유의성 검증]

✓ 공용 변수에 세부분류 변수를 추가한 회귀분석 결과

→ 범주형 변수에 타겟 인코딩을 수행해 동일한 기준으로 상대적 중요도를 평가

OLS Regression Results						
=====						
Dep. Variable:	ECLO	R-squared:	0.052			
Model:	OLS	Adj. R-squared:	0.052			
Method:	Least Squares	F-statistic:	166.4			
Date:	Tue, 08 Jul 2025	Prob (F-statistic):	0.00			
Time:	05:45:22	Log-Likelihood:	-36561.			
No. Observations:	39609	AIC:	7.315e+04			
Df Residuals:	39595	BIC:	7.327e+04			
Df Model:	13					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	37.0300	7.593	4.877	0.000	22.148	51.912
연	-0.0227	0.004	-6.053	0.000	-0.030	-0.015
월	-0.0017	0.001	-1.847	0.065	-0.003	0.000
일	-0.0009	0.000	-2.516	0.012	-0.002	-0.000
시간	-0.0015	0.001	-2.668	0.008	-0.003	-0.000
요일	0.9892	0.082	12.087	0.000	0.829	1.150
기상상태	0.2556	0.579	0.441	0.659	-0.880	1.391
사고유형	0.3272	0.042	7.746	0.000	0.244	0.410
도로형태1	0.3706	0.089	4.176	0.000	0.197	0.545
도로형태2	0.4570	0.075	6.097	0.000	0.310	0.604
노면상태	0.9092	0.455	1.998	0.046	0.017	1.801
구	0.2075	0.123	1.683	0.092	-0.034	0.449
동	0.9405	0.050	18.826	0.000	0.843	1.038
사고유형 - 세부분류	0.7711	0.034	22.526	0.000	0.704	0.838

! 사고유형-세부분류에 대한 유의성 검증 결과

- 회귀분석 결과
 - 모델링에 사용되는 공용 변수를 포함한 회귀모형에 사고유형-세부분류 변수를 추가해 ECLO와의 관계를 분석한 결과, $p < 0.05$ 수준에서 통계적으로 매우 유의미한 독립변수로 확인되었다.
 - 또한 회귀계수(coef)와 t-값이 높은 수준을 보여, 종속변수(ECLO)에 미치는 영향력이 크다는 점이 검증되었다.

✓ 연관성 높은 공용 변수 탐색

- ECLO 기반 모든 변수를 타겟 인코딩 수행 후, 사고유형 - 세부분류 변수를 종속변수, 공용 변수를 독립변수로 설정해, 종속변수에 가장 높은 영향을 미치는 공용 변수 탐색
- 탐색 기준: p-value가 유의미하게 나타나는 변수 중, 높은 회귀계수(coef)와 t-값을 나타내는 공용 변수를 탐색(단, 범주 수가 상대적으로 작은 사고유형 변수는 제외한다)

OLS Regression Results

=====

Dep. Variable:

사고유형 - 세부분류

R-squared:

0.323

Model:

OLS

Adj. R-squared:

0.323

Method:

Least Squares

F-statistic:

1577.

Date:

Tue, 08 Jul 2025

Prob (F-statistic):

0.00

Time:

05:47:17

Log-Likelihood:

39458.

No. Observations:

39609

AIC:

-7.889e+04

Df Residuals:

39596

BIC:

-7.878e+04

Df Model:

12

Covariance Type:

nonrobust

=====

coef

std err

t

P>|t|

[0.025

0.975]

const

0.4023

0.158

2.554

0.011

0.094

0.711

요일

-0.0111

0.012

-0.919

0.358

-0.035

0.013

기상상태

0.0716

0.085

0.841

0.400

-0.095

0.238

사고유형

0.6840

0.005

132.120

0.000

0.674

0.694

도로형태1

0.1330

0.013

10.224

0.000

0.107

0.158

도로형태2

-0.0929

0.011

-8.449

0.000

-0.114

-0.071

노면상태

-0.0278

0.067

-0.416

0.678

-0.159

0.103

구

-0.2727

0.018

-15.117

0.000

-0.308

-0.237

동

0.0705

0.007

9.628

0.000

0.056

0.085

연

0.0297

0.025

1.171

0.242

-0.020

0.079

월

0.0539

0.026

2.086

0.037

0.003

0.105

일

0.0599

0.024

2.458

0.014

0.012

0.108

시간

0.1086

0.013

8.203

0.000

0.083

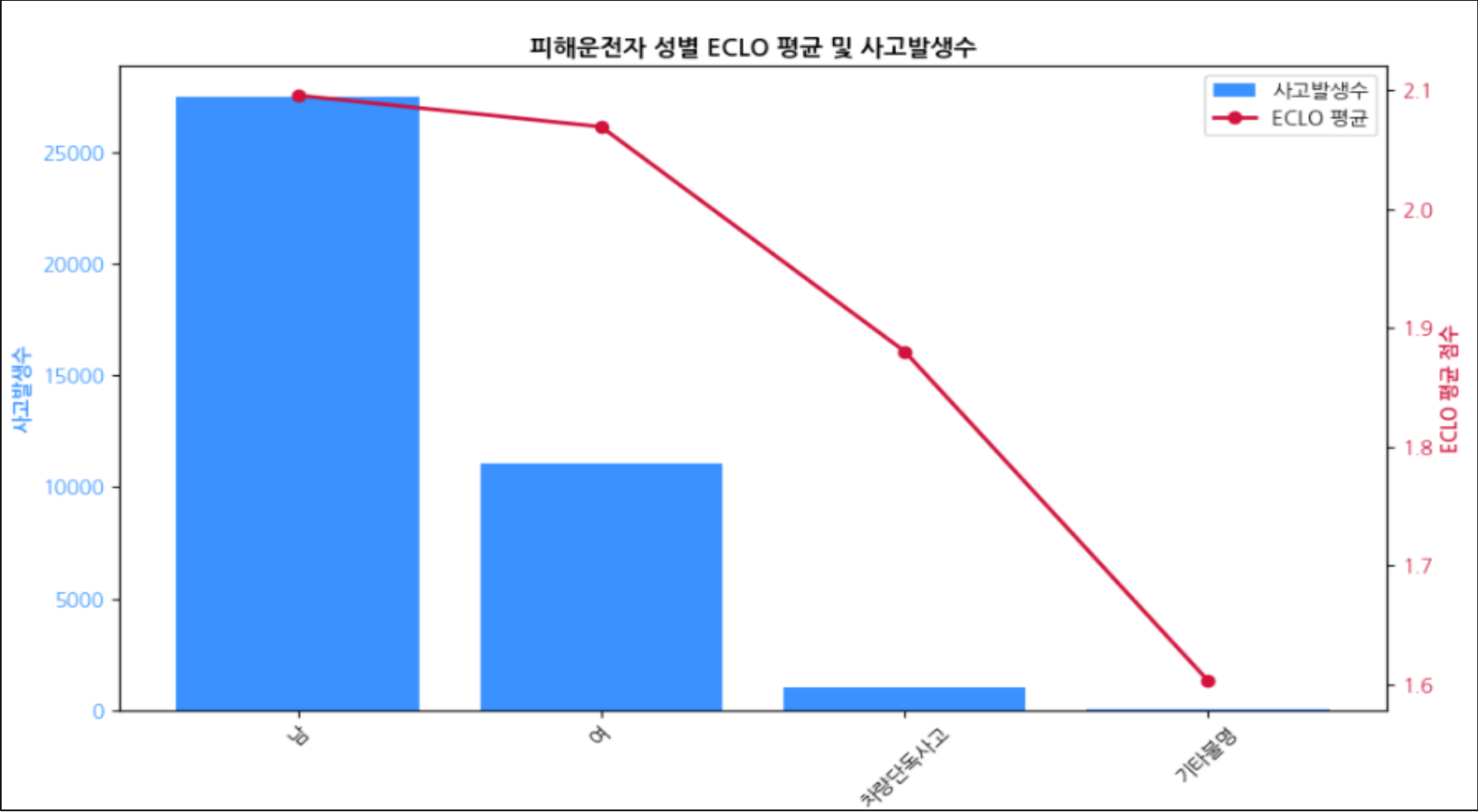
0.134

=====

- 연관성 높은 공용 변수 탐색 결과
 - 훈련 데이터에만 존재하는 사고유형-세부분류 특성을 모델 특성에 반영하기 위해 세부분류 특성과 연관성이 높은 공용 변수 탐색 결과, 사고유형 변수를 제외하고 도로형태1 변수가 가장 높은 회귀계수와 t-값을 기록하여, 사고유형-세부분류와 가장 밀접한 연관성을 지닌 변수로 확인되었다.
 - 따라서, 고위험 사고를 주로 유발하는 특정 사고유형-세부분류 범주에 대해 도로형태1 기반의 고위험 범주 발생 확률을 파생변수로 추가하는 방안을 고려할 수 있다.
 - 이를 통해 사고유형-세부분류의 특성을 확률적 형태로 반영함으로써, 모델의 예측 성능과 설명력을 동시에 강화할 수 있을 것으로 기대된다.

◆ 훈련데이터에만 존재하는 변수 - 피해운전자 성별

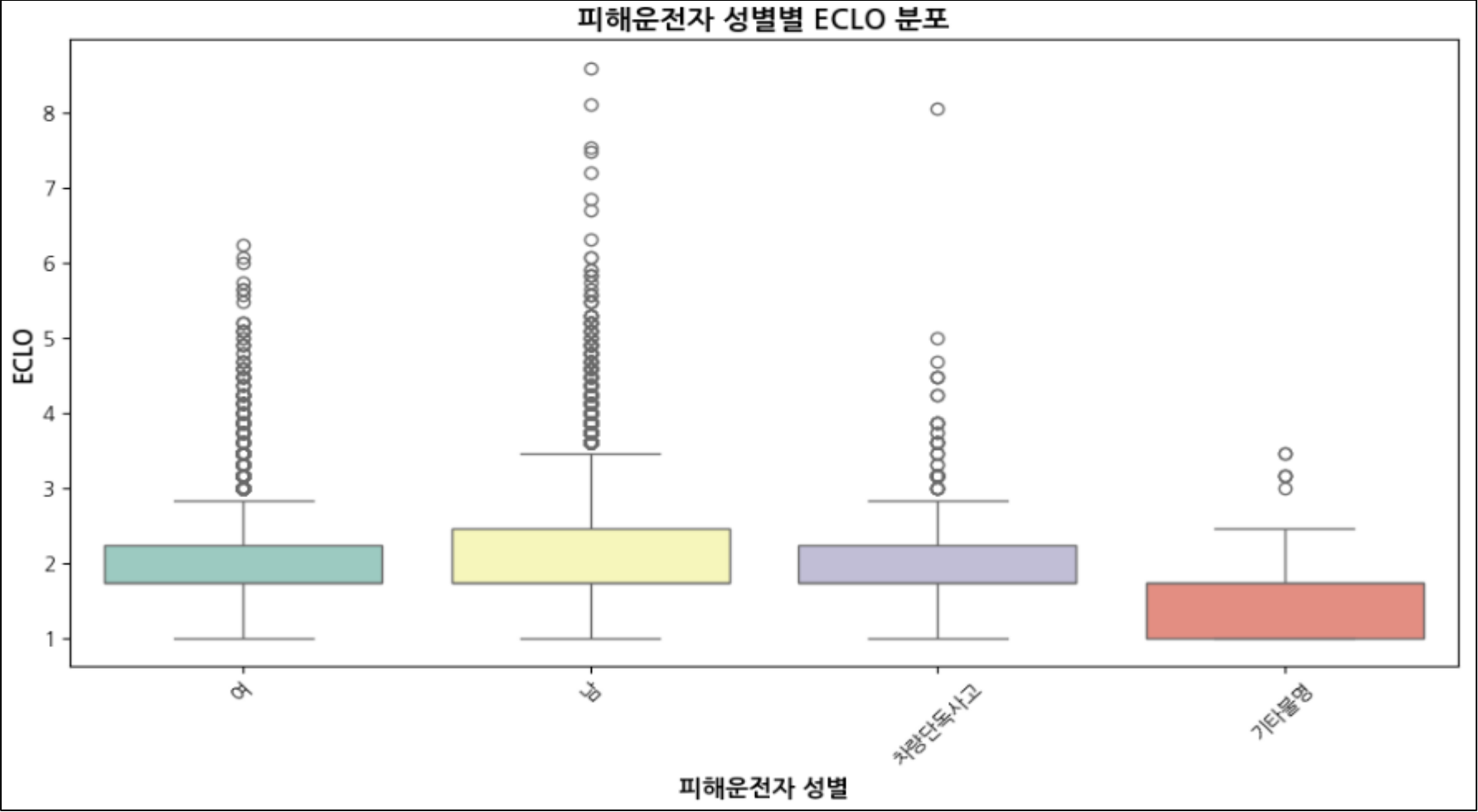
✔ 피해운전자 성별에 따른 ECLO 평균 및 사고발생빈도 비교



[피해운전자 성별에 따른 ECLO 통계량]

피해운전자 성별	Count	Mean	Std	Min	25%	50%	75%	Max
남	27505	2.095772	0.637247	1.0	1.732051	1.732051	2.449490	8.602325
여	11048	2.069313	0.583937	1.0	1.732051	1.732051	2.236068	6.244998
차량단독사고	991	1.880239	0.686464	1.0	1.732051	1.732051	2.236068	8.062258
기타불명	65	1.602951	0.684489	1.0	1.000000	1.732051	1.732051	3.464102

✔ 피해운전자 성별에 따른 ECLO 분포



! 피해운전자 성별별 ECLO 특성 분석

- ✔ 피해운전자 성별이 남성일경우, 여성보다 평균적으로 높은 사고 심각도를 보이며, 분포의 상단(고위험 ECLO 구간)에서 차이가 더욱 두드러지는 모습을 보인다.
- ✔ 차량단독사고 및 기타불명 범주는 평균 ECLO가 낮고 사고 건수가 제한적이므로, 주요 분석보다는 보조적 참고 변수로 활용하는 것이 적절해보임
- ✔ 따라서, 피해운전자 성별 특성을 반영한 위험도 분석에서는 남성 피해운전자군을 상대적으로 높은 위험도로 분류할 필요가 있으며, 고위험 사고 관리 전략에서도 성별 차이를 고려할 수 있다.

◆ 훈련데이터에만 존재하는 변수 - 피해운전자 성별

[피해운전자 성별 변수에 대한 유의성 검증]

- ✓

공용 변수에 피해운전자 성별 변수를 추가한 회귀분석 결과(범주형 변수 타겟 인코딩 수행)
- ✓

연관성 높은 공용 변수 탐색(타겟인코딩된 변수를 기반으로 종속변수를 피해운전자 성별로 설정)

OLS Regression Results

Dep. Variable:

ECLO

R-squared:

0.040

Model:

OLS

Adj. R-squared:

0.040

Method:

Least Squares

F-statistic:

127.5

Date:

Fri, 04 Jul 2025

Prob (F-statistic):

0.00

Time:

07:20:17

Log-Likelihood:

-36803.

No. Observations:

39609

AIC:

7.363e+04

Df Residuals:

39595

BIC:

7.375e+04

Df Model:

13

Covariance Type:

nonrobust

coef

std err

t

P>|t|

[0.025

0.975]

const

37.5482

7.641

4.914

0.000

22.571

52.525

연

-0.0230

0.004

-6.105

0.000

-0.030

-0.016

월

-0.0018

0.001

-1.987

0.047

-0.004

-2.43e-05

일

-0.0009

0.000

-2.473

0.013

-0.002

-0.000

시간

-0.0019

0.001

-3.441

0.001

-0.003

-0.001

요일

0.9876

0.082

11.994

0.000

0.826

1.149

기상상태

0.4101

0.583

0.703

0.482

-0.733

1.553

사고유형

0.7923

0.038

20.981

0.000

0.718

0.866

도로형태1

0.4645

0.089

5.208

0.000

0.290

0.639

도로형태2

0.3994

0.075

5.297

0.000

0.252

0.547

노면상태

0.8550

0.458

1.867

0.062

-0.043

1.753

구

0.0060

0.124

0.048

0.961

-0.236

0.248

동

1.0041

0.050

19.976

0.000

0.906

1.103

피해운전자 성별

0.3872

0.083

4.647

0.000

0.224

0.551

• 회귀분석 결과

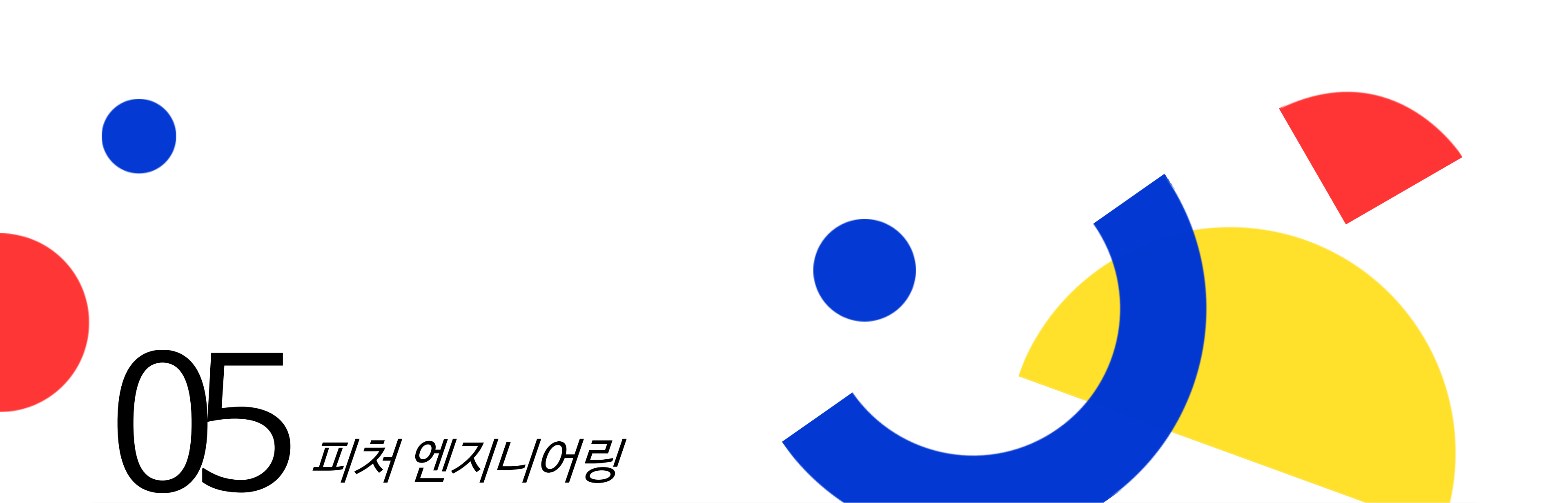
OLS Regression Results						
=====						
Dep. Variable:	피해운전자 성별		R-squared:	0.126		
Model:	OLS		Adj. R-squared:	0.125		
Method:	Least Squares		F-statistic:	473.9		
Date:	Fri, 04 Jul 2025		Prob (F-statistic):	0.00		
Time:	07:24:10		Log-Likelihood:	74410.		
No. Observations:	39609		AIC:	-1.488e+05		
Df Residuals:	39596		BIC:	-1.487e+05		
Df Model:	12					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	2.1090	0.065	32.359	0.000	1.981	2.237
요일	-0.0047	0.005	-0.950	0.342	-0.014	0.005
기상상태	-0.1679	0.035	-4.768	0.000	-0.237	-0.099
사고유형	0.1563	0.002	72.988	0.000	0.152	0.161
도로형태1	0.0270	0.005	5.026	0.000	0.017	0.038
도로형태2	-0.0336	0.005	-7.394	0.000	-0.043	-0.025
노면상태	0.0914	0.028	3.309	0.001	0.037	0.146
구	-0.0259	0.007	-3.463	0.001	-0.040	-0.011
동	-0.0295	0.003	-9.750	0.000	-0.035	-0.024
연	0.0040	0.011	0.380	0.704	-0.017	0.025
월	0.0013	0.011	0.125	0.900	-0.020	0.022
일	-0.0035	0.010	-0.351	0.726	-0.023	0.016
시간	-0.0279	0.005	-5.086	0.000	-0.039	-0.017
=====						

! 피해운전자 성별에 대한 유의성 검증 결과

- 회귀분석 결과
 - 회귀분석을 통해 공용 변수에 피해운전자 성별을 추가하여 분석한 결과, 해당 변수는 $p < 0.05$ 수준에서 통계적으로 유의미한 독립변수로 확인되었다.
 - 또한 회귀계수와 t-값 역시 일정 수준 이상으로 나타나, 피해운전자 성별이 종속변수(ECLO)에 유의미한 영향을 미치는 요인임을 검증할 수 있었다.

- 연관성 높은 공용 변수 탐색 결과
 - 피해운전자 성별을 종속변수로 설정한 회귀분석 결과, 사고유형 변수를 제외하고 기상상태와 도로형태2가 피해운전자 성별과 가장 밀접한 연관성을 보이는 변수로 확인되었다. 기상상태는 피해운전자 성별에 대해 가장 높은 회귀계수를 나타냈으며, 도로형태2는 절대값 기준 가장 높은 t-값을 보여 통계적으로 강한 설명력을 가진 것으로 나타났다.
 - 따라서, 특정 피해 성별 운전자가 주로 유발하는 고위험 사고 특성을 반영하기 위해 기상상태 및 도로형태2 기반의 고위험 범주 발생 확률을 파생변수로 추가하는 방안을 고려할 수 있다.



05

피쳐 엔지니어링

- 연도 변환
- 시계열 변수 Sin/Cos 변환
- 시간적 변수 관련 파생변수 추가
- 공간적 변수 관련 파생변수 추가
- 환경적 변수 변형
- 통계기반 파생변수 추가

연도 변환에 따른 모델 성능 분석

❖ 연도 변수 변환의 필요성

- 연도 변수는 값 크기 자체가 커서, 모델 학습 과정에서 초기 가중치가 불균형적으로 크게 설정되는 문제가 발생
- 이에 따라, 연도 변수를 스케일링 처리하여 입력 범위를 안정화함으로써 학습 과정의 수렴을 촉진하고, 나아가 예측 성능 개선 여부에 대한 검증은 수행한다.

✓

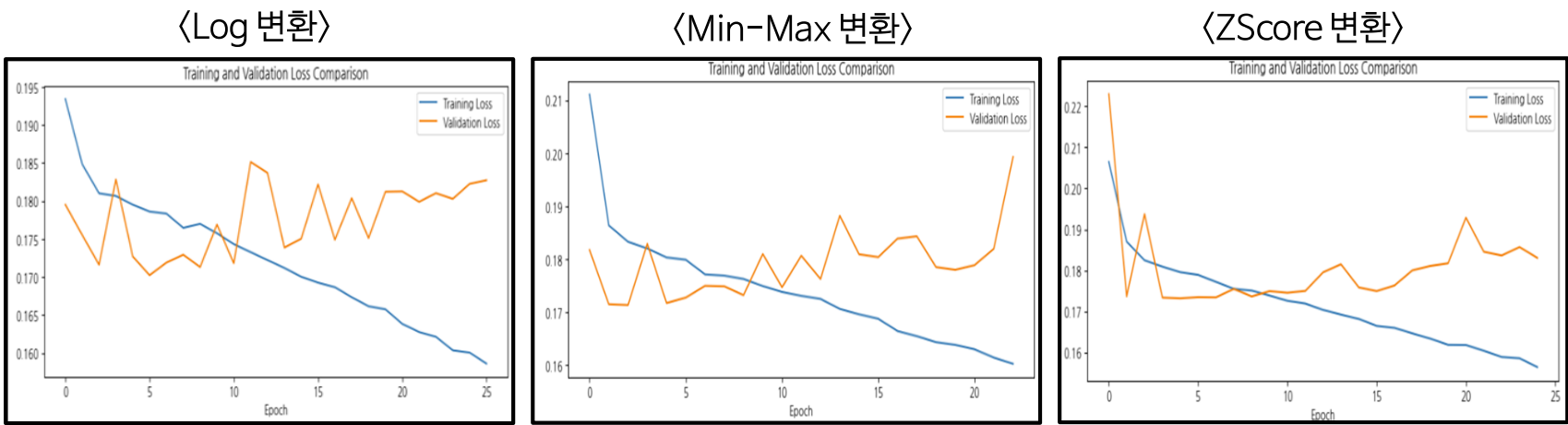
실험 조건

구분	설명
스케일링 방식	Log / MinMax / Zscore
인코딩 방식	One-Hot Encoding
모델 파라미터	• 이전 최적화 수행 모델 구조를 동일하게 사용
	Layers: 64/32/1
	Opitmizer: Adam
	LR: 0.0001
	Loss: Huber
종속변수 변환 방식	Activation: Relu
	Epochs: 200 / Patience: 20
	Sqrt 변환
반복 성능 측정 횟수	5회
데이터 분리 방식	훈련 데이터: 2019~2021년 데이터
	검증 데이터: 훈련 데이터의 20%
	테스트 데이터: 2021년 데이터

✓

연도 변환 시 발생하는 과적합 문제 해소

- 연도 변환 시 발생하는 문제점



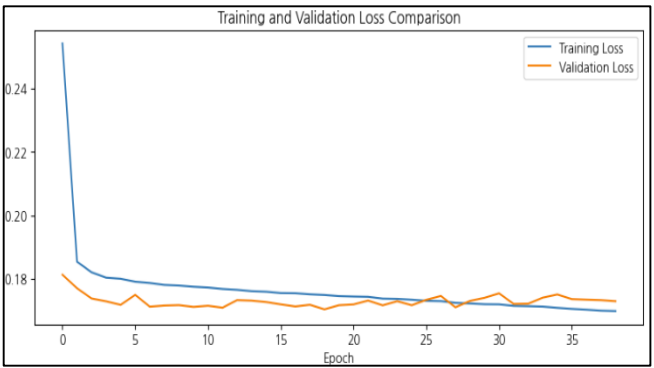
- 연도를 스케일링 할 시, 모든 변환 방식에서 과적합이 발생하는 문제 확인
- 이는 연도를 변환할 시, 모델이 특정 연도별 특성을 지나치게 학습해 새로운 데이터에 대한 일반화 성능이 떨어져서 발생하는 문제로 추정할 수 있다.

✓

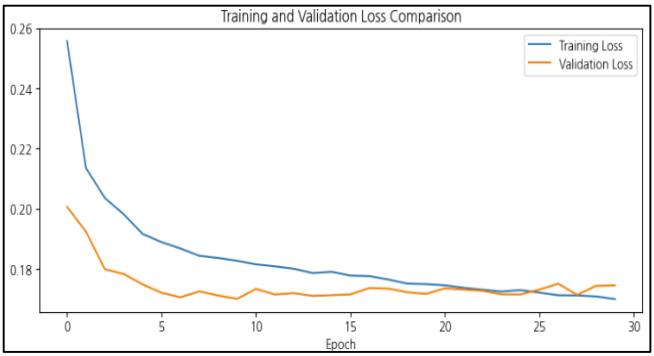
과적합 완화 위한 모델 파라미터 변경

- 과적합 문제를 완화시키기 위해 학습률 감소, Dropout, L2 정규화 추가와 같은 파라미터 추가 및 수정 작업을 수행

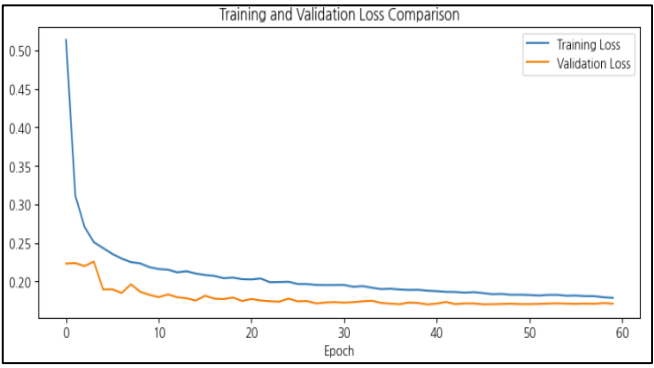
1. 학습률 감소(0.001 → 0.0001)



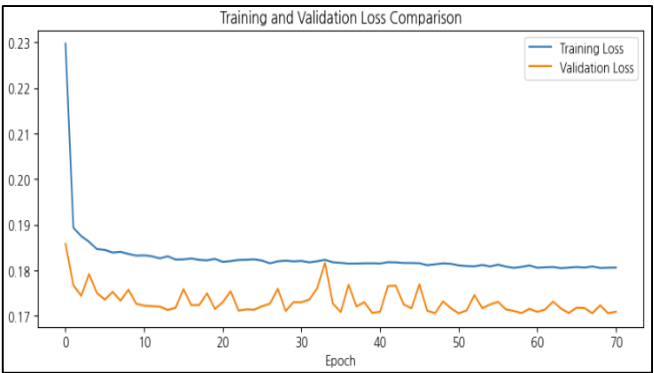
2. Dropout 추가



3. 학습률 감소 + Dropout 추가



4.. L2 정규화 추가(0.001)



연도 변환에 따른 모델 성능 분석

연도 변환 방식별 모델 일반화 성능 측정

- 과적합 완화 위한 모델 파라미터 조정 결과
 - 학습률 감소 + Dropout 적용 및 L2 정규화 기법을 추가한 모델 구성이 과적합 현상이 완화되는것을 확인
 - 이에 따라, 연도 변수 변환 방식별로 위와 같은 규제 기법을 적용한 모델의 일반화 성능을 반복 실험(5회)을 통해 검증함으로써 최적의 연도 변환 방식을 탐색하고자 한다.

일반화 성능 평가대상 모델 구성

- Non-Scale (연도 변환 미적용)
- 학습률 감소 + Dropout 적용
- L2 정규화 ($\lambda = 0.001$)
- L2 정규화 ($\lambda = 0.0005$)

연도 변환 적용 시, 초기 가중치 변화 검증

〈연도 스케일링 적용 X〉

Epoch 1/200	687/687	3s 3ms/step	loss: 5.3317	val_loss: 0.3476
Epoch 2/200	687/687	2s 2ms/step	loss: 0.3917	val_loss: 0.8891

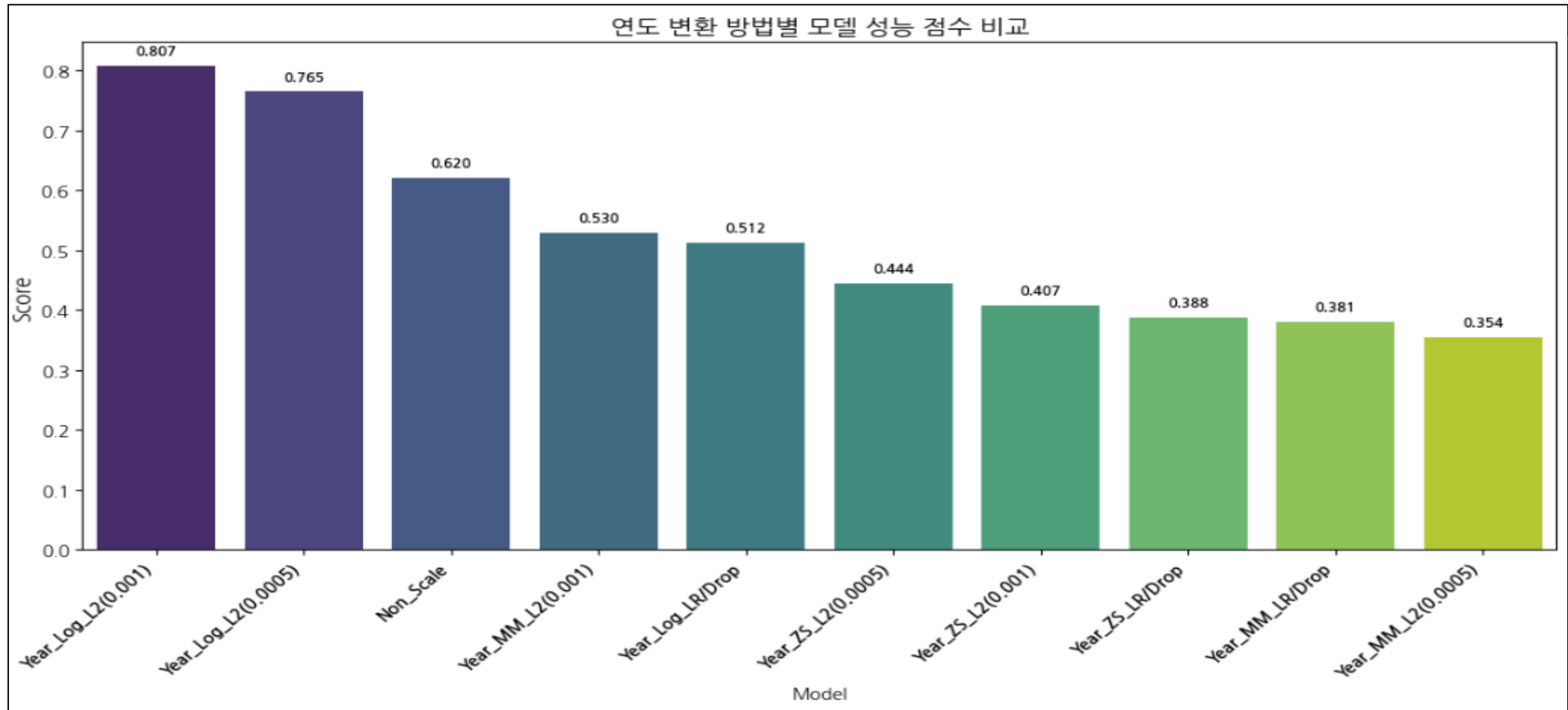
모델 초기 가중치 개선 확인

〈연도 Log 변환 적용〉

Epoch 1/200	687/687	3s 3ms/step	loss: 0.8991	val_loss: 0.1908
Epoch 2/200	687/687	2s 3ms/step	loss: 0.3327	val_loss: 0.1915

일반화 성능 측정 결과

- 종합 성능 점수 계산 함수 활용 모델별 종합 성능 평가 시각화



연도 변환 방식에 따른 모델 성능 비교 결과

- 연도 변환 방식과 과적합 완화 기법 적용 여부에 따른 모델 일반화 성능을 비교한 결과, 연도 변수에 로그 변환을 적용하고 L2 정규화($\lambda = 0.001$)를 병행한 모델이 가장 우수한 성능을 보였다.
- 이는 변환을 적용하지 않은 기존 모델(Non-Scale)에 비해 향상된 성능을 나타냈으며, 전반적으로 연도 변수에 로그 변환을 수행한 모델들이 상대적으로 높은 성능을 보였다.
- 따라서, 연도 변환은 예측 성능 향상에 유의미한 기여를 하는 것으로 판단되며, 향후 모델링에서는 연도 로그 변환 + L2 정규화($\lambda = 0.001$) 조합을 최적 모델 구성으로 채택한다.

시계열 변수 Sin/Cos 변환에 따른 모델 성능 분석

✓ 월, 일, 시간, 요일에 대한 시계열적 특성을 보유하고 있는 변수에 대한 Sin/Cos 변환시의 모델 일반화 성능 분석

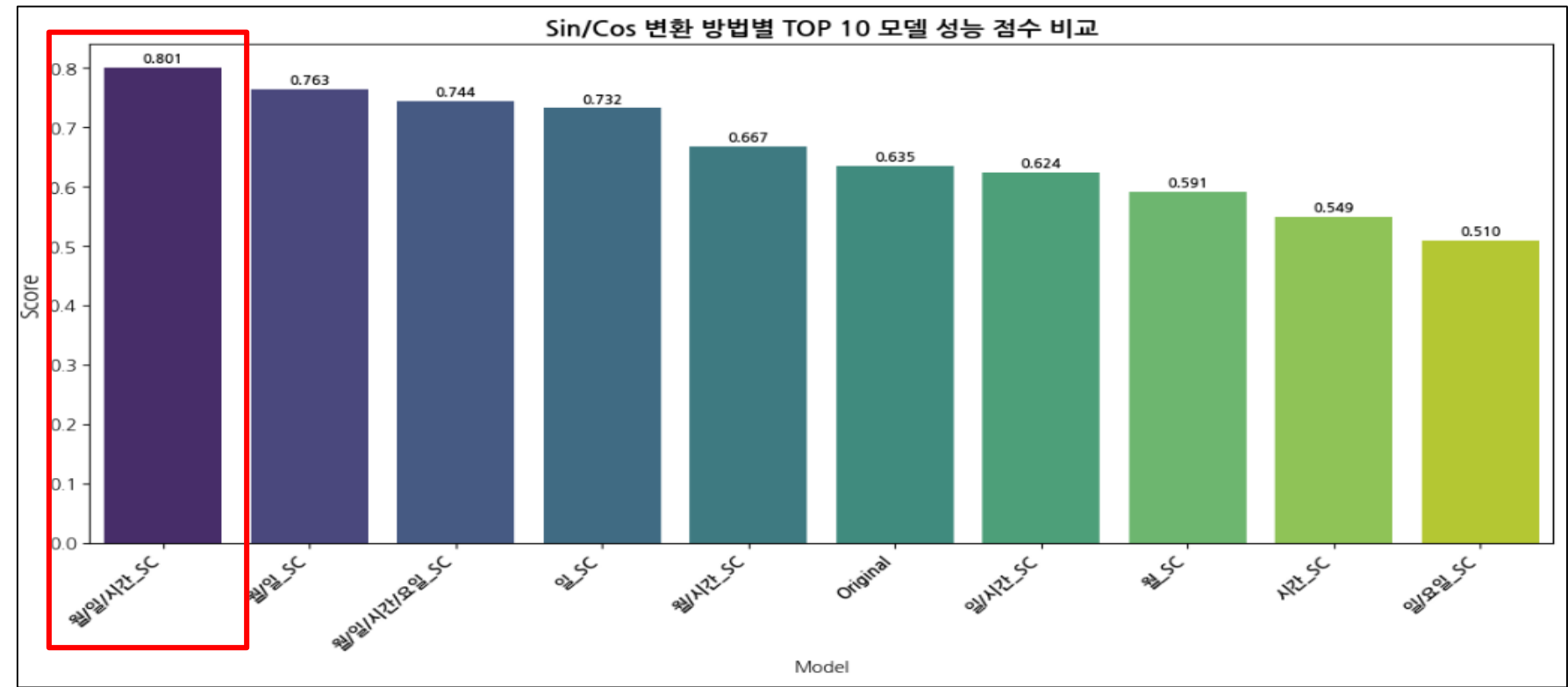
❖ 시계열 변수의 Sin/Cos 변환의 필요성

- 대구 교통사고 데이터는 시계열적 특성을 지니고 있어 순서를 보존할 때 성능이 향상
- 시간, 요일과 같은 변수는 주기적 속성을 가지므로, 단순 정수로 표현할 경우 23시와 0시처럼 실제 인접한 값이 멀리 떨어진 값으로 인식되는 주기성 왜곡 문제가 발생
- 따라서, 이러한 시계열 변수를 Sin/Cos 변환하여 연속성과 주기성을 보존함으로써, 예측 성능 향상 여부를 검증한다.

✓ 실험 조건

구분	설명
Sin/Cos 변환 방식	월, 일, 시간, 요일에 대해 Sin/Cos 변환을 수행해 단일 및 복수 조합에 대한 일반화 성능 측정
인코딩 방식	One-Hot Encoding
모델 파라미터	• 이전 최적화 수행 모델 구조를 동일하게 사용
	Layers: 64/32/1
	Opitmizer: Adam
	LR: 0.0001
	Loss: Huber
	Activation: Relu
	Epochs: 200 / Patience: 20
	+ L2 정규화($\lambda = 0.001$)
종속변수 변환 방식	Sqrt 변환
연도 변환 방식	Log 변환
반복 성능 측정 횟수	10회
데이터 분리 방식	훈련 데이터: 2019~2021년 데이터
	검증 데이터: 훈련 데이터의 20%
	테스트 데이터: 2021년 데이터

✓ 일반화 성능 측정 결과 (Score 기준 상위 10개 모델)



[상위 5개 모델 성능 지표]

Model	MSE	RMSE	RMSLE	R ²	Score
월/일/시간_Sin/Cos	9.278452	3.046051	0.425806	0.004981	0.800545
월/일_Sin/Cos	9.271956	3.044988	0.425989	0.005677	0.762992
월/일/시간/요일_Sin/Cos	9.275606	3.045583	0.425956	0.005286	0.744039
일_Sin/Cos	9.270346	3.044722	0.426072	0.005850	0.731947
월/시간_Sin/Cos	9.270437	3.044734	0.426183	0.005840	0.667323



여러 시계열 변수 조합에 대해 Sin/Cos 변환을 적용한 결과, 월+일+시간 조합을 변환한 모델이 가장 우수한 일반화 성능을 보였다.

해당 변환 방식은 대체로 기존 모델 대비 우수한 성능을 보였으며, 이는 Sin/Cos 변환이 예측 성능 개선에 효과적인 기법임을 알 수 있다.

따라서, 월+일+시간 조합을 Sin/Cos 변환의 최적 조합으로 판단하고, 이를 향후 모델링에서 기본 변환 방식으로 채택한다.

시간적 변수 관련 파생변수 추가에 따른 모델 성능 분석

✓ 시계열 변수가 모델에 미치는 영향을 보다 명확히 반영하기 위해, 시계열 관련 파생변수를 추가하여 예측 성능 개선 효과를 검증한다.

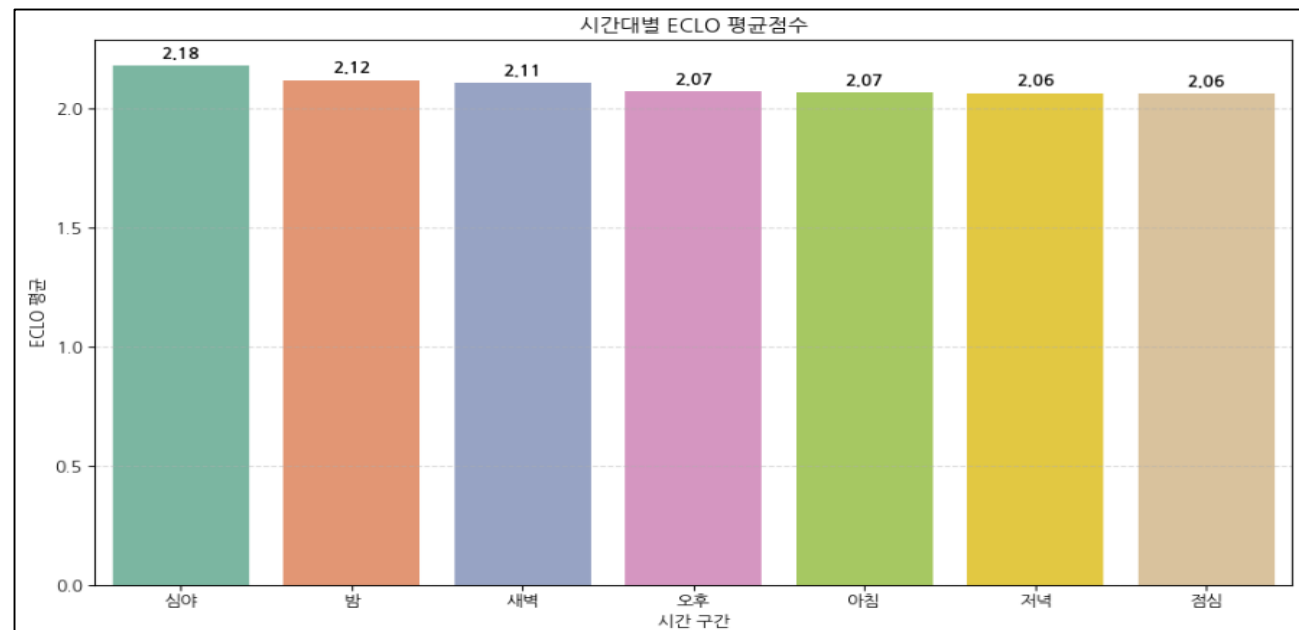
❖ 추가할 파생변수 목록 : 시간대구분, 주말구분, 공휴일구분, 특성교차(주말+공휴일)

❖ 시간대 구분 변수의 유의미성 검증

✓ 시간대 구간 정의

구간	구분
0 ~ 3	심야
4 ~ 7	새벽
8 ~ 10	아침
11 ~ 13	점심
14 ~ 17	오후
18 ~ 20	저녁
21 ~ 23	밤

✓ 시간대구분에 따른 사고 위험도 평균(ECLO) 변화



✓ 시간대구분에 따른 구간별 ECLO 차이에 대한 ANOVA 분석

ANOVA 분석 지표	값
F-statistic	16.01
P-value	0.000

시간대 구분에 따른 ECLO 점수 차이는 **통계적으로 유의하게** 나타났다.

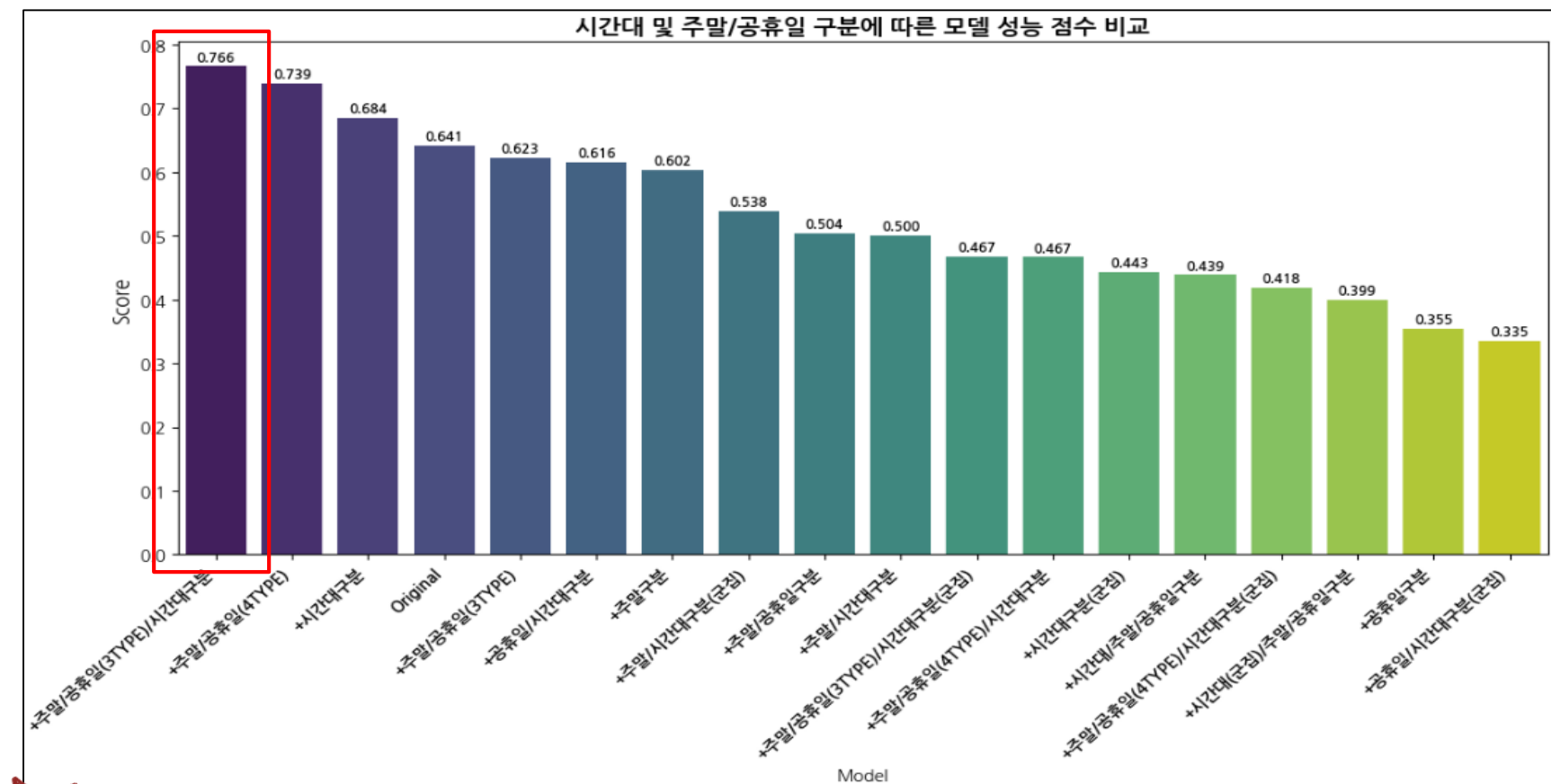
이에 따라, 해당 파생변수를 추가해 시간 요인이 미치는 영향을 보다 **명확히 반영하는** 것은 예측 성능 개선에 효과적일것으로 기대한다.

❖ 특성교차(주말 + 공휴일) 생성 방식

- 주말/공휴일(3type): 주말과 주중 구분을 기본으로 하되, 해당 날짜가 공휴일인 경우에는 '공휴일'로 재분류 (범주 예시: 주말 / 주중 / 공휴일)
- 주말/공휴일(4type): 주말 여부와 공휴일 여부를 교차 결합하여 새로운 범주를 생성 (범주 예시: 주말_공휴일 / 주말_공휴일아님 / 주중_공휴일 / 주중_공휴일아님)

✓ 일반화 성능 측정 결과

- 기존 모델 및 변수 구조에서 시계열 변수에 대한 최적 Sin/Cos 변환 파생변수를 추가



시계열 변수 파생변수 조합별 일반화 성능을 비교한 결과, **특성교차(주말/공휴일_3TYPE) + 시간대구분 변수를 추가한 모델**이 가장 우수한 성능을 보였다.

전반적으로 시간대구분과 특성교차 파생변수를 포함한 모델이 기존 모델(Sin/Cos 변환 변수 포함)보다 높은 성능 점수를 나타냈으며, 이는 **시계열 변수 기반 파생변수 추가가 예측 성능 향상에 유의미한 개선 효과**를 제공함을 시사한다.

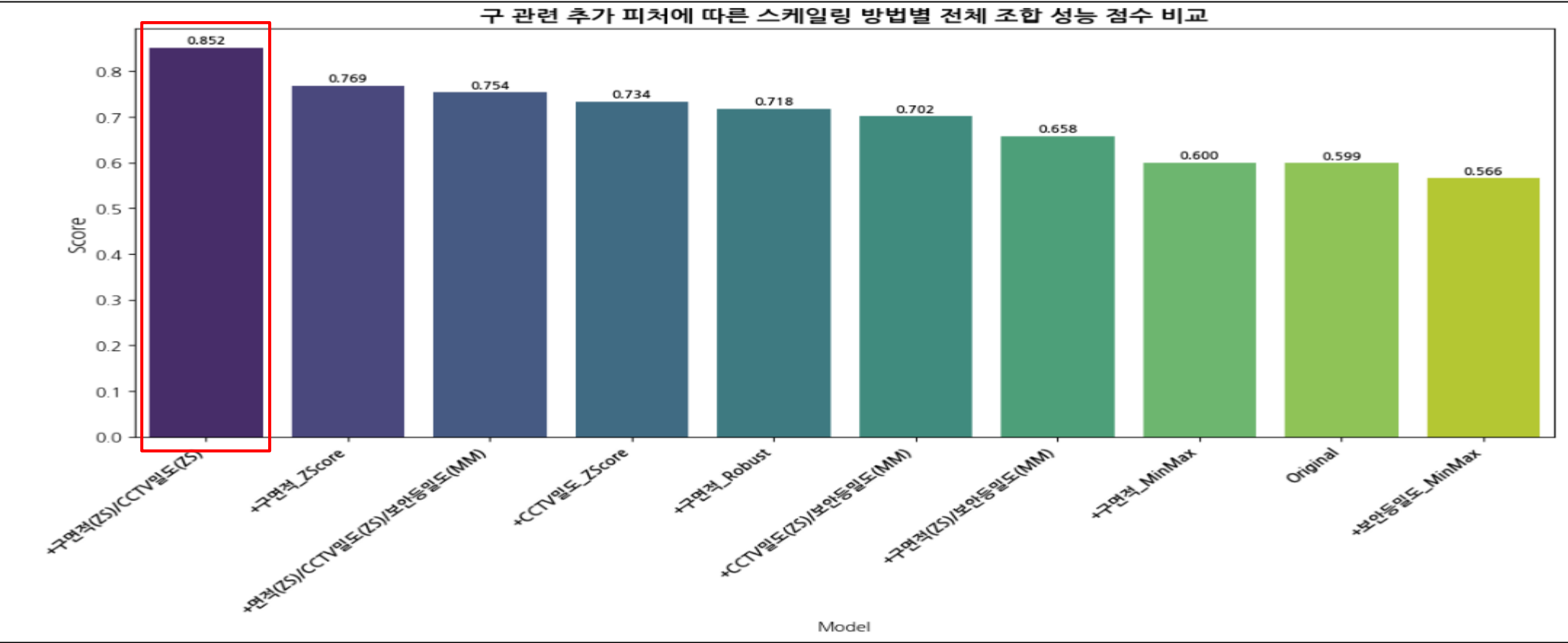
따라서, **특성교차(주말/공휴일_3TYPE)**와 **시간대구분** 변수를 결합한 조합을 **최적 성능 조합**으로 판단하며, 이를 반영한 모델 구성을 **기본 모델로 채택**한다.

공간적 변수 관련 파생변수 추가에 따른 모델 성능 분석

✓ 공간적 변수가 모델에 미치는 영향을 보다 명확히 반영하기 위해, 구 변수와 통계적으로 유의미한 연관성을 보였던 **구별 면적 / 구별 보안등 및 설치밀도 파생변수** 추가에 따른 예측 성능 개선 효과를 검증한다.

- ✓ 실험 설계
- 구 관련 파생변수에 대해 **단일 피쳐 추가 및 다중 피쳐 조합별 일반화 성능**을 측정하였으며, 모델 구조는 동일하게 유지
 - 이전 실험에서 가장 우수한 성능을 보인 파생변수 조합을 포함한 상태에서 성능을 검증
 - 각 변수의 값 크기가 상이하므로, **조합에 따른 스케일링 기법별 성능 차이**를 검증하였으며, 스케일링 기법은 **Min-Max, Z-score, Robust**을 사용

✓ 일반화 성능 측정 결과



[상위 모델 성능 지표]	Model	MSE	RMSE	RMSLE	R ²	Score
	면적(ZS)/CCTV(ZS)	9.247332	3.040938	0.426062	0.008318	0.852049
	면적 ZScore	9.261803	3.043316	0.425998	0.006766	0.768591
	면적(ZS)/CCTV(ZS)/ 보안등(MM)	9.255111	3.042218	0.426097	0.007484	0.753596
	CCTV ZScore	9.256516	3.042447	0.426106	0.007333	0.733882
	면적 Robust	9.249443	3.041284	0.426211	0.008092	0.718166

『공간적 변수 파생변수 추가에 따른 조합별 일반화 성능 분석 결과』

구별 면적과 CCTV 설치 밀도 변수에 **Z-Score 표준화**를 적용한 모델이 가장 우수한 일반화 성능을 보였다.

해당 모델은 전반적인 성능 지표에서 **가장 안정적인 결과**를 나타냈으며, 동시에 **가장 높은 성능 점수**(Score)를 기록하였다.

전반적으로 구 단위 공간적 파생변수를 추가한 모델은 기존 모델(Original) 대비 성능이 **향상**되었으며, 이는 공간적 파생변수의 추가가 **예측 성능 개선에 유의미한 효과**를 제공함을 시사한다.

특히, **구별 면적**(Z-Score) + **CCTV 설치 밀도**(Z-Score) 조합이 **최적의 성능**을 보였으며, 향후 모델링에서 해당 변수 조합을 사용한 모델을 기본 모델로 채택한다.

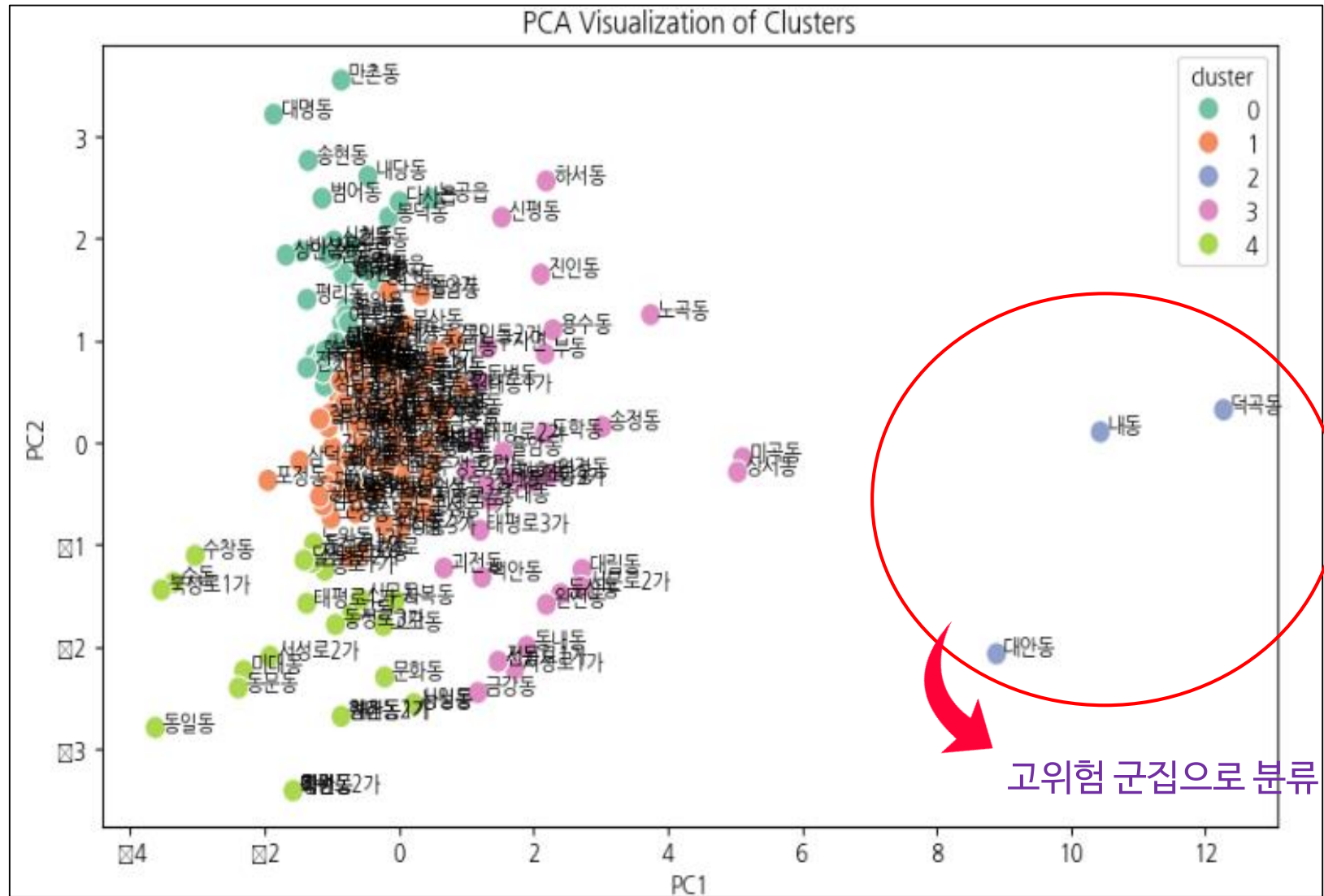
공간적 변수 관련 파생변수 추가에 따른 모델 성능 분석

✓ 기존 현황 분석에서 통계적으로 ECLO와 유의미한 연관성을 보였던 다변량 군집분석을 활용한 동별 위험도 군집 파생변수 추가에 따른 예측 성능 개선 효과를 검증

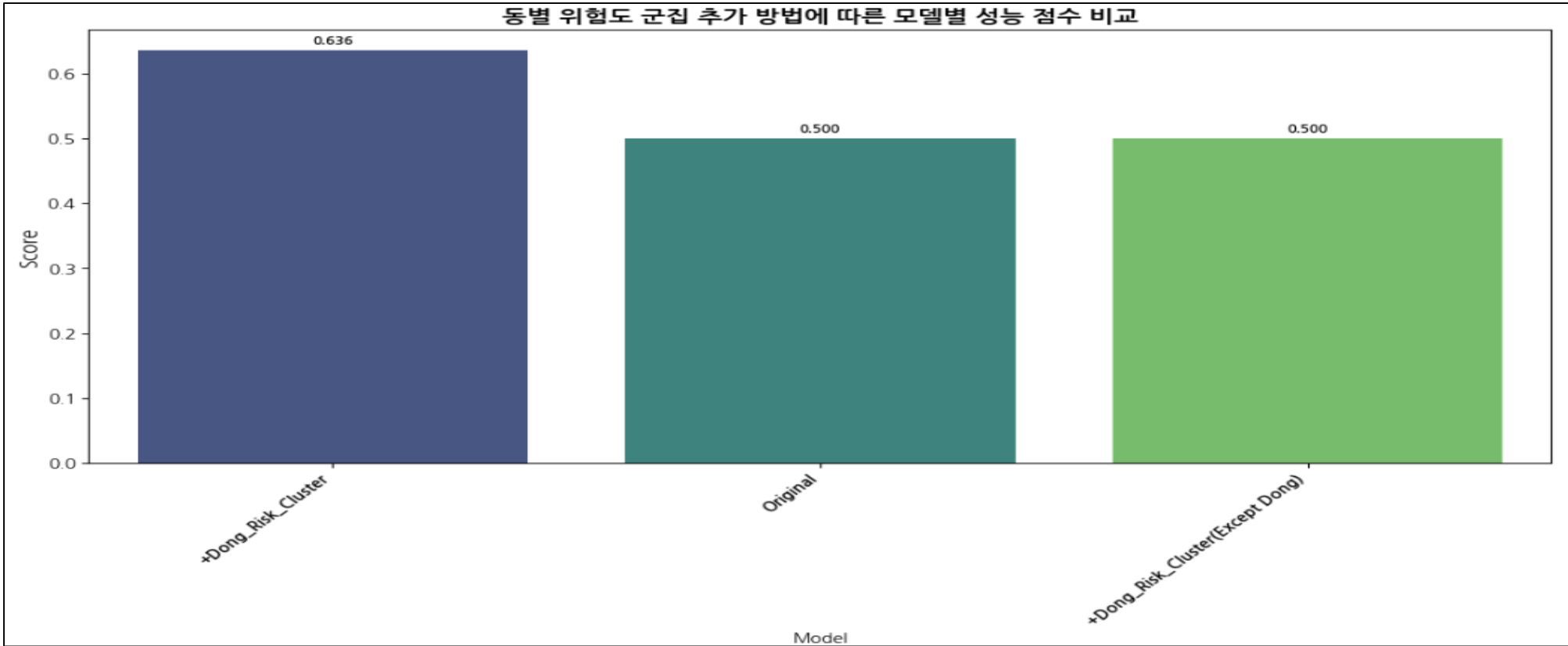
✓ 실험 설계

- 훈련 데이터 기반으로 생성한 동별 위험도 군집 파생변수를 활용하여,
 - 동 + 위험도 군집을 함께 추가한 모델과
 - 동 변수를 제외한 위험도 군집만 추가한 모델의 일반화 성능을 비교
- 이를 통해 기존 모델 대비 성능 개선 효과가 존재하는지를 검증

[훈련 데이터 기반 동별 위험도 PCA 군집 시각화]



✓ 일반화 성능 측정 결과



Model	MSE	RMSE	RMSLE	R ²	Score
동 위험도 군집 추가	9.245813	3.040692	0.426374	0.008481	0.635711
Original	9.265839	3.043966	0.426277	0.006333	0.500000
동 위험도 군집 추가 (동 피쳐 제외)	9.239597	3.039659	0.426474	0.009148	0.500000

[동 위험도 군집 파생변수 성능 분석 결과]

- ✓ 동 위험도 군집 파생변수 추가에 따른 조합별 일반화 성능을 비교한 결과, 동 변수를 포함한 동 위험도 군집 파생변수를 적용한 모델이 가장 우수한 성능을 보임
- ✓ 해당 모델은 이전 실험에서의 최적 모델(Original) 대비 성능이 향상되었으며, 반대로 동 변수를 제외하고 위험도 군집 변수만 추가한 경우에는 성능 개선 효과가 나타나지 않았다.
- ✓ 따라서, 동 변수를 포함한 동 위험도 군집 파생변수 추가 방식은 예측 성능 개선에 유의미한 효과를 제공하는 것으로 판단되며, 이를 반영한 모델 구성을 기본 모델로 채택한다.

환경적 변수 변형에 따른 모델 성능 분석

- ✓ 환경적 변수가 ECLO에 미치는 영향을 보다 명확히 반영하기 위해, **현황 분석 결과**에 기반해 불필요한 범주의 결합 및 제거와 범주 수가 많은 변수에 대한 **군집화**를 수행
- ✓ 이후 각 변형 방법별 일반화 성능을 측정하여, **예측 성능 개선에 가장 효과적인 변수 변형 방법**을 탐색

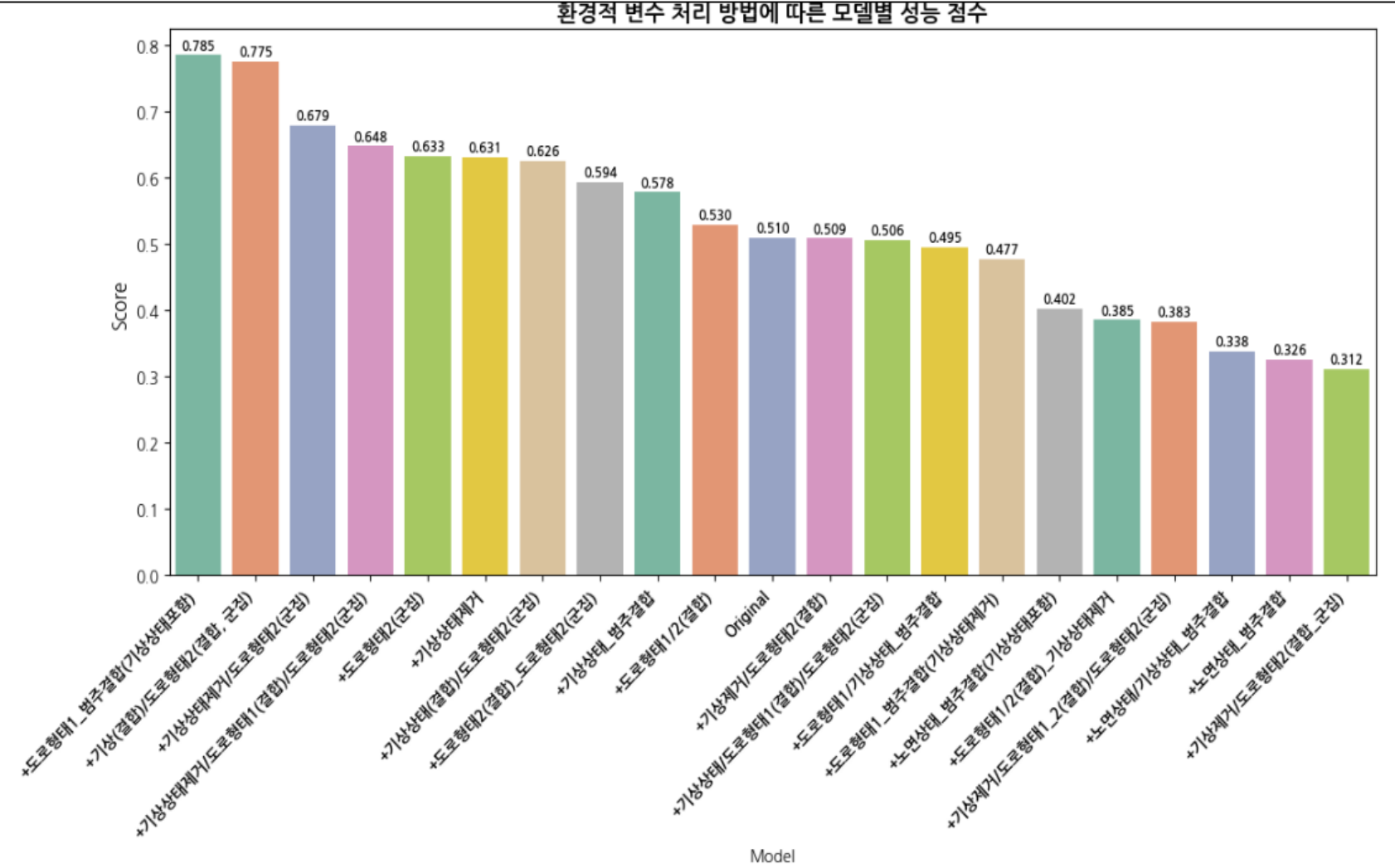
✓

환경적 변수별 변수 변형 방법

- 기상상태
 - 범주 결합: 상대적으로 사고 발생 수가 적은 범주인 **안개**와 **눈** 범주를 각각 **흐림**, **비** 범주와 결합
 - 변수 제거: 현황 분석 결과에 기반해 해당 변수는 ECLO에 미치는 영향이 **통계적으로 유의하지 않음**으로 변수 제거
 - 노면상태
 - 범주 결합: 상대적으로 사고 발생 수가 매우 적은 **침수**를 비슷한 성격의 **젖음/습기** 범주와 **결합해 습기계로 적설을 서리/결빙 범주와 결합해 결빙계**로 통합
- 도로형태1
 - 범주 결합: 사고 발생 수가 8건인 **미분류** 범주를 **주차장과 결합해 비일반도로 범주로 수정**
 - 도로형태2
 - 군집화 수행: 동 변수 다음으로 범주 수가 많은 범주에 대해 ECLO 통계량 기반 다변량 군집 분석을 수행해 위험도 구분을 실시
 - 범주 결합: 도로형태1과 마찬가지로 미분류와 주차장 범주를 결합해 비일반도로 범주로 수정

✓

조합 방식별 일반화 성능 측정 결과



[상위 모델 성능 지표]

Model	MSE	RMSE	RMSLE	R²	Score
도로형태1 (범주 결합)	9.231661	3.038357	0.426007	0.009999	0.790102
기상상태 제거 + 도로형태2(군집화)	9.262912	3.043494	0.425805	0.006647	0.680839
기상상태 제거 + 도로형태1 (범주 결합) + 도로형태2(군집화)	9.246761	3.040844	0.426047	0.008379	0.651566
도로형태2(군집화)	9.236315	3.039123	0.426196	0.009500	0.637434
기상상태 제거	9.245592	3.040649	0.426088	0.008505	0.635160



「환경적 변수 변형에 따른 조합별 일반화 성능 분석 결과」

환경적 변수 변환 조합별 일반화 성능을 비교한 결과, **도로형태1** 변수에서 **미분류** 범주를 **주차장** 범주와 **결합한 방식**이 **가장 높은 성능 점수**를 기록하였다.

해당 방식은 RMSLE 지표에서 최고 성능을 보이진 않았으나, 전반적으로 **안정적**이고 **우수한 성능**을 나타냈으며 기존 모델 대비 **개선 효과가 확인**되었다.

따라서, 해당 변환 방식을 기본 모델링 구성으로 채택해 반영하도록 한다.

통계기반 파생변수 추가에 따른 모델 성능 분석

- ✓ 훈련데이터에만 존재하는 피처 중 ECLO에 유의미한 영향을 미치는 변수에 대해, 고위험 사고 유발 범주를 반영한 공용 변수 기반 통계적 파생변수를 추가하여 예측 성능 향상
- ✓ 공용 변수 지정은 현황 분석에서 수행한 ECLO 기반 타겟 인코딩을 진행한 뒤 해당 피처를 종속변수, 공용 변수를 독립변수로 설정한 회귀분석을 실시하고, 종속변수에 가장 큰 영향력을 보이는 변수를 선택하는 방식으로 진행한다.

✓ 고위험 ECLO 분포 정의

- 훈련 데이터의 ECLO 점수를 기반으로 K-Means 군집 분석을 수행하여, 훈련데이터에만 존재하는 특정 피처의 고위험 사고 유발 범주를 식별한다.
- 군집별 평균 ECLO 점수를 전체 ECLO 점수를 기준으로 산출한 IQR 이상치 탐색 방법에 적용하고, 기준값을 초과하는 군집을 '고위험 ECLO 군집'으로 정의한다. 이후 각 피처 범주가 해당 군집에 속할 비율을 산출하여, 고위험 사고 유발 가능성이 높은 범주를 도출한다.

❖ K-Means 활용 군집별 ECLO 통계량

[IQR 이상치 탐색 방법을 적용한 ECLO 기준값 : 3.525648145604629]

군집 평균 ECLO 점수가 기준값을 초과하는 Cluster 5, 4를 고위험 ECLO 분포로 설정

eclo_cluster	count	mean	std	min	25%	50%	75%	max
0	5	201.0	4.725975	0.602897	4.242641	4.358899	4.582576	4.898979
1	4	847.0	3.668928	0.216099	3.464102	3.464102	3.605551	3.872983
2	1	2147.0	3.008673	0.158803	2.828427	2.828427	3.000000	3.162278
3	2	6724.0	2.332403	0.114242	2.236068	2.236068	2.236068	2.449490
4	0	10553.0	1.740928	0.062070	1.414214	1.732051	1.732051	2.000000
5	3	1508.0	1.000000	0.000000	1.000000	1.000000	1.000000	1.000000

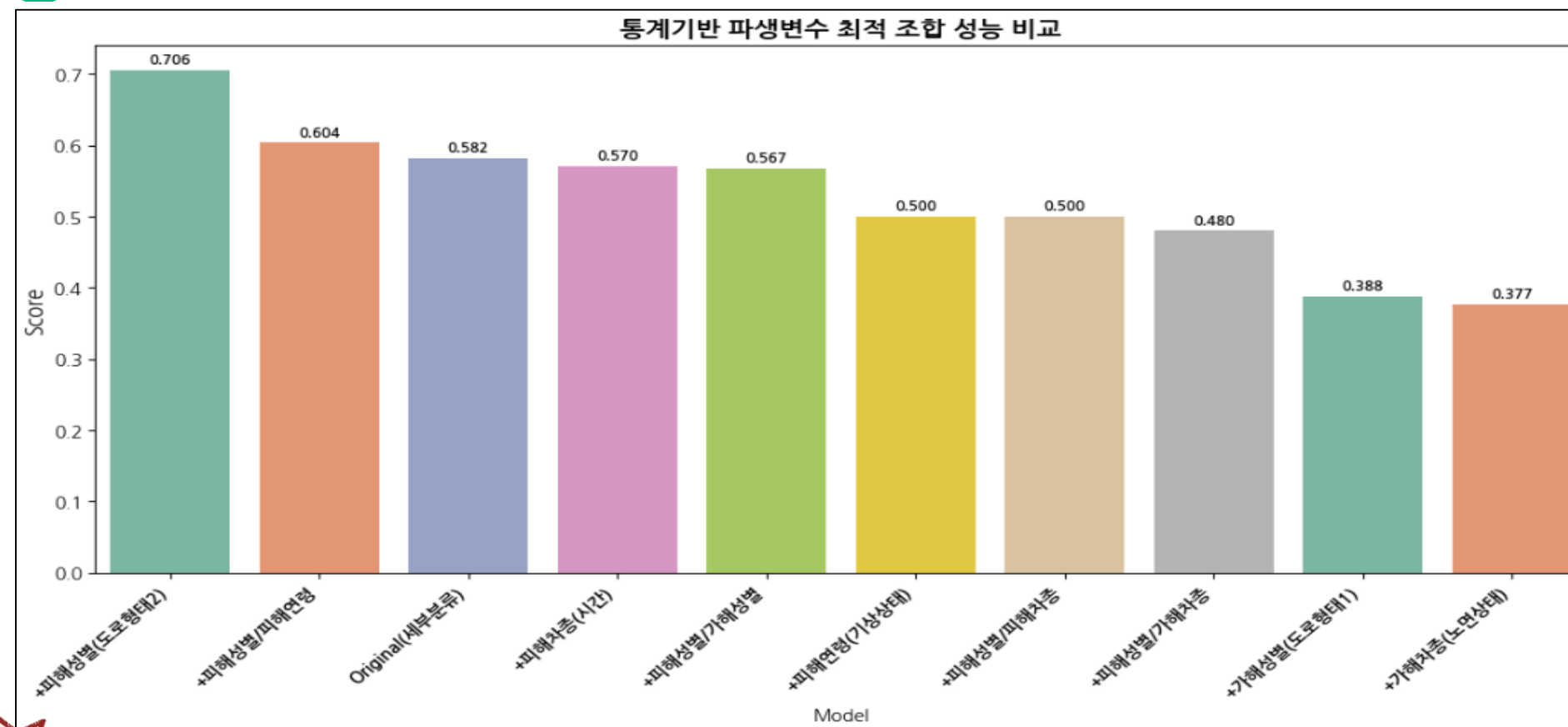
✓ 조합별 성능 측정 방식

파생변수 생성 방식에 따른 단일 피처 추가 일반화 성능 측정

단일 피처 추가 실험에서 가장 우수한 성능 변수를 기본 파생변수로 채택 후,
추가 파생변수 조합에 대한 일반화 성능을 검증

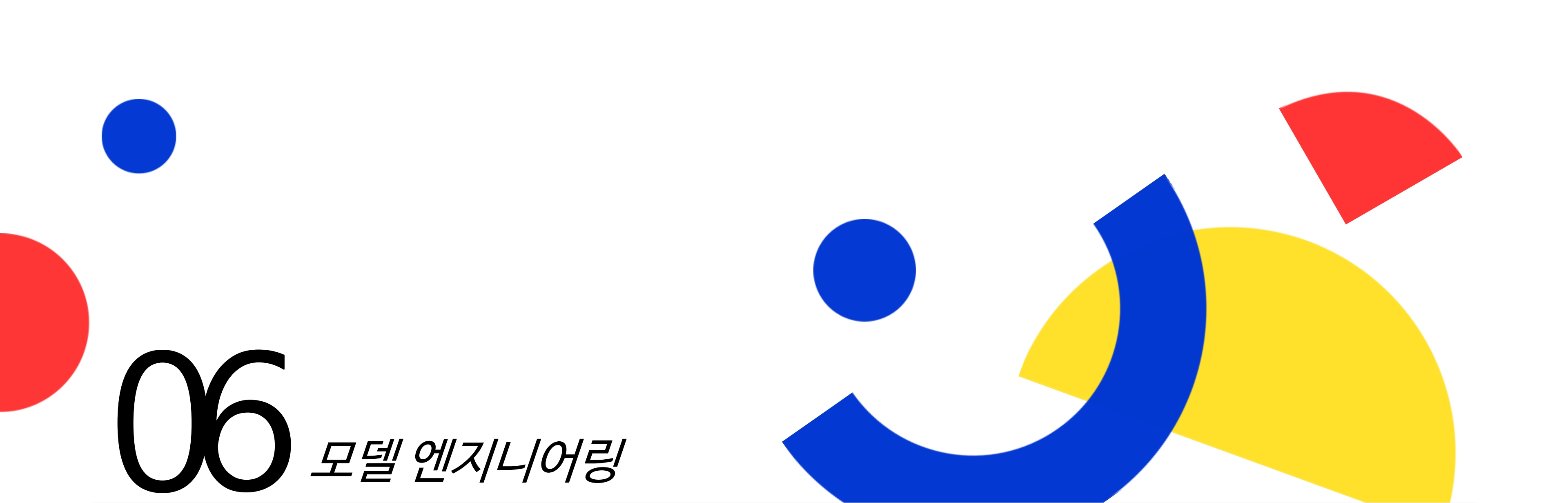
모든 조합에 대해 최종 성능을 비교하여 최적 조합을 도출

✓ 통계기반 파생변수 생성 방식에 따른 일반화 성능 측정 결과



• 파생변수 조합별 성능 분석 결과

- 훈련데이터에만 존재하는 변수에 대한 통계기반 단일 파생변수 추가 시 도로형태1 기반 고위험 사고유형-세부분류 발생 확률 변수가 가장 우수한 성능을 보였다.
- 해당 변수를 기본 파생변수로 채택한 후 추가 조합을 검증한 결과, 도로형태2 기반 피해운전자 성별 평균연령 파생변수를 함께 포함한 모델이 가장 높은 성능을 기록하였다.
- 최종적으로, 도로형태1 기반 고위험 세부분류 발생확률 + 도로형태2 기반 피해운전자 평균연령 파생변수 조합이 최적 성능을 보였으며, 이는 기존 모델 대비 우수한 일반화 성능을 나타냈다.
- 따라서, 해당 파생변수 조합을 기본 모델링 방식으로 채택하고, 이를 기반으로 한 피처 엔지니어링 결과를 최종 변수 생성 방식으로 활용하여 최종 모델링을 수행한다.



06

모델 엔지니어링

- 모델 엔지니어링 개요 및 프로세스
- 하이퍼파라미터 최적화(HPO) 설계
- 일반화 성능 측정 및 최종 모델 선정

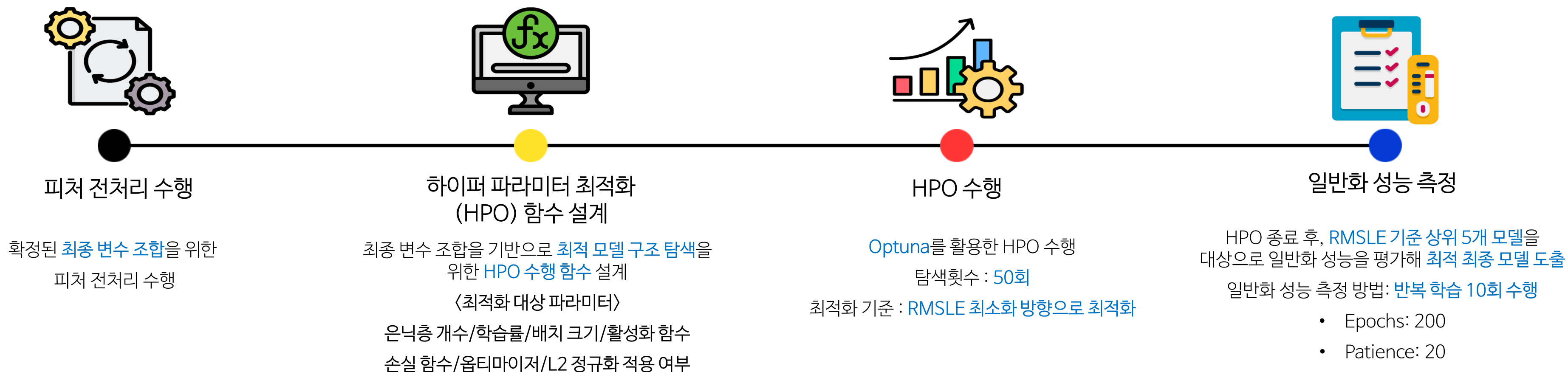
❖ 모델 엔지니어링 개요

- 피처 엔지니어링 과정을 통해 최종 변수 조합을 확정
- 확정된 변수를 활용하여 모델 성능 극대화를 목표로 다양한 조합의 하이퍼파라미터 최적화 수행
- 다양한 모델 구조 및 파라미터 탐색을 통해 최종 최적 모델 구성 도출

❖ 최종 변수 조합 목록

- | | | |
|------------------------|---------------------|-----------------------------------|
| • 월/일/시간 변수 Sin/Cos 변환 | • 시간대 구분 | • 도로형태1 기반 고위험 사고유형-세부분류 발생 확률 |
| • 연도 Log 변환 | • 도로형태1 범주 결합 | • 도로형태2 기반 고위험 피해운전자 성별 발생 확률 |
| • 주말/공휴일 특성교차(3TYPE) | • 구별 면적 및 CCTV 설치밀도 | • 동별 위험도 구분 군집 • 종속변수 제공근 변환 |

❖ 모델 엔지니어링 프로세스



Hyper Parameter Optimization Function Design

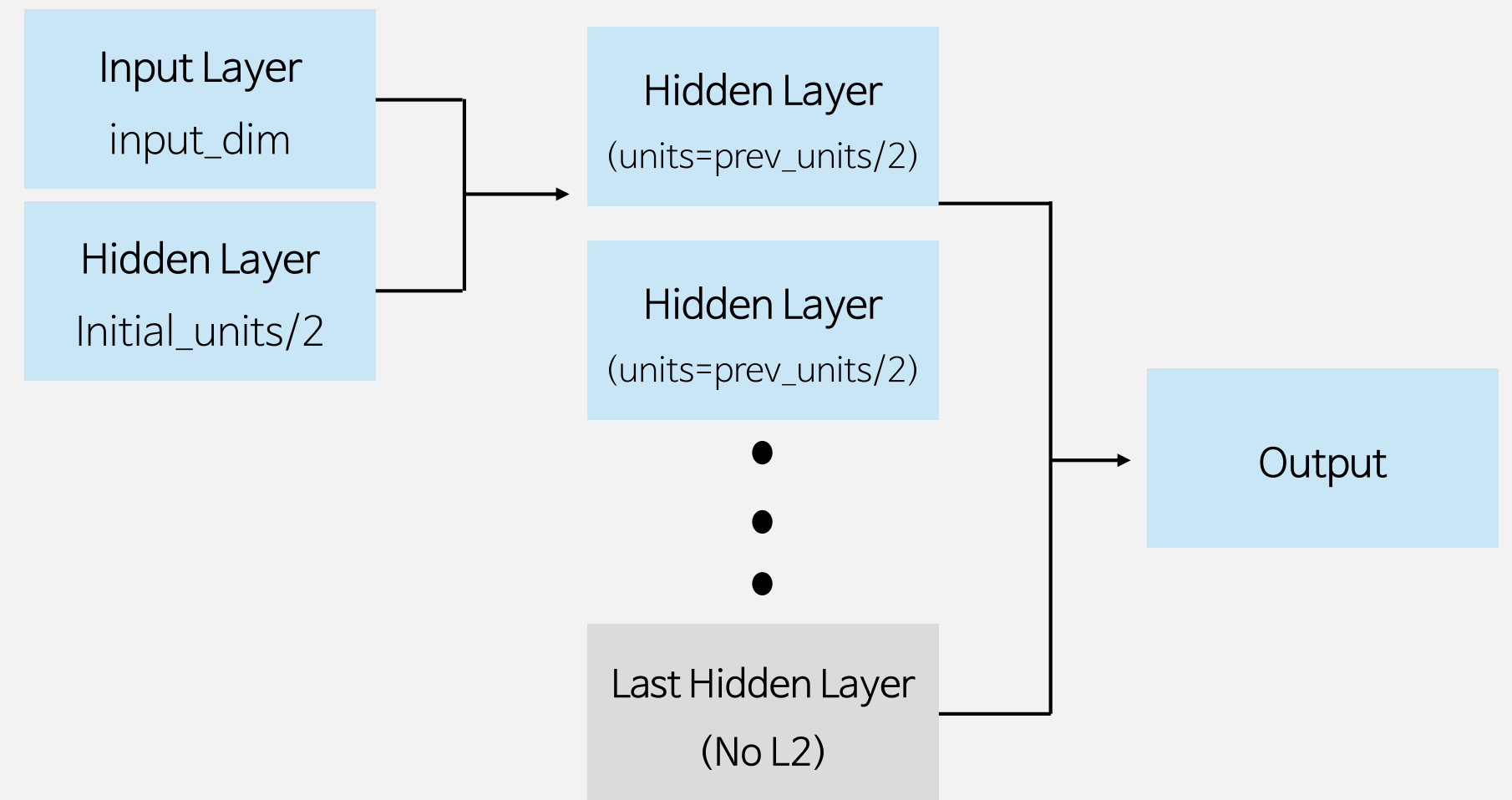
Parameter Search using Optuna Trial

Parameter	Range
N_layers	3 - 7
Initial_units	64, 128, 256, 512
Batch_size	32, 64, 128
Learning_rate	1e-4 to 1e-2
Activation	relu, softplus, elu
Loss_func	MSE, MAE, RMSE, RMSLE, Huber
Optimizer	Adam, RMSprop, Nadam
Use_L2	True, False
L2_Lambda	1e-3 to 1e-1

Key Strategies

- Epochs = 200, Early stopping - Patience = 20
- MSE, RMSE, RMSLE, R^2 및 모든 사용 파라미터 기록
- RMSLE 기준 최적화 수행

Model Construction Logic



Optimization Goal : RMSLE 최소화

하지만, 모델의 전반적인 성능을 알기 위해서 다른 성능 지표도 함께 기록

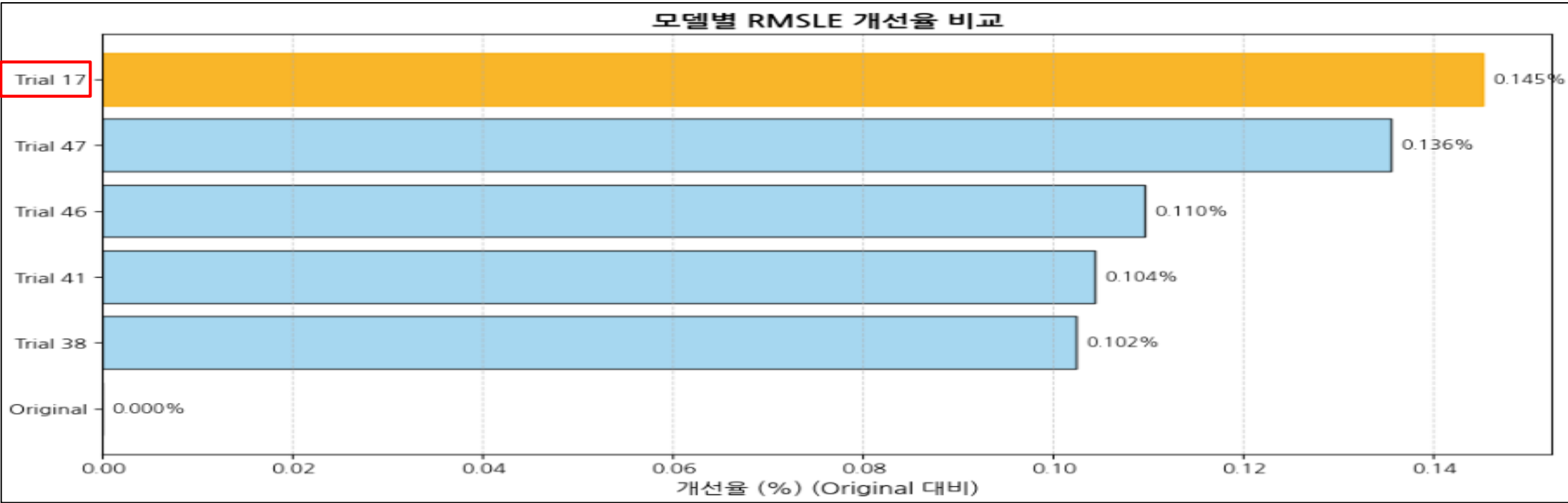
❖ 하이퍼 파라미터 최적화(HPO) 수행 결과

- RMSLE 기준 상위 성능 5개 모델

Trial_Num	Layers	Activation	Optimizer	Loss_Func	Batch_Size	LR	L2_Lambda	MSE	RMSE	RMSLE	R2
17	[256, 128, 64, 32]	relu	adam	rmsle	64	0.000204	0.001043	9.371663	3.061317	0.424579	-0.005015
47	[128, 64, 32, 16]	relu	adam	rmsle	128	0.001195	0.001886	9.338837	3.055951	0.424925	-0.001495
46	[128, 64, 32, 16]	relu	adam	rmsle	64	0.000281	0.001872	9.371971	3.061368	0.425000	-0.005048
41	[128, 64, 32, 16]	elu	adam	rmsle	64	0.000367	0.003674	9.353382	3.058330	0.425007	-0.003055
38	[128, 64, 32, 16]	elu	adam	rmsle	64	0.000536	0.007229	9.362782	3.059866	0.425019	-0.004063

❖ 상위 성능 모델에 대한 일반화 성능 측정 결과

- 기존 모델(Original) 대비 성능 개선율 비교 시각화 → 개선율 계산 방식: $\frac{Original\ RMSLE - Model\ RMSLE}{Original\ RMSLE} \times 100$



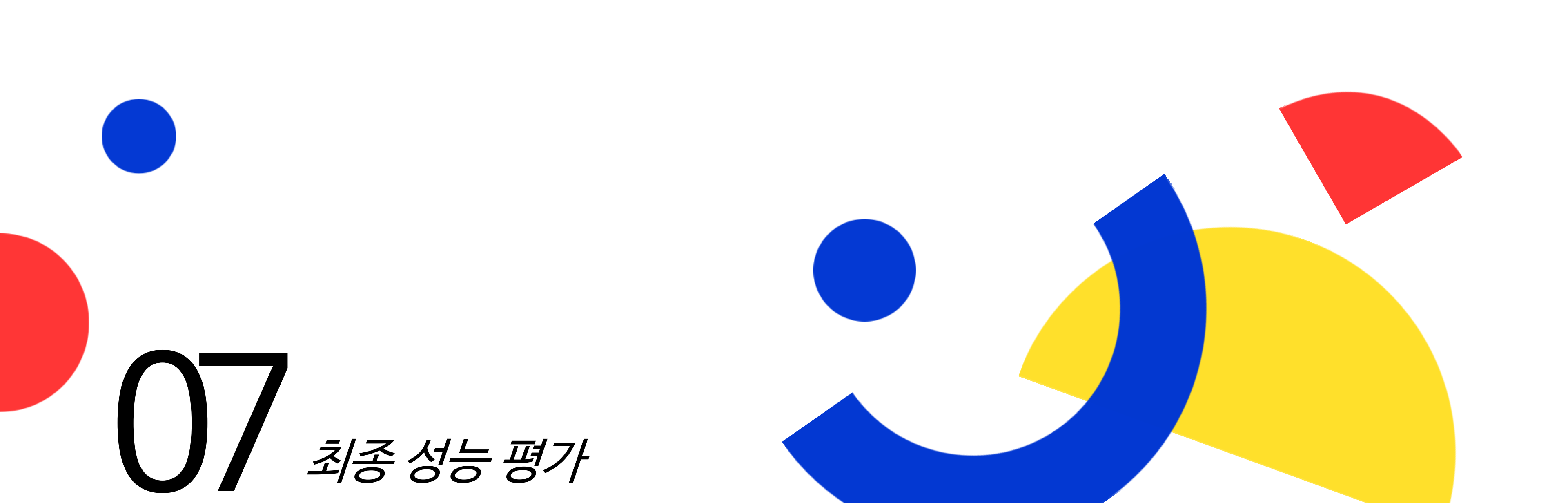
Model	RMSLE	Improvement(%)
Trial 17	0.424946	0.145202
Trial 47	0.424987	0.135525
Trial 46	0.425098	0.109585
Trial 41	0.425120	0.104417
Trial 38	0.425128	0.102339

[모델 성능 분석]

- ✓ 80회 탐색 결과, Trial 17 모델이 RMSLE 성능 최적
- ✓ 은닉층은 128~256 유닛 기반의 중간 규모 구조에서 안정적 성능 확인
- ✓ 해당 결과는 단일 측정 기준이므로, 상위 5개 모델을 대상으로 반복 실험(5회)을 통해 일반화 성능을 검증 후 최종 최적 모델을 도출

★ 『일반화 성능 측정 결과를 통한 최종 모델 선정』

- ✓ RMSLE 기준 상위 5개 모델의 일반화 성능을 검증한 결과, HPO 수행 결과와 동일하게 Trial 17 모델이 최적 성능을 보였다.
- ✓ Trial 17 모델은 기존 모델(Original) 대비 가장 높은 개선율과 최적의 RMSLE 성능을 기록하였다.
- ✓ 다만, RMSLE 성능 향상과 동시에 MSE, RMSE, R² 지표는 성능이 저하되는 경향이 확인되었는데, 이는 손실 함수로 RMSLE를 사용함에 따라 모델이 해당 지표 최적화에 집중했기 때문으로 보인다.
- ✓ 이에 따라, Trial 17 모델을 최종 최적 모델로 선정하고, 대회 기준 테스트 데이터에 적용하여 기존 모델 대비 성능 개선 폭을 검증한다.



07

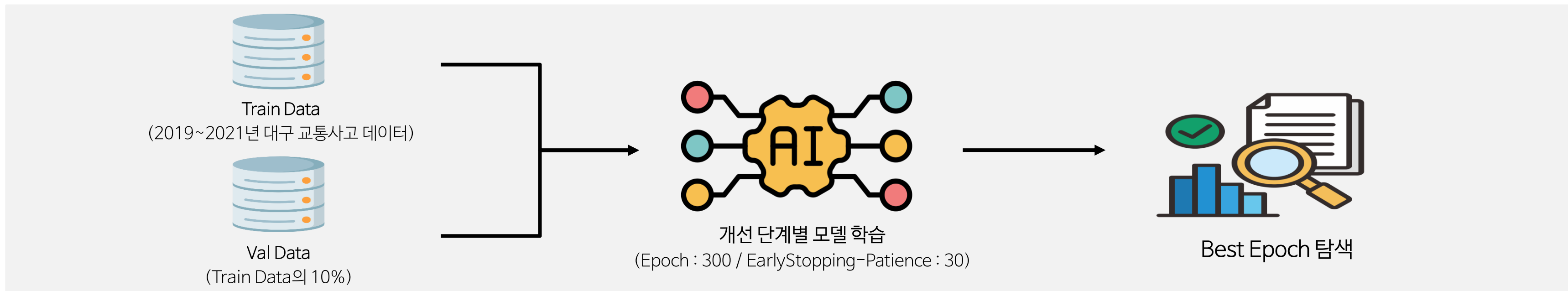
최종 성능 평가

- 공식 평가 점수 측정 방식

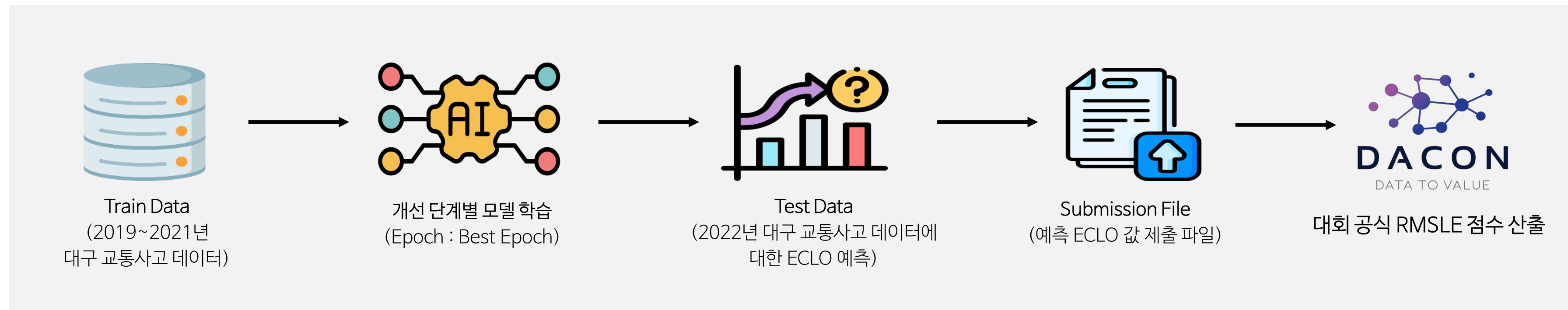
- 모델 개선 단계별 성능 비교

❖ 공식 평가 점수(RMSLE) 측정 방식

1. Best Epoch 탐색 → 검증 데이터를 활용한 조기 종료(EarlyStopping) 기법을 적용하여 Best Epoch를 산출하고, 이를 통해 과적합을 방지하면서 최적의 학습 횟수를 탐색



2. 전체 훈련 데이터 활용 공식 평가 점수(RMSLE) 산출 → 검증 데이터로 탐색한 Best Epoch 기준으로 전체 훈련 데이터를 재학습 후 예측값을 DACON에 제출하여 공식 RMSLE 점수를 산출



모델 개선 단계별 성능 비교

각 단계별 예측 결과를 DACON에 제출하여 산출된 공식 평가 점수(RMSLE)를 비교

Baseline Model → Feature Engineering Model → Final Model 순으로 평가를 수행하여, Baseline Model 대비 최종 모델의 성능 개선율을 평가

ANN Baseline Model

기본 전처리 + 단순 구조 모델

Dense(64) → Dense(32) → Dense(1)

Loss : RMSE

Activation : RELU

Learning Rate : 0.001

Batch Size : 32

RMSLE : 0.4367

ANN Feature Engineering Model

피처 변환 및 파생변수 생성 + 과적합 완화 모델

Dense(64) → Dense(32) → Dense(1)

Loss : Huber (delta=1.0)

Activation : RELU

Learning Rate : 0.001

Batch Size : 32

종속변수 제공근 변환

L2 정규화 ($\lambda=0.001$)

RMSLE : 0.4285

ANN Final Model

하이퍼파라미터 최적화(HPO)를 통한 모델
엔지니어링 후 최종 모델

Dense(256) → Dense(128) → Dense(64)
→ Dense(32) → Dense(1)

Loss : RMSLE

Activation : RELU

Learning Rate : 0.000204

Batch Size : 64

L2 정규화 ($\lambda=0.0010432$)

RMSLE : 0.4274



Baseline Model 대비 최종 모델의 성능은 약 2.13% (0.4367 → 0.4274)의 개선 효과를 확인



08

결론 및 향후 개선 방향

- 최종 프로젝트 결론
- 향후 개선 방향
- 프로젝트 회고

📌 최종 프로젝트 결론

본 프로젝트는 2019~2021년 대구시 교통사고 데이터를 활용하여 ECLO(피해 심각도) 예측 모델을 개발하였다.

시간·공간적 요인(연·월·일·요일·시간, 구·동 단위 공간 변수), 환경적 요인(기상상태, 노면상태, 도로형태), 외부 요인(CCTV 설치 현황, 보안등 개수)등을 다양한 성능 비교 최적화 실험을 통해 최적 인코딩 및 스케일링 전략으로 변환하고, 피처 변환 및 파생변수 생성과 같은 피처 엔지니어링 과정을 통해 사고 위험 요인을 구조적으로 반영하였다.

또한, 성별, 연령, 차종 등 훈련 데이터에만 존재하는 변수를 활용하여 통계기반 파생변수를 도출함으로써 모델 성능 향상에 기여하였다.

실험 결과, 베이스라인 모델(RMSLE: 0.4367) 대비,

- 피처 엔지니어링 적용 후 모델은 0.4285,
- 하이퍼파라미터 최적화(HPO)를 통한 모델 엔지니어링 적용 후 최종 모델은 0.4274의 RMSLE를 기록하며, 최종적으로 약 2.13%의 성능 개선을 달성하였다.

성능 향상의 주요 원인은 피처 엔지니어링 단계에서의 변수 설계였으며, 이는 현황 분석과 EDA를 기반으로 교통사고 데이터의 특수성을 효과적으로 반영한 결과임을 보여준다.

📌 향후 개선 방향

◆ 데이터 확장 및 정규화

- 기상청 실시간 강수량, 교통량, 도로 구조적 특성 등 외부 연계 데이터 추가를 통해 예측력 강화 필요
- 구·동 단위의 면적당 사고 발생률, 차량 등록 대수 대비 사고율 등 정규화된 지표 도입 필요
- 전국 교통사고 데이터를 포함한 대용량 데이터 실험을 통해, 대구 지역 모델의 예측 성능 개선 여부 검증 필요

◆ 고급 모델링 기법 적용

- 딥러닝 기반 Embedding 레이어 활용으로 대규모 범주형 변수('동', '차종' 등)의 효과적 학습 적용
- 앙상블(Ensemble) 및 AutoML 기법 적용을 통한 성능 극대화 검토

📌 프로젝트 회고

◆ 아쉬웠던 점

- 동별 면적 및 교통량과 같은 세부 공간 데이터 부족으로 인해 보다 정밀한 공간 기반 모델링이 제한적이었다.
- 노트북 환경에서 수행하다 보니 연산 성능 제약으로 인해 전국 단위 교통사고 데이터와 대구 교통사고 데이터를 함께 활용한 대규모 학습 및 성능 검증을 진행하지 못한 점이 아쉬웠다.

◆ 좋았던 점

- 다양한 피처 전처리 및 인코딩 기법, 스케일링 전략을 실험하며 데이터 처리 능력을 향상시킬 수 있었다.
- Optuna 기반 하이퍼파라미터 최적화, 딥러닝 모델 구조 설계 등 모델 최적화 기술을 습득하며 실전적인 경험을 쌓을 수 있었다.
- 결과적으로, 교통사고 데이터의 특성을 반영한 피처 엔지니어링이 모델 성능 향상에 결정적 역할을 한다는 점을 체감할 수 있었다.

Thank you :)